

친절한 SQL 튜닝 전범위 25문항

제작 기준

- 원천: **친절한 SQL 튜닝.pdf** 560페이지 전체 이미지 추출 및 한국어 OCR
- 범위: 1장 SQL 처리 과정과 I/O부터 7장 SQL 옵티마이저, 부록 SQL 분석 도구까지
- 출제 방식: 단순 개념 암기보다 **쿼리, 실행계획, I/O 통계, 인덱스 구성, 조인 메커니즘**을 해석하는 문제 중심
- 저작권 처리: OCR 원문 문장을 그대로 옮기지 않고 학습용 사례와 문항으로 재구성

커버리지

범위	핵심 내용	문항
1장 SQL 처리 과정과 I/O	파싱/최적화, 바인드 변수, 논리/물리 I/O, Full SCAN과 INDEX SCAN	1~3
2장 인덱스 기본	B-tree 탐색, RANGE SCAN 조건, 인덱스 확장 스캔	4~6
3장 인덱스 튜닝	테이블 랜덤 액세스, 부분범위 처리, 스캔 효율화, 옵션 조건, 인덱스 설계	7~12
4장 조인 튜닝	NL, Sort MERGE, HASH, 서브쿼리/뷰/스칼라 서브쿼리	13~16
5장 소트 튜닝	소트 수행, UNION/EXISTS, 인덱스 정렬 생략, Top N, Sort Area	17~19
6장 DML 튜닝	Array Processing, Direct Path I/O, 파티션 DML, Lock/채번	20~23
7장/부록	통계정보, 옵티마이저, AutoTrace, SQL Trace, DBMS_XPLAN, V\$SQL	24~25

문제 1. 하드 파싱과 바인드 변수

다음 애플리케이션 코드가 이벤트 시간대에 초당 수천 번 실행된다.

```
String sql =  
    "SELECT 고객ID, 고객명, 등급코드 " +  
    "FROM 고객 " +  
    "WHERE 로그인ID = '" + loginId + "'";  
Statement st = conn.createStatement();  
ResultSet rs = st.executeQuery(sql);
```

가장 적절한 튜닝 방향은?

- ① SQL 문자열을 매번 새로 만들지 말고 바인드 변수를 사용해 커서 공유 가능성을 높인다.
- ② 로그인ID 컬럼에 인덱스를 추가하면 하드 파싱 문제도 함께 사라진다.
- ③ **select-list** 컬럼 수를 줄이면 SQL 텍스트가 같아져 라이브러리 캐시 경합이 사라진다.
- ④ 동일 SQL이므로 대소문자와 공백 차이는 커서 공유에 영향을 주지 않는다.

문제 2. SQL Trace의 I/O 통계 해석

어떤 SQL의 Trace 통계가 다음과 같다.

Call	Count	Disk	Query	Current	Rows
Parse	1	0	0	0	0
Execute	1	0	5	10	1
Fetch	101	12000	480000	200	1000

이 SQL의 튜닝 방향을 판단한 설명으로 가장 적절한 것은?

- ① 물리 I/O는 12,000이고 논리 I/O는 480,215이므로, 우선 읽는 총 블록 수를 줄이는 방향을 검토한다.
- ② Disk보다 Query가 훨씬 크므로 캐시 효율은 양호하며, 인덱스나 조인 구조보다 메모리 증설을 먼저 검토한다.
- ③ Rows가 1,000건이면 Fetch Call이 101회인 점이 핵심이므로, 논리 I/O가 480,215여도 정상 범위로 본다.
- ④ Parse Count가 1이면 커서 공유는 성공한 상태이므로, 실행계획과 I/O 효율 문제는 별도로 분석하지 않는다.

문제 3. Full SCAN과 INDEX RANGE SCAN 선택

주문 테이블은 1억 건이고, 하루 평균 300만 건의 주문이 발생한다. 아래 SQL은 일 배치에서 하루치 주문을 모두 집계한다.

```
SELECT 주문일자, SUM(주문금액)
FROM 주문
WHERE 주문일자 = :target_date
GROUP BY 주문일자;
```

주문일자 인덱스가 있을 때 가장 적절한 판단은?

- ① 인덱스가 있으므로 INDEX RANGE SCAN을 우선 고정하고, 필요하면 힌트로 Full SCAN과 HASH 집계를 함께 방지한다.
- ② Full SCAN이 보이면 인덱스 통계가 누락된 신호이므로, 통계 수집 후에도 INDEX SCAN을 유도하고 ROWID 반복 비용은 무시한다.
- ③ 처리 대상이 300만 건 수준이면 ROWID 기반 테이블 랜덤 액세스가 커질 수 있어 Full SCAN 또는 파티션 Pruning을 검토한다.
- ④ GROUP BY가 있으면 정렬 또는 해시 집계가 필요하므로, 인덱스와 파티션 전략은 성능 판단에서 제외한다.

문제 4. B-tree 인덱스와 RANGE SCAN 가능 조건

다음 인덱스가 있다.

```
CREATE INDEX 고객_X1 ON 고객(생년월일);
```

아래 조건 중 일반적인 B-tree 인덱스 RANGE SCAN의 시작/종료 지점을 가장 명확히 찾을 수 있는 것은?

- ① WHERE SUBSTR(생년월일, 5, 2) = '05'처럼 월만 추출하면 찾을 값이 명확하므로 시작 지점도 명확하다.
- ② WHERE 생년월일 BETWEEN '20070101' AND '20070131'
- ③ WHERE 생년월일 LIKE '%0505%'처럼 포함 검색을 쓰면 원하는 값만 좁게 모여 Range Scan된다.
- ④ WHERE NVL(생년월일, '99991231') >= '20070101'처럼 NULL을 치환하면 원본 인덱스 순서가 그대로 유지된다.

문제 5. OR, IN 조건과 실행계획

다음 SQL과 실행계획을 보자. **고객_전화번호_IDX(전화번호)** 인덱스가 있다.

```
SELECT *  
FROM 고객  
WHERE 전화번호 IN (:tel1, :tel2, :tel3);
```

Id	Operation	Name
0	SELECT STATEMENT	
1	INLIST ITERATOR	
2	TABLE ACCESS BY INDEX ROWID	고객
3	INDEX RANGE SCAN	고객_전화번호_IDX

가장 적절한 설명은?

- ① IN 조건은 OR 조건의 축약 표현이므로, 여러 값이 있으면 옵티마이저는 항상 Full SCAN을 선택한다.
- ② 전화번호 인덱스가 유니크가 아니면 중복 후보가 생기므로, IN 조건에는 인덱스를 사용할 수 없다.
- ③ INLIST ITERATOR는 인덱스 탐색을 포기하고, IN-list 값을 메모리에서만 비교한다는 의미다.
- ④ IN-list 값마다 인덱스 RANGE SCAN을 반복하는 방식으로 처리될 수 있다.

문제 6. 인덱스 확장 스캔 방식

다음 SQL은 인덱스에 포함된 컬럼만 조회한다.

```
CREATE INDEX 주문_X1 ON 주문(고객ID, 주문일자, 주문금액);  
  
SELECT /*+ INDEX_FFS(o 주문_X1) */  
      고객ID, 주문일자, 주문금액  
FROM   주문 o  
WHERE  주문일자 BETWEEN :d1 AND :d2;
```

INDEX FAST FULL SCAN이 선택되었을 때 옳은 설명은?

- ① 인덱스의 논리적 리프 연결 순서대로 읽으므로 **고객ID, 주문일자** 정렬 결과가 보장된다.
- ② 인덱스 세그먼트를 테이블처럼 읽을 수 있지만, 인덱스 키 순서 정렬은 보장하지 않는다.
- ③ 반드시 테이블을 ROWID로 방문해야 하므로 인덱스만 읽고 처리할 수 없다.
- ④ Single Block I/O만 사용하므로 대량 스캔에 불리하다.

문제 7. 테이블 랜덤 액세스와 클러스터링 팩터

다음 두 인덱스 중 하나를 선택해야 한다.

주문_X1(고객ID)
- 예상 인덱스 스캔 건수: 10,000
- 클러스터링 팩터: 매우 나쁨

주문_X2(주문일자)
- 예상 인덱스 스캔 건수: 40,000
- 클러스터링 팩터: 매우 좋음

조회 SQL은 테이블 컬럼을 여러 개 읽어야 해서 **TABLE ACCESS BY INDEX ROWID**가 필요하다. 가장 적절한 판단은?

- ① 스캔 건수가 적은 **주문_X1**이 항상 빠르므로, 클러스터링 팩터보다 선택도만 우선 비교한다.
- ② 클러스터링 팩터는 Full SCAN에서 멀티블록 I/O 효율을 판단하는 지표이므로 인덱스 선택과 테이블 랜덤 액세스와는 무관하다.
- ③ 클러스터링 팩터가 나쁜 인덱스는 같은 테이블 블록을 반복 방문할 수 있으므로 테이블 액세스 비용까지 비교해야 한다.
- ④ 인덱스를 타면 ROWID가 물리 포인터처럼 바로 레코드를 가리키므로 테이블 액세스 비용은 거의 없다고 본다.

문제 8. 부분범위 처리와 화면 응답 속도

상담 화면에서 특정 고객의 최근 거래 30건만 먼저 보여주려고 한다.

```
SELECT *  
FROM (  
  SELECT 거래번호, 고객ID, 거래일시, 거래금액  
  FROM 거래  
  WHERE 고객ID = :cust_id  
  ORDER BY 거래일시 DESC  
)  
WHERE ROWNUM <= 30;
```

가장 적절한 인덱스는?

- ① 거래_X1(거래금액, 거래일시)
- ② 거래_X2(거래일시, 고객ID)
- ③ 거래_X3(고객ID, 거래일시 DESC)
- ④ 거래_X4(고객ID, 거래금액, 거래일시)

문제 9. 액세스 조건과 필터 조건

다음 인덱스가 있다.

```
CREATE INDEX 가입상품_X1 ON 가입상품(회사코드, 지역코드, 상품명);
```

아래 SQL에서 **지역코드** 값을 항상 입력한다고 가정한다.

```
SELECT 고객ID, 상품명  
FROM 가입상품  
WHERE 회사코드 = :company  
AND 지역코드 LIKE :region || '%'  
AND 상품명 LIKE :product || '%';
```

지역코드 = :region으로 바꾸었을 때 기대할 수 있는 가장 중요한 효과는?

- ① **지역코드**가 테이블 필터 조건으로만 바뀐다.
- ② LIKE와 등치 조건은 B-tree 인덱스에서 항상 같은 스캔 범위를 만든다.
- ③ 인덱스 선두 컬럼인 **회사코드**를 더 이상 사용할 수 없다.
- ④ 뒤 컬럼인 **상품명**까지 인덱스 액세스 범위 축소에 더 잘 활용될 수 있다.

문제 10. BETWEEN을 IN-list로 바꾸는 경우

인덱스가 다음과 같다.

```
CREATE INDEX 일별거래_X1 ON 일별거래(거래일자, 시장구분, 종목코드);
```

아래 SQL은 특정 3일의 특정 시장, 특정 종목을 조회한다.

```
SELECT *  
FROM 일별거래  
WHERE 거래일자 BETWEEN :d1 AND :d3  
AND 시장구분 = :market  
AND 종목코드 = :item;
```

:d1, :d2, :d3가 정확히 3개 날짜일 때 다음처럼 바꾸는 튜닝이 효과적일 수 있는 이유는?

```
WHERE 거래일자 IN (:d1, :d2, :d3)  
AND 시장구분 = :market  
AND 종목코드 = :item
```

- ① 거래일자 조건이 사라지는 효과가 있어 옵티마이저가 테이블 Full SCAN을 더 쉽게 선택하기 때문이다.
- ② 각 날짜별로 **시장구분, 종목코드**까지 더 좁은 인덱스 탐색 범위를 만들 수 있기 때문이다.
- ③ IN-list는 선두 컬럼을 여러 값으로 나누므로 후행 컬럼을 모두 필터 조건으로 강등시키기 때문이다.
- ④ BETWEEN은 하나의 연속 구간만 읽으므로 값 개수와 후행 조건에 관계없이 항상 더 적은 블록을 읽는다.

문제 11. 옵션 조건 처리 방식

검색 화면에서 **고객ID**는 선택 입력이고 **거래일자**는 필수 입력이다. 다음 SQL을 하나로 통합하려고 한다.

```
SELECT *  
FROM 거래  
WHERE (:cust_id IS NULL OR 고객ID = :cust_id)  
AND 거래일자 BETWEEN :d1 AND :d2;
```

인덱스 후보는 **거래_X1(고객ID, 거래일자)**와 **거래_X2(거래일자, 고객ID)**이다. 가장 적절한 설명은?

- ① 고객ID가 선택 입력이면 SQL을 분기하거나 UNION ALL로 나누어 각 경우에 맞는 액세스 경로를 만드는 방식을 검토한다.
- ② OR 옵션 조건은 **고객ID**가 입력된 경우에만 평가되므로, **거래_X1**의 선두 컬럼 액세스 조건으로 안정적으로 작동한다.
- ③ **거래_X1**만 있으면 고객ID를 입력하지 않은 경우에도 거래일자 조건이 후행 컬럼에서 충분히 좁은 범위를 만들고 RANGE SCAN이 안정적이다.
- ④ 옵션 조건은 실행 시점에 값이 결정되므로 인덱스 컬럼 순서보다 바인드 변수 타입을 우선 조정해야 한다.

문제 12. 인덱스 설계와 중복 제거

다음 인덱스들이 있다.

```
고객_X1(고객ID)  
고객_X2(고객ID, 가입일자)  
고객_X3(고객ID, 가입일자, 상태코드)  
고객_X4(가입일자, 상태코드)
```

주요 SQL은 **고객ID = :id** 단건 조회, **고객ID = :id AND 가입일자 BETWEEN ...** 조회, **가입일자 BETWEEN ... AND 상태코드 = ...** 집계다. 가장 적절한 판단은?

- ① **고객_X1**, **고객_X2**, **고객_X3**는 선두 컬럼이 같아도 길이가 다르므로 옵티마이저가 항상 서로 다른 용도로 쓰며 중복으로 보지 않는다.
- ② **고객_X4**는 **고객ID**가 없으므로 고객 테이블의 주요 SQL과 무관하며, 가입일자 집계에도 도움이 되지 않는 중복 인덱스다.
- ③ 인덱스가 많을수록 조회 후보가 늘어나므로 DML 유지 비용보다 옵티마이저 선택 폭 확대를 우선한다.
- ④ **고객_X3**가 **고객_X1**, **고객_X2** 역할을 상당 부분 대체할 수 있는지 검토하고, DML 부하까지 고려해 중복 제거를 판단한다.

문제 13. NL 조인 실행계획 해석

다음 실행계획을 보자.

Id	Operation	Name	Starts	A-Rows	Buffers
0	SELECT STATEMENT		1	500	82000
1	NESTED LOOPS		1	500	82000
2	TABLE ACCESS BY INDEX ROWID	주문	1	10000	12000
3	INDEX RANGE SCAN	주문_X1	1	10000	2000
4	TABLE ACCESS BY INDEX ROWID	주문상세	10000	500	70000
5	INDEX RANGE SCAN	주문상세_X1	10000	500	40000

가장 적절한 분석은?

- ① **주문상세_X1**의 Starts가 10,000이므로 선행 **주문** 결과마다 후행 인덱스 탐색이 반복되었다.
- ② **주문상세**의 A-Rows가 500이므로 최종 반환 건수는 작고, 후행 인덱스 반복 비용도 작다고 판단한다.
- ③ NL 조인은 인덱스 기반 조인이므로 Buffers보다 A-Rows를 우선 보며, 버퍼 읽기는 대체로 무시할 수 있다.
- ④ **주문**의 A-Rows가 10,000이어도 후행 인덱스가 있으면 랜덤 액세스가 순차 I/O로 바뀌어 병목이 사라진다.

문제 14. Sort MERGE JOIN 선택 상황

두 테이블을 조인하는 SQL에서 양쪽 입력 집합이 이미 조인 컬럼 순서로 정렬되어 있거나, 각각 인덱스 Full SCAN으로 정렬 순서를 얻을 수 있다. 조인 결과는 대량이며 후행 랜덤 액세스를 반복하는 NL 조인은 부담스럽다.

가장 적절한 판단은?

- ① Sort MERGE JOIN은 이름에 Sort가 포함되어 있으므로 인덱스 정렬 순서가 있어도 항상 새 정렬을 수행한다.
- ② Sort MERGE JOIN은 조인 전에 양쪽을 정렬하지만, 정렬은 디스크가 담당하므로 PGA 사용량과 무관하다.
- ③ 정렬 부담이 작거나 이미 정렬된 입력을 활용할 수 있다면 Sort MERGE JOIN이 합리적일 수 있다.
- ④ Sort MERGE JOIN은 조인 조건이 약하거나 결과가 대량일수록 항상 HASH JOIN보다 안정적인 선택이다.

문제 15. HASH JOIN과 Build Input

다음 SQL은 대량 집합을 조인한다.

```
SELECT /*+ USE_HASH(o) */  
      c.고객등급, SUM(o.주문금액)  
FROM   고객 c  
      JOIN 주문 o ON o.고객ID = c.고객ID  
WHERE  o.주문일자 BETWEEN :d1 AND :d2  
GROUP BY c.고객등급;
```

고객은 50만 건, 기간 조건을 만족하는 주문은 2천만 건이다. HASH JOIN에 대한 설명으로 가장 적절한 것은?

- ① Build Input은 SQL에 나중에 적은 테이블이 되므로, FROM 절 순서만 조정해도 메모리 사용량을 예측할 수 있다.
- ② 일반적으로 더 작은 입력인 **고객**을 Build Input으로 삼아 해시 테이블을 만드는 것이 유리할 수 있다.
- ③ HASH JOIN은 인덱스 탐색을 반복하지 않기 때문에 대량 처리에서는 항상 NL 조인보다 빠르다고 단정할 수 있다.
- ④ HASH JOIN은 해시 테이블을 만들지만 Temp로 넘기지 않으므로, 메모리 크기보다 조인 컬럼 NDV가 중요하다.

문제 16. 서브쿼리 변환과 스칼라 서브쿼리

다음 SQL을 보자.

```
SELECT c.고객ID, c.고객명,  
      (SELECT MAX(o.주문일자)  
       FROM 주문 o  
       WHERE o.고객ID = c.고객ID) AS 최근주문일자  
FROM   고객 c  
WHERE  c.상태코드 = 'A';
```

가장 적절한 설명은?

- ① 스칼라 서브쿼리는 바깥쪽 고객 건수만큼 실행되므로, 주문 테이블 Full SCAN이 반복된다고 보는 것이 안전하다.
- ② 옵티마이저는 상황에 따라 서브쿼리를 조인 형태로 변환하거나, 인덱스와 캐싱 효과를 활용해 처리할 수 있다.
- ③ 서브쿼리는 SELECT 절에 있으면 실행계획에서 조인으로 변환되지 않고 항상 별도 필터 단계로만 남는다.
- ④ 스칼라 서브쿼리는 여러 건을 반환해도 가장 먼저 찾은 한 건만 쓰므로, MAX 같은 집계 함수는 성능상 불필요하다.

문제 17. UNION, EXISTS, 소트 발생

다음 두 SQL은 같은 업무 요건을 구현하려고 작성한 후보안이다.

```
-- A안
SELECT 고객ID FROM 온라인고객
UNION
SELECT 고객ID FROM 오프라인고객;

-- B안
SELECT 고객ID FROM 온라인고객
UNION ALL
SELECT 고객ID FROM 오프라인고객;
```

두 집합에 중복 고객ID가 없다는 업무 규칙이 보장될 때 가장 적절한 판단은?

- ① 중복이 없더라도 **UNION ALL**은 데이터 품질 검증을 생략하므로, 업무 규칙이 명확해도 사용하지 않는 편이 안전하다.
- ② B안은 두 집합을 단순 연결하므로 중복 제거 비용은 없지만, 중복이 없을 때도 결과 건수는 A안보다 적어진다.
- ③ **UNION**과 **UNION ALL**은 모두 집합 연산이므로 옵티마이저가 중복 여부를 판단해 같은 실행 방식으로 바꾼다.
- ④ A안은 중복 제거가 필요하므로 불필요한 소트 또는 해시 중복 제거가 발생할 수 있다.

문제 18. 인덱스를 이용한 ORDER BY, MIN/MAX 생략

다음 인덱스가 있다.

```
CREATE INDEX 체결_X1 ON 체결(종목코드, 체결일시 DESC, 체결가격);
```

아래 SQL의 기대 실행 방식으로 가장 적절한 것은?

```
SELECT *
FROM (
  SELECT 체결번호, 종목코드, 체결일시, 체결가격
  FROM 체결
  WHERE 종목코드 = :code
  ORDER BY 체결일시 DESC
)
WHERE ROWNUM <= 5;
```

- ① 인덱스가 DESC 컬럼을 포함하면 정렬 전용으로만 쓸 수 있고, 조건 검색을 위한 RANGE SCAN은 불가능하다.
- ② **ORDER BY**가 있으면 인덱스 순서와 무관하게 Sort Order By가 필요하므로 전체 후보를 정렬해야 한다.
- ③ **종목코드** 등치 조건으로 인덱스 범위를 잡고 **체결일시 DESC** 순서대로 읽다가 5건에서 멈출 수 있다.
- ④ **ROWNUM <= 5**는 정렬이 끝난 뒤 적용되는 필터이므로, Stopkey 여부와 인덱스 순서는 성능에 영향을 주지 않는다.

문제 19. Sort Area를 적게 쓰는 SQL 작성

아래 두 SQL 중 Sort Area 사용량을 줄이는 방향으로 더 적절한 것은?

```
-- A안
SELECT LPAD(상품번호, 30) || LPAD(상품명, 100) || TO_CHAR(주문일시, 'yyyymmdd hh24:mi:ss') AS line
FROM 주문상품
WHERE 주문일시 BETWEEN :d1 AND :d2
ORDER BY 상품번호;

-- B안
SELECT LPAD(상품번호, 30) || LPAD(상품명, 100) || TO_CHAR(주문일시, 'yyyymmdd hh24:mi:ss') AS line
FROM (
    SELECT 상품번호, 상품명, 주문일시
    FROM 주문상품
    WHERE 주문일시 BETWEEN :d1 AND :d2
    ORDER BY 상품번호
);
```

Sort Area 사용량을 줄이는 개선안으로 가장 적절한 것은?

- ① B안이 더 적절하다. 정렬은 원본 컬럼 기준으로 수행하고 포매팅은 정렬 이후로 미룰 수 있다.
- ② A안이 더 적절하다. 정렬 전에 문자열을 완성하면 정렬 후 추가 연산이 줄어 전체 메모리 사용량도 감소한다.
- ③ 두 SQL은 같은 행과 같은 컬럼을 최종 출력하므로, 옵티마이저 입장에서는 정렬 대상 데이터 폭도 항상 동일하다.
- ④ Sort Area는 SQL 작성 방식보다 PGA 설정값으로 결정되므로, 표현식 위치를 바꿔도 Temp 사용량은 같다.

문제 20. Array Processing과 Database Call

다음 프로그램은 10만 건을 한 건씩 INSERT한다.

```
for (Row r : rows) {
    PreparedStatement ps = conn.prepareStatement(
        "INSERT INTO 주문_임시(주문번호, 고객ID, 주문금액) VALUES (?, ?, ?)");
    ps.setLong(1, r.orderNo);
    ps.setString(2, r.customerId);
    ps.setBigDecimal(3, r.amount);
    ps.executeUpdate();
}
```

가장 적절한 개선 방향은?

- ① 한 건씩 처리하면 각 로우의 Lock 유지 시간이 짧아지고 실패 지점도 명확해지므로, 대량 INSERT에서도 전체 처리량이 가장 안정적이다.
- ② PreparedStatement를 재사용하고 배치 바인딩/Array Processing으로 Database Call 횟수를 줄인다.
- ③ 인덱스를 충분히 추가하면 INSERT 대상 블록과 중복 여부를 더 빨리 찾을 수 있으므로 Database Call 횟수도 함께 감소한다.
- ④ Commit을 마지막에 한 번만 수행하면 로그 기록과 Lock 관리가 사라지므로, 장애 복구 측면에서도 가장 안전하다.

문제 21. Direct Path Insert

대량 이관 테이블에 야간 배치로 5천만 건을 적재한다. 기존 데이터와 동시 조회/갱신 충돌은 없고, 적재 후 인덱스 생성과 통계 수집을 수행할 수 있다.

가장 적절한 검토 사항은?

- ① Direct Path Insert를 사용하면 통계정보 수집이 필요 없다.
- ② Direct Path Insert는 항상 OLTP 단건 INSERT에도 적합하다.
- ③ Direct Path Insert는 Redo, Undo, Lock, 인덱스 유지 비용을 전혀 발생시키지 않는다.
- ④ **APPEND** 또는 병렬 Direct Path Insert를 검토해 버퍼 캐시 경유 부담을 줄일 수 있다.

문제 22. 파티션과 대량 DML

주문 테이블은 주문일자 기준 RANGE 파티션이고, 과거 1개월 데이터를 삭제해야 한다. 다음 중 가장 적절한 전략은?

- ① 로컬 인덱스는 파티션과 1:1로 대응하므로, 파티션 구조를 변경하면 글로벌 인덱스보다 항상 더 쉽게 Unusable이 된다.
- ② 파티션 테이블이라도 인덱스 정합성을 지키려면 과거 데이터 삭제는 Row 단위 DELETE로 처리해야 한다.
- ③ 월 파티션이 분리되어 있으면 **DELETE**보다 파티션 **TRUNCATE** 또는 **DROP** 방식이 훨씬 빠를 수 있다.
- ④ 파티션 Pruning은 조회 대상 세그먼트를 줄이는 기능이므로 SELECT에는 유효하지만 DELETE/INSERT에는 적용되지 않는다.

문제 23. Lock, 트랜잭션, 채번 방식

다음 방식으로 주문번호를 채번한다.

```
SELECT MAX(주문번호) + 1  
FROM 주문  
WHERE 주문일자 = :today;
```

동시 주문이 매우 많은 시스템에서 이 방식의 문제로 가장 적절한 것은?

- ① 같은 번호를 계산한 트랜잭션들이 충돌할 수 있어 직렬화나 Lock 경합 문제가 생길 수 있다.
- ② **max + 1** 방식은 날짜별 마지막 번호를 직접 확인하므로, 캐시된 시퀀스보다 업무 규칙과 확장성 측면에서 유리하다.
- ③ 주문일자와 주문번호 인덱스가 있으면 최댓값 탐색이 빨라지고, 같은 번호를 계산하는 동시성 문제도 함께 제거된다.
- ④ 채번은 INSERT 전에 수행되는 조회일 뿐이므로, 실제 INSERT 처리량과 Lock 경합에는 직접 영향을 주지 않는다.

문제 24. 선택도, 카디널리티, 통계정보

상품 테이블은 1,000만 건이고 상품분류코드의 NDV는 100이다. 히스토그램이 없고 값 분포가 균일하다고 가정하면 다음 조건의 단순 카디널리티 추정값으로 가장 가까운 것은?

```
WHERE 상품분류코드 = :cls
```

- ① 약 1만 건
- ② 약 1,000만 건
- ③ 약 100만 건
- ④ 약 10만 건

문제 25. SQL 분석 도구 선택

운영 중 특정 SQL이 간헐적으로 느리다. 예상 실행계획만으로는 원인을 알 수 없고, 실제 실행 시 각 오퍼레이션의 **A-Rows**, **Buffers**, **Starts**를 보고 싶다.

가장 적절한 방법은?

- ① **EXPLAIN PLAN**에 표시되는 예상 Rows와 Cost를 확인하면, 실제 실행 시 Row Source 통계도 같은 값으로 해석할 수 있다.
- ② **dbms_xplan.display_cursor**에 **ALLSTATS LAST** 형식으로 실제 커서의 Row Source 통계를 확인한다.
- ③ **V\$SQL**의 누적 **buffer_gets**, **executions**를 나누어 평균 I/O를 구하면 실행계획 단계별 병목까지 식별할 수 있다.
- ④ AutoTrace의 예상 실행계획은 SQL을 실행하지 않고도 통계를 보여주므로, 간헐적 지연 분석에 가장 적합하다.

정답 및 해설

정답 한 번에 보기

문제	정답	문제	정답	문제	정답	문제	정답	문제	정답
1	①	2	①	3	③	4	②	5	④
6	②	7	③	8	③	9	④	10	②
11	①	12	④	13	①	14	③	15	②
16	②	17	④	18	③	19	①	20	②
21	④	22	③	23	①	24	④	25	②

문제 1 해설

정답: ①

핵심 개념

SQL 최적화는 CPU를 많이 쓰는 무거운 작업이며, 하드 파싱 결과는 라이브러리 캐시에 커서로 저장된다. SQL 텍스트가 매번 달라지면 의미가 같아도 커서 공유가 어려워진다.

정답 도출

문자열 결합 방식은 로그인ID 값마다 서로 다른 SQL 텍스트를 만든다. 바인드 변수로 바꾸면 SQL 텍스트가 안정되어 하드 파싱과 라이브러리 캐시 경합을 줄일 수 있다.

```
SELECT 고객ID, 고객명, 등급코드
FROM 고객
WHERE 로그인ID = :login_id;
```

오답 분석

선택지	이유
②	인덱스는 데이터 액세스 문제를 줄일 수 있지만, SQL 텍스트가 달라지는 하드 파싱 문제를 직접 해결하지 못한다.
③	select-list 축소는 전송량과 테이블 액세스에는 영향을 줄 수 있지만 커서 공유의 핵심은 SQL 텍스트와 바인드 사용이다.
④	DBMS는 SQL 텍스트를 기준으로 커서를 찾으므로 대소문자, 공백, 주석, 스키마 표기 차이가 별도 커서를 만들 수 있다.

정리

인덱스 튜닝 전에 SQL 공유와 재사용 구조를 먼저 안정화해야 한다. OLTP 로그인, 조회, 주문처럼 반복 호출되는 SQL은 바인드 변수가 기본이다.

문제 2 해설

정답: ①

핵심 개념

SQL Trace의 **Query + Current**는 논리 I/O, **Disk**는 물리 I/O로 해석한다. SQL 튜닝의 핵심은 통제 가능한 논리 I/O를 줄이는 것이다.

정답 도출

이 SQL의 전체 논리 I/O는 다음과 같다.

Query (5 + 480,000) + Current (10 + 200) = 480,215

물리 I/O는 12,000이다. 물리 I/O가 상대적으로 작아 보여도, 1,000건을 가져오기 위해 48만 블록 이상을 읽는다면 인덱스/조인/소트/필터 조건을 재검토해야 한다.

오답 분석

선택지	이유
②	버퍼 캐시 히트율이 높아도 같은 블록을 비효율적으로 반복 읽으면 SQL은 느릴 수 있다.
③	반환 건수만으로 정상 여부를 판단할 수 없다. 핵심은 결과를 얻기 위해 읽은 블록 수다.
④	Parse Count가 작다는 것은 파싱 횟수 문제와 관련될 뿐, 실행계획이나 I/O 효율을 보장하지 않는다.

정리

BCHR보다 개별 SQL의 논리 I/O가 더 직접적인 튜닝 지표다.

문제 3 해설

정답: ③

핵심 개념

인덱스는 소량 데이터를 빨리 찾는 도구다. 대량 데이터를 인덱스로 찾은 뒤 테이블을 반복 방문하면 Full SCAN보다 느릴 수 있다.

정답 도출

하루치 데이터가 300만 건이면 **주문일자** 인덱스로 300만 ROWID를 얻고 테이블을 랜덤 액세스할 수 있다. 이 경우 대량 순차 처리 관점에서 Full SCAN, 파티션 Pruning, 병렬 처리, 집계용 인덱스 등을 함께 검토한다.

오답 분석

선택지	이유
①	인덱스 존재가 최적 경로를 보장하지 않는다.
②	Full SCAN은 대량 처리에서 정상적이고 유리한 선택일 수 있다.
④	GROUP BY가 있어도 인덱스나 파티션, Full SCAN 중 비용이 낮은 경로를 선택할 수 있다.

정리

Full SCAN이 나쁜 것이 아니라, 처리량과 액세스 패턴에 맞지 않는 경로가 나쁘다.

문제 4 해설

정답: ②

핵심 개념

B-tree RANGE SCAN은 정렬된 인덱스에서 시작 지점과 종료 지점을 찾을 수 있어야 한다. 컬럼을 함수로 가공하거나 중간 문자열을 찾으면 범위를 직접 잡기 어렵다.

정답 도출

생년월일 BETWEEN '20070101' AND '20070131'은 인덱스에 저장된 원본 값 기준으로 범위를 지정한다. 따라서 시작 위치와 종료 위치가 명확하다.

오답 분석

선택지	이유
①	substr 로 컬럼을 가공하면 일반 인덱스의 원본 정렬 구조를 직접 활용하기 어렵다.
③	앞쪽 와일드카드 검색은 값이 인덱스 전체에 흩어져 있어 시작 지점을 잡기 어렵다.
④	nvl 로 컬럼을 가공했으므로 함수 기반 인덱스가 없다면 정상적인 RANGE SCAN이 어렵다.

정리

인덱스 컬럼은 가능한 한 가공하지 말고 원본 컬럼 기준의 범위로 표현한다.

문제 5 해설

정답: ④

핵심 개념

IN 조건은 OR 조건의 다른 표현이지만, 옵티마이저가 IN-list Iterator로 각 값에 대해 RANGE SCAN을 반복하는 방식으로 처리할 수 있다.

정답 도출

실행계획에 **INLIST ITERATOR**와 **INDEX RANGE SCAN**이 함께 나타난다. 이는 **:tel1**, **:tel2**, **:tel3** 각각에 대해 전화번호 인덱스 범위를 탐색하는 구조로 이해할 수 있다.

오답 분석

선택지	이유
①	IN 조건도 적절한 인덱스가 있으면 RANGE SCAN 반복으로 처리될 수 있다.
②	유니크 인덱스가 아니어도 RANGE SCAN은 가능하다.
③	실행계획에 명확히 INDEX RANGE SCAN 이 있다.

정리

OR/IN 조건은 실행계획에서 쿼리 변환, Concatenation, INLIST ITERATOR 여부를 확인해야 한다.

문제 6 해설

정답: ②

핵심 개념

INDEX Fast Full SCAN은 인덱스 트리의 논리 순서를 따라가는 것이 아니라 인덱스 세그먼트를 빠르게 읽는다. 정렬 순서를 보장하지 않는 대신 대량 인덱스 스캔에 유리할 수 있다.

정답 도출

조회 컬럼이 모두 **주문_X1**에 포함되어 있으므로 테이블 방문 없이 인덱스만 읽을 수 있다. 그러나 Fast Full SCAN은 물리적 저장 순서에 가깝게 읽으므로 **ORDER BY** 생략 근거가 되지 않는다.

오답 분석

선택지	이유
①	정렬 순서를 보장하는 쪽은 보통 RANGE SCAN 또는 INDEX Full SCAN 계열이다.
③	필요한 컬럼이 모두 인덱스에 있으면 테이블 액세스를 생략할 수 있다.
④	Fast Full SCAN은 Multiblock I/O와 병렬 처리에 유리할 수 있다.

정리

INDEX FULL SCAN과 INDEX FAST FULL SCAN은 이름이 비슷하지만 정렬 보장과 I/O 방식이 다르다.

문제 7 해설

정답: ③

핵심 개념

인덱스 튜닝은 인덱스 스캔량뿐 아니라 테이블 랜덤 액세스 비용까지 봐야 한다. 클러스터링 팩터가 나쁘면 인덱스 순서와 테이블 저장 순서가 맞지 않아 블록 방문이 늘어난다.

정답 도출

주문_X1은 인덱스 스캔 건수는 적지만 클러스터링 팩터가 매우 나쁘다. 반면 주문_X2는 더 많이 스캔해도 테이블 블록 접근이 연속적일 수 있다. 따라서 실행계획의 Buffers, 테이블 액세스 횟수, 실제 응답 시간을 비교해야 한다.

오답 분석

선택지	이유
①	인덱스 스캔 건수만으로 전체 비용을 판단할 수 없다.
②	클러스터링 팩터는 인덱스 ROWID로 테이블을 방문할 때 특히 중요하다.
④	ROWID는 곧바로 메모리 포인터처럼 연결되는 주소가 아니므로 액세스 비용이 발생한다.

정리

실행계획에서 TABLE ACCESS BY INDEX ROWID를 보면 랜덤 액세스 비용을 반드시 떠올려야 한다.

문제 8 해설

정답: ③

핵심 개념

부분범위 처리는 전체 결과를 모두 만들지 않고 필요한 일부를 먼저 반환하는 전략이다. Top N 화면에서는 조건과 정렬을 동시에 만족하는 인덱스가 중요하다.

정답 도출

고객ID = :cust_id로 특정 고객 범위를 찾고, 그 범위 안에서 거래일시 DESC 순서대로 읽으면 최근 거래부터 반환할 수 있다. ROWNUM <= 30에서 멈추므로 초기 응답 속도가 빨라진다.

오답 분석

선택지	이유
①	고객 조건을 이용할 수 없고 정렬 목적과도 맞지 않는다.
②	거래일시가 선두이면 전체 거래 시간 순서에서 고객ID를 필터링해야 할 수 있다.
④	고객ID 이후 거래금액이 끼어 있어 거래일시 정렬 순서를 직접 활용하기 어렵다.

정리

Top N 인덱스는 **필수 등치 조건 + 정렬 컬럼** 순서를 우선 검토한다.

문제 9 해설

정답: ④

핵심 개념

결합 인덱스에서 앞 컬럼이 범위 조건이 되면 뒤 컬럼이 스캔 범위를 줄이는 힘이 약해질 수 있다. 등치 조건은 후속 컬럼의 군집성을 살리는 데 유리하다.

정답 도출

지역코드 LIKE :region || '%'는 입력값이 정확한 코드라도 범위 조건이다. 지역코드 = :region으로 바꾸면 (회사코드, 지역코드)까지 정확히 좁힌 뒤 상품명 LIKE 'xxx%' 범위를 더 잘 활용할 수 있다.

오답 분석

선택지	이유
①	등치 조건은 오히려 액세스 조건으로 쓰이기 쉽다.
②	LIKE는 패턴에 따라 범위가 넓어질 수 있고 등치 조건과 같지 않다.
③	선두 컬럼 회사코드 등치 조건은 그대로 사용할 수 있다.

정리

입력값이 필수이고 정확히 일치해야 한다면 습관적인 LIKE보다 = 조건이 인덱스 효율에 유리하다.

문제 10 해설

정답: ②

핵심 개념

선두 컬럼이 범위 조건이면 후행 컬럼의 탐색 효율이 제한될 수 있다. 범위가 소수의 이산 값이라면 IN-list로 분해해 후행 등치 조건을 더 잘 활용할 수 있다.

정답 도출

BETWEEN은 :d1부터 :d3까지 하나의 연속 구간을 스캔한다. 반면 IN-list는 세 날짜 각각에 대해 (거래일자, 시장구분, 종목코드)의 더 좁은 구간을 탐색할 수 있다.

오답 분석

선택지	이유
①	거래일자 조건은 사라지지 않는다.
③	IN-list는 오히려 반복 RANGE SCAN으로 후행 조건을 더 잘 살릴 수 있다.
④	BETWEEN과 IN-list의 우열은 값 개수, 분포, 후행 조건에 따라 달라진다.

정리

BETWEEN을 IN-list로 바꾸는 튜닝은 값 개수가 적을 때만 검토한다. 값이 많으면 반복 탐색 비용이 더 커질 수 있다.

문제 11 해설

정답: ①

핵심 개념

옵션 조건을 OR로 한 SQL에 몰아넣으면 인덱스 액세스 조건으로 쓰기 어려워질 수 있다. 조건 입력 여부에 따라 다른 최적 경로가 필요하다.

정답 도출

고객ID를 입력한 경우에는 (고객ID, 거래일자) 인덱스가 유리할 수 있고, 입력하지 않은 경우에는 (거래일자, 고객ID) 또는 날짜 파티션/Full SCAN이 유리할 수 있다. SQL 분기나 UNION ALL로 두 경로를 분리하면 각 경우에 맞는 실행계획을 만들기 쉽다.

오답 분석

선택지	이유
②	:cust_id IS NULL OR 고객ID = :cust_id는 일반적으로 고객ID 인덱스 액세스 조건으로 쓰기 어렵다.
③	고객ID가 NULL이면 선두 컬럼 조건이 없어져 거래_X1로 소량 스캔을 보장할 수 없다.
④	옵션 조건은 인덱스 컬럼 순서와 SQL 분기 전략에 큰 영향을 준다.

정리

검색 화면 SQL은 개발 편의성보다 입력 조합별 실행계획 안정성이 더 중요할 때가 많다.

문제 12 해설

정답: ④

핵심 개념

선두 컬럼이 같은 인덱스는 상위 인덱스가 하위 인덱스 역할을 대체할 수 있는지 검토해야 한다. 중복 인덱스는 DML 부하와 저장 공간을 늘린다.

정답 도출

고객_X3(고객ID, 가입일자, 상태코드)는 고객ID 단건 조회와 고객ID + 가입일자 조회에도 사용 가능하다. 다만 인덱스 크기, 클러스터링 팩터, Unique 여부, 자주 쓰는 SQL의 응답 요구를 확인한 뒤 제거 여부를 결정한다. 고객_X4는 가입일자/상태코드 집계용으로 별도 가치가 있다.

오답 분석

선택지	이유
①	같은 선두 컬럼 인덱스가 모두 필요한 것은 아니다.
②	고객_X4는 가입일자와 상태코드를 조건으로 하는 집계 SQL에 유용할 수 있다.
③	인덱스가 많으면 DML마다 유지 비용이 증가한다.

정리

인덱스 설계는 추가보다 정리도 중요하다. SQL별 필요성과 DML 비용을 함께 본다.

문제 13 해설

정답: ①

핵심 개념

NL 조인은 선행 집합의 각로우마다 후행 집합을 반복 탐색한다. Starts는 특정 오퍼레이션이 몇 번 실행되었는지 보여준다.

정답 도출

주문에서 10,000건이 나오고, 주문상세_X1의 Starts도 10,000이다. 이는 선행 주문 10,000건 각각에 대해 후행 주문상세 인덱스를 탐색했다는 뜻이다. A-Rows가 최종 500이어도 반복 탐색 과정에서 Buffers가 70,000까지 늘었다.

오답 분석

선택지	이유
②	결과 건수만 적어도 반복 탐색과 버퍼 읽기가 많으면 비효율이다.
③	NL 조인은 반복 랜덤 액세스로 Buffers가 커질 수 있다.
④	후행 인덱스가 있어도 반복 횟수가 많으면 병목이 된다.

정리

NL 조인 튜닝은 **선행 건수 × 후행 접근 비용**을 줄이는 게임이다.

문제 14 해설

정답: ③

핵심 개념

Sort MERGE JOIN은 양쪽 입력을 조인 키 순서로 정렬해 병합한다. 이미 정렬된 경로를 활용하거나 대량 조인에서 랜덤 액세스를 피할 때 고려할 수 있다.

정답 도출

문제 조건은 이미 정렬된 입력을 얻기 쉬우며, NL의 반복 랜덤 액세스가 부담스럽다. 이런 상황에서는 Sort MERGE JOIN이 선택될 수 있다. 단, 정렬 비용과 PGA/Temp 사용 가능성을 함께 봐야 한다.

오답 분석

선택지	이유
①	입력이 이미 정렬되어 있으면 정렬 부담이 작거나 생략될 수 있다.
②	Sort MERGE JOIN은 정렬 과정에서 PGA와 Temp를 사용할 수 있다.
④	조인 조건과 데이터량에 따라 적합성이 달라진다.

정리

조인 방식은 고정 답이 없다. 데이터량, 정렬 여부, 랜덤 액세스 비용을 함께 판단한다.

문제 15 해설

정답: ②

핵심 개념

HASH JOIN은 한쪽 입력으로 해시 테이블을 만들고 다른 입력을 읽으며 해시 테이블을 탐색한다. 보통 작은 쪽을 Build Input으로 삼는 것이 메모리 부담을 줄인다.

정답 도출

고객 50만 건과 주문 2천만 건이라면 일반적으로 고객 쪽이 Build Input 후보가 된다. 물론 필터 조건, 조인 컬럼 분포, 필요한 컬럼, 병렬 분배 방식에 따라 실제 계획은 달라질 수 있다.

오답 분석

선택지	이유
①	SQL에 적은 순서가 Build Input을 결정하는 절대 기준은 아니다.
③	HASH JOIN은 대량 집합 조인에서 자주 유리하다.
④	Build Input이 메모리에 다 들어가지 않으면 Temp 사용과 성능 저하가 생길 수 있다.

정리

HASH JOIN에서는 Build Input 크기와 메모리 사용량을 반드시 확인한다.

문제 16 해설

정답: ②

핵심 개념

옵티마이저는 서브쿼리를 그대로 반복 실행하지 않고 조인 형태로 변환하거나 캐싱, 인덱스 탐색 등을 활용할 수 있다. 이를 쿼리 변환 관점에서 이해해야 한다.

정답 도출

문제 SQL은 고객별 최근 주문일자를 구한다. **주문(고객ID, 주문일자)** 인덱스가 있으면 고객별 MAX를 빠르게 찾을 수 있고, 경우에 따라 조인/집계로 변환하는 방식도 가능하다.

오답 분석

선택지	이유
①	항상 Full SCAN한다고 볼 수 없다. 인덱스, 캐싱, 변환에 따라 달라진다.
③	서브쿼리는 실행계획에서 세미 조인, 안티 조인, 일반 조인, 스칼라 서브쿼리 형태로 나타날 수 있다.
④	스칼라 서브쿼리는 한 로우 한 컬럼을 반환해야 하며 여러 건이면 오류가 발생할 수 있다.

정리

서브쿼리 성능은 문법 모양보다 옵티마이저가 어떤 실행 형태로 바꾸는지를 봐야 한다.

문제 17 해설

정답: ④

핵심 개념

UNION은 중복 제거를 포함한다. 중복 제거에는 정렬 또는 해시 작업이 필요할 수 있다. **UNION ALL**은 단순 결합이므로 중복 제거 비용이 없다.

정답 도출

업무 규칙상 두 집합에 중복이 없다면 **UNION**의 중복 제거는 불필요한 비용이다. 이런 경우 **UNION ALL**이 결과 의미를 유지하면서 더 가볍다.

오답 분석

선택지	이유
①	중복 없음이 보장되면 UNION ALL 을 적극 검토한다.
②	중복이 없으면 결과 건수는 같다. 중복이 있으면 UNION ALL 이 더 많다.
③	두 연산자는 중복 처리 방식과 실행 비용이 다르다.

정리

중복 제거가 필요하지 않은데 **UNION**, **DISTINCT**, 불필요한 **GROUP BY**를 쓰면 소트 비용이 생긴다.

문제 18 해설

정답: ③

핵심 개념

인덱스는 정렬된 구조다. 조건과 정렬 방향이 인덱스 구성과 맞으면 **ORDER BY** 소트를 생략하고 Top N에서 빨리 멈출 수 있다.

정답 도출

인덱스가 (종목코드, 체결일시 DESC, 체결가격)이고 SQL은 종목코드 = :code, **ORDER BY** 체결일시 DESC다. 특정 종목 범위 안에서 이미 최신순으로 정렬되어 있으므로 5건만 읽고 멈출 수 있다. 다만 **체결번호**는 인덱스에 포함되어 있지 않으므로, 최종 5건에 대해서만 **TABLE ACCESS BY INDEX ROWID**가 발생한다. 정렬 생략과 Stopkey 이점은 그대로 유지된다.

오답 분석

선택지	이유
①	Descending RANGE SCAN도 가능하다.
②	ORDER BY가 있어도 인덱스 순서를 활용하면 별도 Sort가 생략될 수 있다.
④	Stopkey 조건은 인덱스 순서와 결합될 때 큰 성능 이점을 준다.

정리

Top N은 정렬 생략 + 조기 중단이 핵심이다.

문제 19 해설

정답: ①

핵심 개념

Sort Area 부담은 정렬 대상 로우 수와 로우 폭에 영향을 받는다. 정렬 전에 불필요하게 큰 문자열을 만들면 메모리와 Temp 부담이 커질 수 있다.

정답 도출

A안은 정렬 전에 **line** 문자열을 크게 만든다. B안은 인라인 뷰에서 **상품번호, 상품명, 주문일시** 같은 원본 컬럼만 정렬한 뒤 바깥에서 문자열 포매팅을 수행한다. 정렬 대상의 행 폭을 줄일 수 있으므로 B안이 Sort Area 사용량을 줄이는 방향에 더 적절하다.

오답 분석

선택지	이유
②	문자열을 크게 만든 뒤 정렬하면 정렬 대상 폭이 커질 수 있다.
③	정렬 전에 만드는 데이터 폭이 다를 수 있다.
④	SQL 작성 방식은 정렬 입력량과 폭에 영향을 준다.

정리

정렬 전에는 행 수와 컬럼 폭을 줄이고, 포매팅은 가능하면 정렬 이후로 미룬다.

문제 20 해설

정답: ②

핵심 개념

DML 성능에는 인덱스와 Redo/Undo뿐 아니라 Database Call 횟수가 크게 영향을 준다. 한 건씩 호출하면 네트워크 왕복과 Parse/Execute 비용이 반복된다.

정답 도출

PreparedStatement를 루프 밖에서 만들고, 여러 건을 바인딩한 뒤 배치 실행하면 Database Call 횟수를 크게 줄일 수 있다. Array Processing은 대량 DML의 기본 개선 포인트다.

오답 분석

선택지	이유
①	한 건씩 처리한다고 Lock이 없어지지 않는다. 호출 비용만 늘 수 있다.
③	인덱스 추가는 INSERT 유지 비용을 늘릴 수 있고 Call 횟수와 무관하다.
④	Commit을 무작정 미루면 Undo, Lock 유지, 장애 복구 부담이 커질 수 있다.

정리

대량 DML은 “한 번에 많이 보내기”와 “불필요한 인덱스 유지 줄이기”를 함께 검토한다.

문제 21 해설

정답: ④

핵심 개념

Direct Path Insert는 버퍼 캐시를 경유하는 일반 경로와 달리 대량 적재에 유리할 수 있다. 보통 **APPEND**, 병렬 DML, 인덱스 후생성, 통계 수집과 함께 검토한다.

정답 도출

야간 대량 이관이고 동시 충돌이 없다면 Direct Path Insert가 적합한 후보가 된다. 다만 Redo/Undo, Lock, HWM 증가, 인덱스 관리, 통계정보 등 운영 조건을 함께 확인해야 한다.

오답 분석

선택지	이유
①	대량 적재 후 통계정보가 부정확하면 후속 SQL 실행계획이 나빠질 수 있다.
②	OLTP 단건 INSERT는 버퍼 캐시 재사용과 동시성이 중요하므로 일반적으로 Direct Path가 기본 선택은 아니다.
③	모든 부하가 0이 되는 것은 아니다. 모드와 옵션에 따라 Redo/Undo도 발생할 수 있다.

정리

Direct Path는 대량 처리 도구다. OLTP와 배치의 성격을 구분해야 한다.

문제 22 해설

정답: ③

핵심 개념

파티션은 관리 단위이자 성능 단위다. 대량 삭제를 로우 단위로 수행하지 않고 파티션 단위 작업으로 바꾸면 Undo/Redo, 인덱스 유지, 수행 시간을 크게 줄일 수 있다.

정답 도출

삭제 대상이 월 파티션과 정확히 대응하면 **TRUNCATE PARTITION** 또는 **DROP PARTITION**이 Row 단위 DELETE보다 훨씬 빠를 수 있다. 로컬 인덱스는 파티션과 1:1 대응하므로 관리 측면에서도 유리하다.

오답 분석

선택지	이유
①	로컬 인덱스는 파티션 구조 변경 시 관리 편의성이 높다. 글로벌 인덱스는 Unusable 이슈를 더 주의해야 한다.
②	파티션 단위 DDL로 대량 삭제를 대체할 수 있다.
④	파티션 Pruning과 파티션 단위 DML은 대량 처리 성능에 직접 도움이 된다.

정리

시계열 대량 데이터는 “인덱스로 빨리 지우기”보다 “파티션 단위로 제거하기”가 더 강력할 수 있다.

문제 23 해설

정답: ①

핵심 개념

채번은 동시성 제어와 직결된다. **max + 1** 방식은 여러 세션이 같은 최댓값을 읽고 같은 다음 번호를 계산할 수 있어 충돌과 직렬화가 필요하다.

정답 도출

동시 주문이 많은 시스템에서는 같은 일자의 최댓값을 읽는 세션들이 몰린다. 중복을 막으려면 별도 Lock이나 직렬화가 필요하고, 이 지점이 INSERT 병목이 된다. 시퀀스나 별도 채번 설계를 검토한다.

오답 분석

선택지	이유
②	max + 1 은 직렬화 병목이 생기기 쉬워 고동시성에 취약하다.
③	인덱스는 max 탐색을 빠르게 할 수 있지만 동시성 충돌 자체를 제거하지 않는다.
④	채번 방식은 INSERT 처리량과 Lock 경합에 큰 영향을 준다.

정리

성능 좋은 채번은 빠른 조회보다 경합 없는 동시 처리가 핵심이다.

문제 24 해설

정답: ④

핵심 개념

선택도는 전체 로우 중 조건을 만족할 것으로 예상되는 비율이고, 카디널리티는 예상 로우 수다. 균일 분포의 등치 조건은 단순히 **1 / NDV**로 추정할 수 있다.

정답 도출

선택도 = $1 / 100 = 0.01$
 카디널리티 = $10,000,000 \times 0.01 = 100,000$

따라서 약 10만 건이다.

오답 분석

선택지	이유
①	NDV가 1,000일 때의 단순 추정에 가깝다.
②	조건이 전혀 선택력을 갖지 않을 때에 가깝다.
③	NDV가 10일 때의 단순 추정에 가깝다.

정리

옵티마이저의 비용 계산은 선택도와 카디널리티 추정에서 출발한다. 데이터 분포가 균일하지 않으면 히스토그램 등 추가 통계가 중요해진다.

문제 25 해설

정답: ②

핵심 개념

예상 실행계획은 옵티마이저의 추정이다. 실제 수행 중 각 단계가 몇 번 실행되었고 얼마나 읽었는지는 Row Source 통계를 봐야 한다.

정답 도출

gather_plan_statistics 힌트 또는 세션 통계 설정 후 다음처럼 확인할 수 있다.

```
SELECT *  
FROM TABLE(dbms_xplan.display_cursor(NULL, NULL, 'ALLSTATS LAST'));
```

이 방식은 **Starts, E-Rows, A-Rows, Buffers, Reads** 등을 통해 예상과 실제의 차이를 볼 수 있다.

오답 분석

선택지	이유
①	EXPLAIN PLAN은 실제 실행 통계를 보여주지 않는다.
③	V\$SQL은 누적 통계로 튜닝 대상 선정에는 좋지만 실행계획 단계별 병목을 직접 보여주지는 않는다.
④	실제 성능 문제는 예상 계획만으로 설명되지 않는 경우가 많다.

정리

분석 도구는 목적별로 고른다. 예상 계획은 빠른 확인, SQL Trace와 DBMS_XPLAN ALLSTATS는 실제 병목 분석, V\$SQL은 누적 통계 기반 대상 선정에 강하다.

SQLP 상급 객관식 문제집

25문항 · 4지선다 · SQLP 상 수준

문 제 지

1.

다음 상황에서 파싱 부하를 줄이기 위한 설명으로 가장 적절한 것은?

회원별 주문 조회 API가 초당 수천 번 호출된다. 애플리케이션은 아래 SQL에서 회원번호와 기간 값을 문자열로 붙여 매번 다른 SQL 텍스트를 만들어 전송하고 있다. Library Cache Miss와 Hard Parse가 증가했다.

```
SELECT order_id, order_dt, amount
FROM   orders
WHERE  cust_id = 100431
      AND order_dt >= DATE '2025-01-01'
      AND order_dt <  DATE '2025-02-01';
```

- ① 인덱스 컬럼을 추가하면 SQL 텍스트가 달라도 커서 공유 문제가 해소된다
- ② 동일 의미의 SQL이라면 공백·리터럴이 달라도 같은 커서를 항상 재사용한다
- ③ 리터럴을 바인드 변수로 바꾸면 Soft Parse 가능성이 높아진다
- ④ SELECT 절 컬럼 순서를 변경하면 Library Cache 탐색 비용이 낮아진다

2.

다음 실행계획을 보고 I/O 관점의 병목과 튜닝 방향으로 가장 적절한 것은?

PAYMENTS는 3천만 건이며 PAY_X01은 (PAY_STATUS, PAY_DT) 인덱스다. 조회 결과는 약 90만 건이다.

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		900K	72M	9800
1	TABLE ACCESS BY INDEX ROWID	PAYMENTS	900K	72M	9800
2	INDEX RANGE SCAN	PAY_X01	900K		900

- ① 대량 ROWID 방문이 병목이므로 Full Scan 또는 커버링 인덱스와 비용을 비교한다
- ② 인덱스 단계 Cost가 작으므로 이후 테이블 랜덤 액세스 비용은 무시해도 된다
- ③ Index Range Scan은 Multiblock I/O 방식이라 대량 조회에서도 Full Scan보다 항상 유리하다
- ④ Rows 추정치가 900K이면 옵티마이저는 테이블 블록에 대한 물리적 방문을 생략한다

3.

다음 SQL의 초기 응답 속도를 높이기 위한 접근으로 가장 적절한 것은?

게시글 목록 화면은 최신 글 30건만 먼저 보여준다. BOARD_X01은 (BOARD_TYPE, REG_DT DESC) 인덱스다.

```
SELECT *  
FROM (  
    SELECT board_id, title, writer_id, reg_dt  
    FROM board  
    WHERE board_type = :type  
    ORDER BY reg_dt DESC  
)  
WHERE ROWNUM <= 30;
```

- ① ROWNUM은 정렬 이후에만 평가되므로 전체 게시글을 모두 정렬해야만 한다
- ② ORDER BY가 있으면 인덱스를 사용하더라도 SORT ORDER BY는 반드시 발생한다
- ③ SELECT 컬럼이 인덱스에 없으면 Stopkey 동작 자체가 불가능하고 전체 스캔이 강제된다
- ④ 조건·정렬 순서가 인덱스와 맞으므로 필요 건수에서 멈추는 처리를 기대할 수 있다

4.

인덱스 IDX_EMP_01(DEPTNO, JOB, SAL)가 있을 때 다음 조건의 인덱스 사용 방식으로 가장 적절한 것은?

EMP 테이블은 부서별, 직무별 급여 조회가 자주 발생한다.

```
SELECT empno, ename, sal  
FROM emp  
WHERE deptno = :deptno  
    AND job IN ('CLERK', 'ANALYST')  
    AND sal BETWEEN :low_sal AND :high_sal;
```

- ① JOB이 IN 조건이므로 인덱스 탐색 시 DEPTNO보다 앞선 선두 컬럼으로 자동 재배치된다
- ② DEPTNO 등치 후 JOB 값별 범위 탐색이 가능하고 SAL 범위도 활용될 수 있다
- ③ SAL BETWEEN 조건이 존재하면 앞의 DEPTNO, JOB은 모두 필터 조건으로 밀린다
- ④ IN 조건은 B-Tree 인덱스 탐색에 사용할 수 없어 Full Scan이 강제된다

5.

다음 중 Index Fast Full Scan 선택이 가장 타당한 상황은?

ORDERS 테이블에는 ORD_X02(STATUS, ORDER_DT, AMOUNT, CUST_ID) 인덱스가 있다.

```
SELECT status, COUNT(*), SUM(amount)
FROM   orders
WHERE  order_dt >= DATE '2025-01-01'
GROUP BY status;
```

- ① 인덱스 키 순서대로 읽어 ORDER BY 생략이 필요하며 결과 정렬 순서 보장이 목적일 때
- ② 선두 컬럼 STATUS 등치 단건 조회라 Unique Scan으로 즉시 특정 행을 찾아야 할 때
- ③ ROWID 순서로 테이블 블록을 순차 방문해 랜덤 액세스를 완전히 없애야 할 때
- ④ 필요 컬럼이 인덱스에 있고 정렬 순서보다 대량 스캔 효율이 중요할 때

6.

다음 SQL에서 일반 인덱스 ORD_X01(ORDER_DT)이 있을 때 가장 적절한 튜닝 설명은?

주문일 조건 검색이 느리다. 함수 기반 인덱스는 없다.

```
SELECT order_id, order_dt, amount
FROM   orders
WHERE  TO_CHAR(order_dt, 'YYYYMM') = '202501';
```

- ① 컬럼 가공을 없애고 ORDER_DT 범위 조건으로 바꾸면 인덱스 활용 가능성이 높아진다
- ② TO_CHAR는 표시 전용 함수이므로 WHERE 절에 있어도 인덱스 탐색 방식에 영향이 없다
- ③ SELECT 절에 ORDER_DT가 있으면 Range Scan이 가능하다
- ④ 옵티마이저는 함수 조건을 언제나 원래 날짜 범위로 자동 변환하여 인덱스를 활용한다

7.

다음 실행계획에서 가장 먼저 의심해야 할 성능 문제는?

CUSTOMERS에서 등급 조건으로 20만 건을 찾고, 각 고객의 최근 주문을 조회한다.

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		200K	32M	14500
1	NESTED LOOPS		200K	32M	14500
2	TABLE ACCESS FULL	CUSTOMERS	200K	12M	1100
3	TABLE ACCESS BY INDEX ROWID	ORDERS	1	100	65
4	INDEX RANGE SCAN	ORD_CUST_X	1		4

- ① CUSTOMERS Full Scan은 1회로 고정이라 총 Cost 9,800에서 차지하는 비중이 미미하다
- ② ORD_CUST_X Cost가 4이므로 후행 테이블 탐색은 전체 비용에 거의 영향을 주지 않는다
- ③ 후행 ORDERS 접근이 선행 20만 건만큼 반복되어 랜덤 액세스가 누적될 수 있다
- ④ Nested Loops 방식에서는 테이블 액세스 없이 인덱스 결과만으로 처리가 완료된다

8.

고객별 최근 거래 50건 조회를 위한 인덱스 설계로 가장 적절한 것은?

조건은 CUST_ID 등치이고, 결과는 TRADE_DT DESC, TRADE_ID DESC 순으로 정렬한다.

```
SELECT trade_id, trade_dt, amount
FROM   trades
WHERE  cust_id = :cust_id
ORDER BY trade_dt DESC, trade_id DESC
FETCH FIRST 50 ROWS ONLY;
```

- ① (TRADE_DT DESC, CUST_ID, TRADE_ID DESC) — 전체 최신 거래를 먼저 스캔한 뒤 고객 필터
- ② (CUST_ID, TRADE_DT DESC, TRADE_ID DESC) — 고객 범위 안에서 정렬 순서를 맞춘다
- ③ (AMOUNT, CUST_ID, TRADE_DT) — 금액 선택도를 선두에 두어 스캔 범위를 줄인다
- ④ (CUST_ID) 단일 인덱스 — 고객을 찾은 뒤 ORDER BY는 항상 자동으로 생략된다

9.

다음 조건에서 인덱스 스캔 효율을 가장 정확하게 설명한 것은?

IDX_SALES_01은 (REGION, SALE_DT, PRODUCT_ID) 순서다.

```
WHERE region      = :region
AND  sale_dt      BETWEEN :d1 AND :d2
AND  product_id   = :pid
```

- ① SALE_DT 범위 이후 PRODUCT_ID는 인덱스 필터는 가능하나 스캔 범위 축소 효과가 제한된다
- ② PRODUCT_ID가 등치 조건이므로 인덱스에서 SALE_DT 범위보다 먼저 탐색 기준으로 승격된다
- ③ 세 컬럼 모두 인덱스에 있으므로 조건 종류와 무관하게 필요한 리프 엔트리만 정확히 방문한다
- ④ SALE_DT가 범위 조건이면 선두 컬럼 REGION 조건도 인덱스 탐색에 사용할 수 없다

10.

다음 조건을 처리할 때 인덱스 활용 방식 설명으로 가장 적절한 것은?

IDX_MEMBER_01은 (JOIN_TYPE, JOIN_DT, MEMBER_ID) 순서다. JOIN_TYPE 값 종류는 3개이고, 특정 기간의 가입자를 조회한다.

```
WHERE join_dt >= DATE '2025-01-01'
AND  join_dt <  DATE '2025-02-01'
AND  member_id = :member_id
```

- ① 선두 컬럼 JOIN_TYPE 조건이 없으므로 어떤 경우에도 이 인덱스 자체를 접근할 수 없다
- ② MEMBER_ID 등치 조건이 있으므로 세 번째 컬럼이 자동으로 선두처럼 동작해 탐색된다
- ③ JOIN_DT 범위 조건은 인덱스에서 처리되지 않고 테이블 블록을 방문한 뒤에만 필터링된다
- ④ 선두 컬럼 NDV가 작으면 Skip Scan 후보가 될 수 있다

11.

다음 조인 SQL의 초기 응답과 전체 비용을 함께 고려할 때 가장 적절한 설명은?

ORDERS는 최근 1시간 조건으로 약 300건, ORDER_ITEMS는 주문당 평균 120건이다.
ORDER_ITEMS(order_id) 인덱스가 있다.

```
SELECT o.order_id, i.item_id, i.qty
FROM   orders      o
JOIN   order_items i ON i.order_id = o.order_id
WHERE  o.order_dt >= SYSDATE - 1/24;
```

- ① ORDER_ITEMS 행 수가 많으므로 항상 ORDER_ITEMS를 선행 Full Scan하고 ORDERS를 필터해야 한다
- ② 조인 결과 건수가 많아질 수 있으므로 Sort Merge Join만이 결과 정확성을 보장한다
- ③ ORDERS를 먼저 줄이고 ORDER_ITEMS를 인덱스로 탐색하는 NL 조인이 유리할 수 있다
- ④ ORDER_ITEMS 인덱스는 후행 테이블 쪽에 있으므로 NL 조인 전체 비용과는 무관하다

12.

다음 실행계획에서 Hash Join을 선택한 이유로 가장 설득력 있는 것은?

두 테이블 모두 조건 적용 후에도 대량 결과가 남는다.

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		1800K	180M	7600
1	HASH JOIN		1800K	180M	7600
2	TABLE ACCESS FULL	CUSTOMERS	120K	12M	900
3	TABLE ACCESS FULL	ORDERS	2500K	210M	6200

- ① 양쪽 입력이 조인 키 기준으로 이미 정렬되어 있어 정렬 단계 없이 병합할 수 있기 때문이다
- ② 작은 입력으로 해시 테이블을 만들고 큰 입력을 스캔해 대량 조인을 처리하기 유리하다
- ③ 선행 로우마다 후행 테이블을 인덱스로 한 건씩 반복 탐색하는 방식이 효율적이기 때문이다
- ④ 조인 키 인덱스가 양쪽에 모두 없는 경우에만 Hash Join이 선택될 수 있기 때문이다

13.

Sort Merge Join이 상대적으로 검토될 수 있는 상황으로 가장 적절한 것은?

두 테이블의 조인 조건과 데이터 특성은 다음과 같다.

```
SELECT a.acct_no, b.grade
FROM   account    a
JOIN   grade_range b
      ON a.balance BETWEEN b.min_balance AND b.max_balance
WHERE  a.open_dt >= DATE '2025-01-01';
```

- ① 선행 집합이 극소량이고 후행 테이블에 조인 키 인덱스가 잘 갖춰져 있을 때
- ② 두 테이블 모두 해시 영역에 충분히 들어가며 등치(=) 조인만 존재하는 대량 조인일 때
- ③ 정렬 컬럼이 전혀 없고 조인 조건이 항상 기본키 등치 조건인 단순 조인일 때
- ④ 범위 기반 비등가 조인이고 두 입력을 정렬 후 병합하는 방식이 유리할 때

14.

다음 스칼라 서브쿼리의 성능 문제와 개선 방향으로 가장 적절한 것은?

EMP는 50만 건이고 DEPT는 1천 건이다. DEPTNO별 부서명 조회가 반복된다.

```
SELECT e.empno,
       e.ename,
       (SELECT d.dname
        FROM   dept d
        WHERE  d.deptno = e.deptno) AS dname
FROM   emp e
WHERE  e.hiredate >= DATE '2024-01-01';
```

- ① 외부 로우마다 반복될 수 있으므로 조인 변환이나 캐싱 가능성을 검토한다
- ② 스칼라 서브쿼리는 항상 한 번만 실행되므로 건수가 많아도 성능 문제와 무관하다
- ③ SELECT 절 서브쿼리를 문자열 상수로 바꾸면 결과는 동일하면서 실행 비용이 줄어든다
- ④ 스칼라 서브쿼리가 포함되면 옵티마이저는 어떤 조인 방식도 선택하지 못하고 필터로만 처리한다

15.

다음 SQL에서 UNION 대신 UNION ALL을 검토해야 하는 이유로 가장 적절한 것은?

ONLINE_ORDERS와 STORE_ORDERS는 서로 다른 주문 채널이지만 같은 고객이 양쪽에 모두 존재할 수 있다. 보고서는 중복 제거가 필요 없다.

```
SELECT cust_id, order_id FROM online_orders WHERE order_dt >= :dt
UNION
SELECT cust_id, order_id FROM store_orders WHERE order_dt >= :dt;
```

- ① UNION은 두 결과를 단순 연결만 하므로 UNION ALL과 실행 비용이나 결과가 전혀 다르지 않다
- ② UNION은 중복 제거를 위해 Sort Unique 또는 Hash Unique 연산이 추가될 수 있다
- ③ UNION ALL은 양쪽 결과를 합친 뒤 중복을 제거하므로 UNION보다 결과 정확성이 높다
- ④ 채널이 서로 달라 중복이 없다고 판단하면 옵티마이저가 UNION을 UNION ALL로 자동 변환한다

16.

다음 Top-N SQL에 대한 설명으로 가장 적절한 것은?

ORD_X03은 (STATUS, ORDER_DT DESC) 인덱스다.

```
SELECT *
FROM (
  SELECT order_id, order_dt, amount
  FROM   orders
  WHERE  status = 'C'
  ORDER BY order_dt DESC
)
WHERE ROWNUM <= 10;
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		10	1200	15
1	COUNT STOPKEY				
2	TABLE ACCESS BY INDEX ROWID	ORDERS	10	1200	15
3	INDEX RANGE SCAN	ORD_X03	10		4

- ① COUNT STOPKEY는 COUNT(*) 집계를 의미하므로 Top-N 건수 제한과는 무관하다
- ② Top-N이면 정렬 기준 인덱스가 없어도 항상 전체 정렬이 사라진다
- ③ STOPKEY가 있으면 WHERE 절 조건 필터링이 모두 생략되고 인덱스 스캔만 수행된다
- ④ 필요 건수에 도달하면 하위 Row Source 요청을 멈출 수 있음을 보여 준다

17.

다음 GROUP BY SQL의 소트 발생 가능성에 대한 설명으로 가장 적절한 것은?

SALES_X01은 (SALE_DT, CUST_ID) 인덱스이며 한 달치 매출을 고객별 집계한다.

```
SELECT cust_id, SUM(amount)
FROM   sales
WHERE  sale_dt >= DATE '2025-01-01'
      AND sale_dt <  DATE '2025-02-01'
GROUP BY cust_id;
```

- ① SALE_DT 인덱스가 있으므로 GROUP BY CUST_ID 소트는 항상 완전히 생략된다
- ② SUM 같은 집계 함수가 있으면 옵티마이저는 반드시 Sort Merge Join 방식을 선택하게 된다
- ③ 입력 순서가 그룹 키와 맞지 않으면 HASH/SORT GROUP BY가 발생할 수 있다
- ④ GROUP BY 컬럼이 인덱스 후행에만 있으면 테이블 액세스 없이 항상 Nosort 집계가 가능하다

18.

대량 UPDATE 성능을 개선하기 위한 설명으로 가장 적절한 것은?

일 배치가 SALES 500만 건의 상태 값을 갱신한다. 해당 테이블에는 상태 컬럼과 관련 없는 보조 인덱스가 12개 존재한다.

```
UPDATE sales
SET    close_yn = 'Y'
WHERE  sale_dt < DATE '2025-01-01'
      AND close_yn = 'N';
```

- ① 인덱스 유지 비용, Undo/Redo, Commit 전략, 배치 단위를 종합적으로 검토한다
- ② UPDATE는 SELECT와 달리 인덱스 엔트리 유지가 불필요하므로 인덱스 개수와 무관하다
- ③ 건마다 Commit하면 Redo 로그와 Lock 유지 시간이 분산되어 항상 전체 성능이 향상된다
- ④ 조건 컬럼에 인덱스만 있으면 갱신 대상 탐색 이후의 테이블 블록 변경 비용도 사라진다

19.

대량 INSERT에서 Direct Path Insert를 검토할 때 가장 적절한 설명은?

야간 배치로 외부 파일의 2천만 건을 적재한다. 적재 중 동시 조회는 거의 없다.

```
INSERT /*+ APPEND */ INTO sales_stage
SELECT *
FROM   ext_sales_file;
```

- ① APPEND 힌트를 쓰면 모든 인덱스 유지 비용과 Lock 이슈가 완전히 제거된다
- ② 버퍼 캐시 경유를 줄이고 세그먼트에 직접 적재해 대량 입력에 유리할 수 있다
- ③ Direct Path Insert는 소량 OLTP 단건 입력 상황에서도 항상 최우선 선택이다
- ④ Redo와 Undo가 전혀 발생하지 않는다

20.

파티션을 활용한 대량 삭제 튜닝으로 가장 적절한 것은?

SALES는 SALE_DT 기준 Range 파티션 테이블이다. 2024년 1월 데이터 전체를 삭제해야 하며, 해당 월이 단일 파티션과 정확히 대응한다.

```
DELETE FROM sales
WHERE sale_dt >= DATE '2024-01-01'
AND sale_dt < DATE '2024-02-01';
```

- ① 파티션 테이블은 DELETE 조건과 관계없이 전체 파티션을 모두 스캔한다
- ② Local Index가 있으면 행 단위 DELETE 시 발생하는 Undo와 Redo가 전혀 생성되지 않는다
- ③ Global Index가 있으면 파티션 DROP이나 TRUNCATE 후에도 인덱스 관리 이슈는 절대 없다
- ④ 파티션 범위와 정확히 맞으면 TRUNCATE PARTITION이나 EXCHANGE를 검토할 수 있다

21.

MVCC 환경에서 일반 SELECT와 UPDATE 동시성 설명으로 가장 적절한 것은?

세션 A가 특정 주문 행을 UPDATE 후 아직 COMMIT하지 않았다. 세션 B가 같은 행을 일반 SELECT로 조회한다.

```
-- Session A
UPDATE orders
SET    amount = amount + 1000
WHERE  order_id = 100;

-- Session B
SELECT order_id, amount
FROM   orders
WHERE  order_id = 100;
```

- ① 일반 SELECT는 공유 Lock을 획득하고 UPDATE 트랜잭션이 종료될 때까지 반드시 대기한다
- ② MVCC 환경에서는 UPDATE도 어떤 Lock도 사용하지 않으므로 쓰기 충돌이 존재하지 않는다
- ③ SELECT는 일관된 읽기를 위해 필요한 경우 Undo의 이전 이미지를 참조할 수 있다
- ④ SELECT는 커밋 전 최신 변경값을 읽어야 하므로 Dirty Read가 기본 동작이다

22.

통계정보 오류가 실행계획에 미치는 영향으로 가장 적절한 것은?

옵티마이저가 REGION = :r 조건의 선택도를 실제보다 훨씬 낮게 추정했다. 실제로는 전체의 40%가 반환된다.

```
SELECT c.cust_id, o.order_id
FROM   customers c
JOIN   orders   o ON o.cust_id = c.cust_id
WHERE  c.region = :r;
```

- ① 소량으로 오판해 NL 조인과 인덱스 반복 액세스를 선택할 수 있다
- ② 통계정보는 실행계획 수립에 사용되지 않으므로 추정 오류가 있어도 성능에 영향이 없다
- ③ 선택도 과소평가가 발생하면 옵티마이저는 항상 병렬 Full Scan만을 선택하게 된다
- ④ 카디널리티 추정 오류는 Fetch Call 횟수와 네트워크 전송 건수에만 영향을 준다

23.

다음 상황에서 옵티마이저 힌트 사용에 대한 설명으로 가장 적절한 것은?

개발자가 특정 SQL에 USE_NL 힌트를 추가했지만 실행계획은 여전히 Hash Join이다. 별칭은 아래와 같다.

```
SELECT /*+ USE_NL(orders) */ c.cust_id, o.order_id
FROM   customers c
JOIN   orders   o ON o.cust_id = c.cust_id
WHERE  c.grade = 'VIP';
```

- ① 힌트가 존재하면 문법 오류나 대상 불일치와 무관하게 옵티마이저가 반드시 강제 적용한다
- ② USE_NL은 별칭과 무관하게 실제 테이블명으로 지정하면 항상 유효하게 적용된다
- ③ USE_NL은 Fetch Call 단계에서 전송 건수를 줄이는 명령이라 조인 방식과 무관하다
- ④ 별칭 불일치, 변환, 비용 제약 등으로 힌트가 무시될 수 있다

24.

다음 SQL에서 옵티마이저 변환 가능성을 고려한 설명으로 가장 적절한 것은?

ORDERS(order_id)와 SHIPMENTS(order_id)에 인덱스가 있다. SHIPMENTS는 주문당 최대 한 건이다.

```
SELECT o.order_id
FROM   orders o
WHERE  EXISTS (
        SELECT 1
        FROM   shipments s
        WHERE  s.order_id = o.order_id
        AND    s.ship_status = 'DONE'
      );
```

- ① EXISTS 상관 서브쿼리는 옵티마이저 변환 없이 항상 외부 로우마다 물리적으로 반복 실행된다
- ② 조건이 안전하면 세미 조인으로 변환되어 조인 방식 선택 폭이 넓어질 수 있다
- ③ EXISTS를 NOT EXISTS로 바꾸면 실행 의미는 동일하고 결과 건수만 줄어든다
- ④ 서브쿼리 Unnesting은 SELECT 절에 있는 스칼라 서브쿼리에만 적용이 가능하다

25.

다음 실행계획의 카디널리티 추정 오류를 줄이기 위해 가장 먼저 검토할 항목은?

IS_ACTIVE = 'Y'인 고객은 전체의 98%지만, STATUS와 함께 특정 조합에서만 매우 적다. 단일 컬럼 통계만 수집되어 있다.

```
SELECT *
FROM   customers
WHERE  is_active = 'Y'
      AND status   = 'BLACK'
      AND join_dt  >= DATE '2025-01-01';
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		20	4000	12
1	TABLE ACCESS FULL	CUSTOMERS	20	4000	12

- ① FIRST_ROWS 모드로 변경하면 컬럼 간 상관관계 추정 문제가 자동으로 해소된다
- ② 힌트를 모두 제거하면 옵티마이저가 실행 중 실제 행 수를 자동으로 정확히 파악한다
- ③ 히스토그램·확장 통계로 값 분포와 컬럼 간 상관관계를 보완할 수 있는지 검토한다
- ④ SQL 텍스트의 대소문자를 통일하면 카디널리티 추정 오류가 해결된다

정답 및 해설

번호	정답	번호	정답	번호	정답	번호	정답	번호	정답
1	③	2	①	3	④	4	②	5	④
6	①	7	③	8	②	9	①	10	④
11	③	12	②	13	④	14	①	15	②
16	④	17	③	18	①	19	②	20	④
21	③	22	①	23	④	24	②	25	③

1번

정답: ③

[핵심 포인트] SQL 공유 조건은 실행 의미가 아니라 SQL 텍스트와 바인드 구조의 재사용 가능성이다

[정답 이유] 리터럴 값이 SQL 텍스트에 직접 포함되면 값이 바뀔 때마다 다른 SQL로 인식될 수 있다. 바인드 변수를 사용하면 SQL 형태가 유지되어 라이브러리 캐시에서 기존 커서를 찾고 재사용할 가능성이 커진다.

[오답 분석] ①: 인덱스는 데이터 접근 비용을 줄이는 수단이지 SQL 텍스트 불일치로 생기는 커서 공유 문제를 해결하지 못한다.

[오답 분석] ②: Oracle은 의미가 같다는 이유만으로 모든 SQL을 같은 커서로 취급하지 않는다. 공백·대소문자·리터럴 형태 차이도 공유 실패 원인이 될 수 있다.

[오답 분석] ④: SELECT 절 컬럼 순서 변경은 반환 구조를 바꾸는 작업일 뿐, 리터럴 SQL 반복으로 인한 Hard Parse를 직접 줄이는 대책이 아니다.

[관련 개념] SQL 파싱, Library Cache, Hard Parse/Soft Parse, 바인드 변수

2번

정답: ①

[핵심 포인트] 인덱스를 탔다고 항상 빠른 것이 아니라, 대량 테이블 랜덤 액세스가 핵심 병목이 될 수 있다

[정답 이유] 실행계획에서 인덱스 Range Scan 뒤 TABLE ACCESS BY INDEX ROWID가 90만 건 발생한다. ROWID를 따라 테이블 블록을 반복 방문하므로 Full Scan이 더 낮거나 필요한 컬럼을 인덱스에 포함해 테이블 방문을 줄이는 방향을 검토해야 한다.

[오답 분석] ②: 전체 비용은 테이블 액세스 단계에 집중되어 있다. 인덱스 단계가 낮다고 전체 SQL이 가볍다는 뜻은 아니다.

[오답 분석] ③: Range Scan 뒤 테이블 액세스는 Single Block I/O 방식의 Random Access가 반복된다. 대량이면 랜덤 I/O 부담이 커진다.

[오답 분석] ④: Rows는 예상 반환 건수다. 테이블 컬럼을 읽어야 하면 ROWID를 이용한 테이블 액세스가 수행된다.

[관련 개념] Single Block I/O, Multiblock I/O, Table Full Scan, Index Range Scan, 테이블 랜덤 액세스

3번

정답: ④

[핵심 포인트] 부분범위 처리와 Top-N Stopkey는 조건·정렬·인덱스 순서가 맞을 때 효과가 크다

[정답 이유] BOARD_TYPE 등치 조건으로 인덱스 범위를 잡고 그 범위 안에서 REG_DT DESC 순서로 읽을 수 있다. 바깥 ROWNUM 제한은 필요한 30건을 얻은 뒤 하위 Row Source 요청을 멈추게 할 수 있어 전체 정렬 부담을 줄인다.

[오답 분석] ①: 정렬 순서를 만족하는 인덱스가 있으면 전체 결과를 정렬하지 않고 상위 N건을 반환할 수 있다.

[오답 분석] ②: ORDER BY와 인덱스 컬럼 순서가 맞으면 SORT ORDER BY가 생략될 수 있다.

[오답 분석] ③: 테이블 액세스는 발생할 수 있지만 Stopkey 자체가 SELECT 컬럼 전체의 인덱스 포함을 필수로 요구하지는 않는다.

[관련 개념] Top-N Stopkey, 부분범위 처리, 인덱스를 이용한 ORDER BY 생략

4번

정답: ②

[핵심 포인트] 복합 인덱스에서 선두 등치 조건과 제한된 IN-List는 여러 Range Scan처럼 처리될 수 있다

[정답 이유] DEPTNO가 선두 등치 조건이므로 탐색 범위를 먼저 좁힐 수 있다. JOB의 IN-List는 여러 JOB 값에 대한 반복 탐색으로 처리될 수 있고, 각 JOB 값 안에서 SAL 범위 조건을 활용할 수 있다.

[오답 분석] ①: 옵티마이저가 조건을 평가하더라도 인덱스의 물리적 컬럼 순서가 바뀌는 것은 아니다.

[오답 분석] ③: SAL이 범위 조건이라고 해서 선행 컬럼의 등치/IN 조건이 무효화되지는 않는다.

[오답 분석] ④: IN-List는 경우에 따라 INLIST ITERATOR나 OR 확장 형태로 인덱스 탐색에 활용될 수 있다.

[관련 개념] 복합 인덱스, IN-List Iterator, 등치 조건, 범위 조건

5번

정답: ④

[핵심 포인트] Index Fast Full Scan은 인덱스를 작은 테이블처럼 읽는 대량 스캔 방식이며 키 순서 보장이 목적이 아니다

[정답 이유] 조회에 필요한 STATUS, ORDER_DT, AMOUNT가 인덱스에 포함되어 있다. 정렬 순서가 핵심이 아니라 테이블보다 작은 인덱스 구조를 대량으로 읽어 집계하는 것이 유리한 상황이므로 Fast Full Scan 후보가 될 수 있다.

[오답 분석] ①: 키 순서를 보장하며 ORDER BY 생략을 노리는 방식은 Index Full Scan 또는 Range Scan에 가깝다.

[오답 분석] ②: Unique Scan은 유일 키 등치 탐색에 맞는 방식이다. 이 문제는 대량 집계에 가깝다.

[오답 분석] ③: Fast Full Scan은 ROWID 순서로 테이블을 읽는 방식이 아니다. 테이블 방문을 줄이려면 필요한 컬럼이 인덱스에 있어야 한다.

[관련 개념] Index Fast Full Scan, Index Full Scan, 커버링 인덱스, 정렬 보장 여부

6번

정답: ①

[핵심 포인트] 인덱스 컬럼을 함수로 가공하면 원래 컬럼 기준 B-Tree 정렬을 직접 활용하기 어렵다

[정답 이유] ORDER_DT 인덱스는 원래 날짜 값 순서로 정렬되어 있다. WHERE 절에서 TO_CHAR로 컬럼을 가공하면 그 결과값 기준으로 탐색해야 하므로 일반 인덱스 Range Scan이 어려울 수 있다. 날짜 범위 조건으로 바꾸는 것이 대표적인 튜닝 방향이다.

[오답 분석] ②: 함수가 WHERE 조건의 컬럼에 적용되면 탐색 조건으로 쓰기 어려워질 수 있다.

[오답 분석] ③: SELECT 절 포함 여부는 조건 탐색 가능성과 다르다.

[오답 분석] ④: 모든 함수 조건을 안전하게 날짜 범위로 자동 변환할 수 있는 것은 아니다. 표현식과 NLS 설정 등에 따라 의미가 달라질 수 있다.

[관련 개념] 컬럼 가공, 함수 기반 인덱스, Range Scan 조건, 날짜 범위 검색

7번

정답: ③

[핵심 포인트] NL 조인에서는 후행 테이블 접근 비용이 선행 로우 수만큼 반복되어 누적된다

[정답 이유] 계획상 CUSTOMERS에서 20만 건이 선행으로 나오고 각 로우마다 ORDERS 인덱스와 테이블을 반복 방문한다. 한 번의 Cost가 작아 보여도 반복 횟수가 크면 대량 랜덤 액세스 병목이 된다.

[오답 분석] ①: Full Scan 1회 비용도 중요하지만 이 계획의 핵심은 후행 반복 접근이다.

[오답 분석] ②: 단일 탐색 Cost가 낮아도 선행 20만 건만큼 반복되면 전체 비용이 커진다.

[오답 분석] ④: Nested Loops는 후행 테이블을 반복 탐색하는 방식이다. 필요한 컬럼이 테이블에 있으면 테이블 액세스가 발생한다.

[관련 개념] Nested Loops Join, 후행 테이블 반복 접근, 랜덤 액세스, 조인 순서

8번

정답: ②

[핵심 포인트] 등치 조건 컬럼을 앞에 두고, 그 뒤에 정렬 컬럼을 순서대로 배치하면 Top-N 조회에 유리하다

[정답 이유] CUST_ID로 특정 고객 범위를 먼저 좁히고 그 안에서 TRADE_DT DESC, TRADE_ID DESC 순서대로 읽을 수 있다. 따라서 별도 정렬을 줄이고 필요한 50건에서 멈추는 처리를 기대할 수 있다.

[오답 분석] ①: 선두가 TRADE_DT이면 전체 고객의 최신 거래를 먼저 훑어 특정 고객 조회에 비효율적이다.

[오답 분석] ③: AMOUNT는 조건도 정렬 기준도 아니므로 이 요구에 직접 맞지 않는다.

[오답 분석] ④: CUST_ID 단일 인덱스는 고객 행을 찾을 뿐 후속 정렬 컬럼 순서를 보장하지 않는다.

[관련 개념] 인덱스 설계, 등치 조건 + ORDER BY, Top-N, 복합 인덱스 컬럼 순서

9번

정답: ①

[핵심 포인트] 복합 인덱스에서 범위 조건 뒤 후행 컬럼은 액세스 범위 축소 효과가 제한된다

[정답 이유] REGION은 선두 등치 조건으로 범위를 잡는다. 다음 컬럼 SALE_DT가 BETWEEN 범위 조건이므로 그 이후 PRODUCT_ID 등치는 인덱스 내부에서 필터 역할은 가능하지만 시작·끝 지점을 세밀하게 줄이기 어렵다.

[오답 분석] ②: 인덱스 컬럼 순서를 앞지를 수 없다. PRODUCT_ID는 세 번째 컬럼이다.

[오답 분석] ③: 컬럼이 인덱스에 모두 포함되어도 범위 조건 뒤 후행 컬럼 때문에 불필요한 인덱스 엔트리를 스캔할 수 있다.

[오답 분석] ④: 선두 컬럼 REGION 등치 조건은 여전히 인덱스 탐색에 사용할 수 있다.

[관련 개념] 인덱스 액세스 조건, 필터 조건, 범위 조건, 후행 컬럼 효율

10번

정답: ④

[핵심 포인트] 선두 컬럼 조건이 없을 때도 NDV가 낮은 선두 컬럼이면 Index Skip Scan이 검토될 수 있다

[정답 이유] JOIN_TYPE 조건이 없으므로 일반적인 Range Scan은 어렵다. 하지만 선두 컬럼 값 종류가 매우 적다면 옵티마이저가 JOIN_TYPE 값별로 하위 컬럼 범위를 탐색하는 Skip Scan을 고려할 수 있다. 다만 항상 유리한 것은 아니며 데이터 분포와 비용에 따라 결정된다.

[오답 분석] ①: 선두 컬럼 조건이 없으면 일반 Range Scan은 어렵지만 Skip Scan 등 예외적 접근이 있을 수 있다.

[오답 분석] ②: 후행 컬럼이 물리적으로 선두 컬럼으로 바뀌지는 않는다.

[오답 분석] ③: JOIN_DT는 인덱스에 포함되어 있으므로 Skip Scan이나 Full/FFS 등 인덱스 접근 방식에서 평가될 수 있다.

[관련 개념] Index Skip Scan, 선두 컬럼 누락, NDV, 비용 기반 선택

11번

정답: ③

[핵심 포인트] NL 조인은 선행 집합이 작고 후행 조인 키 인덱스가 있을 때 강점이 있다

[정답 이유] ORDERS가 최근 1시간 조건으로 300건까지 줄어든다면 이를 선행 집합으로 삼아 ORDER_ITEMS를 ORDER_ID 인덱스로 반복 탐색하는 방식이 효과적일 수 있다. 초기 응답도 전체 결과를 모두 만들기 전 반환하기 쉽다.

[오답 분석] ①: 테이블 크기만으로 선행 테이블을 정하지 않는다. 조건 적용 후 건수와 후행 인덱스가 중요하다.

[오답 분석] ②: 모든 조인 방식은 논리적으로 같은 결과를 낼 수 있다. Sort Merge Join만 정합성을 보장한다는 설명은 틀렸다.

[오답 분석] ④: NL 조인에서 후행 테이블 인덱스는 반복 탐색 비용을 좌우하는 핵심 요소다.

[관련 개념] NL 조인, 선행 집합, 후행 테이블 인덱스, 부분범위 처리

12번

정답: ②

[핵심 포인트] Hash Join은 대량 조인에서 작은 입력을 Build Input으로 삼아 큰 입력을 Probe하는 방식에 적합하다

[정답 이유] CUSTOMERS가 상대적으로 작고 ORDERS가 큰 대량 조인이다. Full Scan으로 양쪽을 읽은 뒤 작은 쪽을 해시 테이블로 만들고 큰 쪽을 스캔하며 매칭하면 수많은 인덱스 반복 탐색을 피할 수 있다.

[오답 분석] ①: 정렬된 입력을 병합하는 설명은 Sort Merge Join에 가깝다.

[오답 분석] ③: 선행 로우마다 후행 인덱스를 탐색하는 것은 NL 조인의 특징이다.

[오답 분석] ④: 인덱스가 있어도 비용상 Full Scan + Hash Join이 유리하면 Hash Join이 선택될 수 있다.

[관련 개념] Hash Join, Build Input, Probe Input, 대량 조인, Full Scan

13번

정답: ④

[핵심 포인트] Sort Merge Join은 정렬된 두 입력을 병합하며 등치 조인뿐 아니라 비등가 조인에서 검토될 수 있다

[정답 이유] BALANCE BETWEEN MIN_BALANCE AND MAX_BALANCE는 범위 기반 비등가 조인이다. Hash Join은 등치 조인에 주로 적합하고 NL 조인은 후행 탐색이 효율적일 때 유리하다. 비등가 범위 매칭에서는 정렬 후 병합하는 접근을 검토할 수 있다.

[오답 분석] ①: 선행 소량 + 후행 인덱스 우수는 NL 조인에 더 어울린다.

[오답 분석] ②: 등치 대량 조인과 해시 영역 확보는 Hash Join에 더 맞는 설명이다.

[오답 분석] ③: 기본키 등치 조건이면 NL 또는 Hash Join도 충분히 유리할 수 있으며 Sort Merge Join의 대표 근거가 아니다.

[관련 개념] Sort Merge Join, 비등가 조인, 정렬 비용, 조인 방식 선택 기준

14번

정답: ①

[핵심 포인트] 상관 스칼라 서브쿼리는 반복 수행 구조가 될 수 있어 조인 변환과 캐싱 여부를 확인해야 한다

[정답 이유] 서브쿼리가 E.DEPTNO를 참조하므로 외부 EMP 로우마다 DEPT를 조회하는 구조가 될 수 있다. 옵티마이저가 조인으로 변환하거나 스칼라 서브쿼리 캐싱을 활용하면 반복 비용을 줄일 수 있다.

[오답 분석] ②: 상관 서브쿼리는 외부 로우와 연결되어 반복 실행될 수 있다.

[오답 분석] ③: 상수 치환은 결과 의미를 바꾸므로 튜닝이 아니라 오류다.

[오답 분석] ④: 스칼라 서브쿼리도 상황에 따라 조인 변환 또는 별도 Row Source로 처리될 수 있다.

[관련 개념] 스칼라 서브쿼리, 서브쿼리 캐싱, 조인 변환, 반복 수행 비용

15번

정답: ②

[핵심 포인트] UNION과 UNION ALL의 핵심 차이는 중복 제거 작업이다

[정답 이유] UNION은 결과 합집합에서 중복을 제거해야 하므로 정렬 또는 해시 기반 중복 제거 비용이 추가될 수 있다. 보고서 요구가 중복 허용이라면 UNION ALL이 불필요한 작업을 줄인다.

[오답 분석] ①: 단순 연결은 UNION ALL의 의미다. UNION은 중복 제거가 포함된다.

[오답 분석] ③: UNION ALL은 중복 제거를 하지 않는다.

[오답 분석] ④: 서로 다른 테이블이어도 반환 컬럼 값이 중복될 수 있으며 옵티마이저가 항상 자동 변환할 수 있는 것은 아니다.

[관련 개념] UNION, UNION ALL, SORT UNIQUE, HASH UNIQUE, 소트 튜닝

16번

정답: ④

[핵심 포인트] COUNT STOPKEY는 Top-N 제한이 실행계획에 반영된 형태다

[정답 이유] STATUS 조건과 ORDER_DT DESC 순서가 인덱스와 맞고 COUNT STOPKEY가 있으므로 필요한 10건을 얻으면 더 이상 하위 연산에 로우를 요청하지 않을 수 있다. 따라서 전체 정렬 부담을 줄일 수 있다.

[오답 분석] ①: 여기의 COUNT STOPKEY는 집계 COUNT(*)가 아니라 ROWNUM 제한 처리와 관련된다.

[오답 분석] ②: 인덱스 순서가 맞지 않으면 Top-N Sort나 전체 정렬 부담이 여전히 발생할 수 있다.

[오답 분석] ③: STOPKEY는 반환 건수 제한일 뿐 조건 필터 자체를 제거하지 않는다.

[관련 개념] COUNT STOPKEY, Top-N, ROWNUM, 인덱스 정렬 순서

17번

정답: ③

[핵심 포인트] GROUP BY 소트 생략은 단순히 인덱스가 존재하는지가 아니라 입력 데이터의 정렬 순서가 그룹 키와 맞는지에 달려 있다

[정답 이유] 인덱스는 SALE_DT가 선두이고 CUST_ID가 후행이다. 날짜 범위를 스캔하면 CUST_ID가 전역적으로 정렬되어 있다고 보장하기 어렵다. 따라서 고객별 집계를 위해 해시 집계 또는 소트 집계가 필요할 수 있다.

[오답 분석] ①: 인덱스 존재만으로 GROUP BY 소트가 사라지는 것은 아니다.

[오답 분석] ②: SUM은 집계 함수이고 조인 방식과 직접 관련이 없다. 이 SQL에는 조인도 없다.

[오답 분석] ④: 후행 컬럼이라는 사실만으로 Nosort 집계가 보장되지 않는다. 그룹 키 순서가 입력 순서와 일치해야 한다.

[관련 개념] SORT GROUP BY, HASH GROUP BY, Nosort 집계, 인덱스 순서

18번

정답: ①

[핵심 포인트] DML 튜닝은 조회와 달리 변경 대상 행과 관련 인덱스 유지, 로그, Lock, Commit 비용을 함께 봐야 한다

[정답 이유] 대량 UPDATE는 대상 행 자체를 변경하고 관련 인덱스 엔트리도 유지해야 한다. Undo/Redo 생성, Lock 유지 시간, Commit 빈도, 배치 단위, 인덱스 필요성 등을 종합적으로 검토해야 한다.

[오답 분석] ②: UPDATE는 인덱스 컬럼이 변경되거나 행 위치/상태 관리가 필요할 때 인덱스 유지 비용이 발생한다.

[오답 분석] ③: 과도하게 잦은 Commit은 동기화와 로그 기록 부담을 키우며 전체 성능을 악화시킬 수 있다.

[오답 분석] ④: 조건 인덱스가 대상 검색을 도울 수는 있지만 실제 행 변경과 로그 생성 비용을 제거하지는 못한다.

[관련 개념] DML 튜닝, 인덱스 유지 비용, Undo/Redo, Commit 전략, Array Processing

19번

정답: ②

[핵심 포인트] Direct Path Insert는 대량 적재에 유리할 수 있지만 인덱스, 로깅, Lock 제약을 함께 고려해야 한다

[정답 이유] APPEND 방식은 일반적인 버퍼 캐시 경유 Insert와 다르게 세그먼트 고수위 영역에 직접 적재하는 성격이 있어 대량 입력에 유리할 수 있다. 다만 인덱스 유지, 로깅 모드, 병렬 DML, Lock 영향은 별도로 검토해야 한다.

[오답 분석] ①: 인덱스 유지 비용이나 Lock 제약이 0이 되는 것은 아니다.

[오답 분석] ③: 소량 OLTP 단건 입력은 일반 Insert가 더 적합한 경우가 많다.

[오답 분석] ④: NOLOGGING, FORCE LOGGING, 복구 정책 등에 따라 Redo/Undo 발생 양상은 달라지며 완전 0이라고 일반화할 수 없다.

[관련 개념] Direct Path Insert, APPEND 힌트, 대량 적재, 로깅, 인덱스 유지

20번

정답: ④

[핵심 포인트] 대량 DELETE는 파티션 키와 삭제 범위가 맞을 때 파티션 단위 작업으로 비용을 크게 줄일 수 있다

[정답 이유] 삭제 범위가 단일 파티션과 정확히 대응하면 행 단위 DELETE 대신 파티션 TRUNCATE, DROP, EXCHANGE 같은 메타데이터 중심 작업을 검토할 수 있다. 이는 Undo/Redo와 인덱스 유지 부담을 줄이는 데 유리할 수 있다.

[오답 분석] ①: 파티션 키 조건이 명확하면 Partition Pruning으로 대상 파티션만 접근할 수 있다.

[오답 분석] ②: Local Index가 관리 범위를 줄여도 행 단위 DELETE의 Undo/Redo가 자동으로 0이 되는 것은 아니다.

[오답 분석] ③: Global Index는 파티션 작업 후 unusable 또는 유지 비용 이슈가 생길 수 있다.

[관련 개념] Partition Pruning, 파티션 단위 DML, Local/Global Index, 대량 DELETE

21번

정답: ③

[핵심 포인트] MVCC는 읽기 일관성을 위해 쿼리 시점에 맞는 버전을 제공하고, 일반 SELECT와 DML 차단을 줄인다

[정답 이유] MVCC에서는 일반 SELECT가 UPDATE 중인 행을 읽을 때 쓰기 작업을 무조건 기다리기보다 쿼리 시점의 일관된 이미지를 구성한다. 이 과정에서 Undo 정보를 이용해 변경 전 버전을 볼 수 있다.

[오답 분석] ①: MVCC의 일반 읽기는 공유 Lock으로 UPDATE를 항상 대기시키는 모델과 다르다.

[오답 분석] ②: UPDATE는 쓰기 충돌 제어를 위해 Lock을 사용한다. MVCC가 모든 Lock을 제거하는 것은 아니다.

[오답 분석] ④: 일반적인 읽기 일관성 모델은 커밋되지 않은 변경을 읽지 않는다.

[관련 개념] MVCC, Consistent Read, Undo, Lock, Dirty Read

22번

정답: ①

[핵심 포인트] 비용 기반 옵티마이저는 선택도와 카디널리티 추정에 따라 조인 순서와 접근 방식을 결정한다

[정답 이유] 실제로는 많은 로우가 반환되는데 옵티마이저가 소량으로 추정하면 인덱스 Range Scan과 NL 조인처럼 소량 처리에 유리한 계획을 선택할 수 있다. 실행 시에는 반복 랜덤 액세스가 폭증한다.

[오답 분석] ②: 통계정보는 선택도, 카디널리티, 비용 계산에 직접 사용된다.

[오답 분석] ③: 과소평가 시 대개 소량 처리 계획을 선택할 가능성이 커지며 항상 병렬 Full Scan으로 가지 않는다.

[오답 분석] ④: 카디널리티 오류는 조인 순서, 조인 방식, 인덱스 사용 여부 등 실행계획 전반에 영향을 준다.

[관련 개념] 통계정보, 선택도, 카디널리티, 비용 기반 옵티마이저

23번

정답: ④

[핵심 포인트] 힌트는 옵티마이저에게 주는 지시/권고지만, 정확한 대상 지정과 적용 가능 조건이 맞아야 한다

[정답 이유] SQL에서 ORDERS 테이블의 별칭은 O인데 힌트는 orders를 대상으로 지정했다. 힌트 대상이 잘못되거나 쿼리 변환 후 대상이 달라지거나 적용 불가능한 조건이면 힌트가 무시될 수 있다.

[오답 분석] ①: 힌트 문법이나 대상이 맞지 않으면 적용되지 않을 수 있다.

[오답 분석] ②: 힌트는 보통 쿼리 블록 안의 별칭을 정확히 지정해야 한다.

[오답 분석] ③: USE_NL은 조인 방식에 영향을 주는 힌트이며 Fetch Call 조정 명령이 아니다.

[관련 개념] 옵티마이저 힌트, 테이블 별칭, 쿼리 블록, 힌트 무시 원인

24번

정답: ②

[핵심 포인트] EXISTS 상관 서브쿼리는 경우에 따라 세미 조인으로 변환되어 반복 수행을 줄일 수 있다

[정답 이유] EXISTS는 존재 여부만 확인하므로 옵티마이저가 조건을 만족하면 세미 조인으로 변환할 수 있다. 그러면 외부 로우마다 서브쿼리를 반복하는 형태보다 다양한 조인 순서와 조인 방식 선택이 가능하다.

[오답 분석] ①: 실행 형태는 옵티마이저 변환에 따라 달라질 수 있다. 항상 반복 실행으로 고정되지 않는다.

[오답 분석] ③: NOT EXISTS는 존재하지 않음을 찾는 조건이므로 의미가 반대다.

[오답 분석] ④: Unnesting은 EXISTS, IN 등 여러 상관 서브쿼리에도 적용될 수 있다.

[관련 개념] Subquery Unnesting, Semi Join, EXISTS, 옵티마이저 쿼리 변환

25번

정답: ③

[핵심 포인트] 선택도 추정은 값 분포와 컬럼 간 상관관계에 크게 영향을 받는다

[정답 이유] 단일 컬럼 통계만으로는 IS_ACTIVE와 STATUS의 조합 분포를 정확히 추정하기 어렵다. 특정 값 쓸림은 히스토그램으로, 컬럼 조합의 상관관계는 확장 통계로 보완할 수 있는지 검토한다.

[오답 분석] ①: Optimizer Mode 변경은 목표를 바꾸는 것이지 통계 분포 정보 부족을 직접 해결하지 않는다.

[오답 분석] ②: 힌트 제거만으로 실제 행 수를 정확히 아는 것은 아니다. 정확한 통계가 필요하다.

[오답 분석] ④: SQL 텍스트 공유와 카디널리티 추정은 다른 문제다. 대소문자는 분포 추정을 해결하지 않는다.

[관련 개념] 히스토그램, 확장 통계, 컬럼 상관관계, 카디널리티 추정

SQLP 대비 「친절한 SQL 튜닝」 기반 모의문제 25제

SQL 고급활용 및 튜닝 중심 | 유형 기반 자체 제작 문제 | 문제지 + 정답지

구분	내용
출제 범위	첨부 PDF 「친절한 SQL 튜닝」의 SQL 처리/I/O, 인덱스, 조인, 서브쿼리, 소트, DML, 옵티마이저 관련 범위
문항 수	총 25문항
난이도	하 5문항, 중 12문항, 상 8문항
문제 유형	개념 적용, 실행계획 해석, 인덱스/조인/소트 튜닝 판단, 힌트와 서브쿼리 변환 판단
주의	실제 기출문제를 복원하거나 문장을 베끼지 않은 유형 기반 자체 제작 문제

자체 검수 결과

1. 책 목차에서 확인되는 SQL 튜닝 범위 안의 주제만 사용했다.
2. 실제 기출문제 문장 또는 공개 복원 문장을 그대로 사용하지 않았다.
3. 각 문항의 정답은 하나로 수렴하도록 조건과 전제를 명시했다.
4. 보기는 모두 관련 개념의 오해 가능성을 반영해 난이도 편차가 과도하지 않게 조정했다.
5. 실행계획, 인덱스 구성, 조인 방식, 서브쿼리/뷰/힌트, 소트 판단 문항을 충분히 포함했다.
6. 정답 분포는 1번 6개, 2번 7개, 3번 6개, 4번 6개로 한 번호에 과도하게 몰리지 않도록 구성했다.

문제지

정답은 표시하지 않았습니다. 각 문항은 SQLP 3과목 튜닝형 판단 문제에 맞춰 작성했습니다.

문제 1.

SQL 파싱과 공유에 대한 설명으로 가장 적절한 것은?

- 1) SQL 텍스트가 같더라도 바인드 변수를 사용하면 항상 하드 파싱이 발생한다.
- 2) 바인드 변수를 사용하면 SQL 텍스트 공유 가능성이 높아져 하드 파싱 부담을 줄이는 데 유리하다.
- 3) 소프트 파싱은 실행계획을 새로 만들지 않으므로 라이브러리 캐시 탐색 비용도 전혀 발생하지 않는다.
- 4) 옵티마이저 힌트는 실행계획을 고정하므로 SQL 공유와 파싱 비용에는 영향을 주지 않는다.

문제 2.

데이터베이스 I/O에 대한 설명으로 가장 적절한 것은?

- 1) 논리적 I/O는 항상 디스크에서 블록을 읽는 작업을 의미한다.
- 2) 물리적 I/O가 0이면 SQL 성능 문제는 발생하지 않는다.
- 3) 인덱스 Range Scan은 항상 Table Full Scan보다 적은 블록을 읽는다.
- 4) 논리적 I/O는 버퍼 캐시에서 블록을 읽는 작업까지 포함하며, 튜닝의 핵심은 불필요한 블록 접근 자체를 줄이는 것이다.

문제 3.

아래 인덱스가 있을 때 일반적인 Index Range Scan을 기대하기 가장 어려운 조건은?

인덱스: IDX_ORD_01(주문일자, 고객번호, 주문상태)

- A. WHERE 고객번호 = :cust_no
- B. WHERE 주문일자 BETWEEN :d1 AND :d2
- C. WHERE 주문일자 = :d AND 고객번호 = :cust_no
- D. WHERE 주문일자 >= :d1 AND 주문일자 < :d2 AND 주문상태 = :st

- 1) A
- 2) B
- 3) C
- 4) D

문제 4.

NL 조인에 대한 설명으로 가장 적절한 것은?

- 1) 두 입력 집합을 모두 조인 컬럼 순서로 정렬한 뒤 병합한다.
- 2) 작은 집합을 메모리에 해시 테이블로 만든 뒤 큰 집합을 탐색한다.
- 3) 선행 집합의 각 로우마다 후행 집합을 반복 탐색하므로, 선행 집합 크기와 후행 집합 접근 효율이 성능을 좌우한다.
- 4) 항상 후행 테이블을 한 번만 읽기 때문에 대량 조인에서도 가장 안정적이다.

문제 5.

UNION과 UNION ALL에 대한 설명으로 가장 적절한 것은?

- 1) UNION ALL은 중복 제거를 수행하므로 항상 소트가 필요하다.
- 2) 중복 제거가 필요 없는 경우 UNION보다 UNION ALL이 불필요한 소트 또는 중복 제거 부담을 줄이는 데 유리하다.
- 3) UNION은 두 집합을 단순히 붙이기만 하므로 UNION ALL보다 항상 빠르다.
- 4) UNION과 UNION ALL은 결과 중복 여부만 다르고 실행 부하는 항상 동일하다.

문제 6.

아래 조건과 인덱스에 대한 판단으로 가장 적절한 것은?

인덱스: IDX_SALES_01(부서코드, 매출일자, 상태코드)

```
SELECT *
FROM   매출
WHERE  부서코드 = :dept
AND    매출일자 BETWEEN :d1 AND :d2
AND    상태코드 = :st;
```

- 1) 세 조건 모두 인덱스 액세스 조건으로 사용되어 스캔 범위가 정확히 상태코드까지 좁혀진다.
- 2) 상태코드 조건이 등치 조건이므로 매출일자 범위 조건보다 먼저 액세스된다.
- 3) 매출일자에 BETWEEN이 있으므로 부서코드 조건도 필터 조건으로 밀린다.
- 4) 부서코드와 매출일자는 스캔 범위를 결정하는 데 쓰이고, 상태코드는 매출일자 범위 안에서 필터로 평가될 가능성이 크다.

문제 7.

클러스터링 팩터가 매우 나쁜 인덱스를 이용해 넓은 범위를 조회할 때 발생할 수 있는 현상으로 가장 적절한 것은?

- 1) 인덱스 엔트리 순서와 테이블 블록 배치가 맞지 않아 테이블 랜덤 액세스가 급증하고, Full Scan보다 불리해질 수 있다.
- 2) 클러스터링 팩터가 나쁘면 인덱스 리프 블록 수가 자동으로 감소한다.
- 3) 클러스터링 팩터는 인덱스 Unique Scan에만 영향을 주고 Range Scan에는 영향을 주지 않는다.
- 4) 클러스터링 팩터가 나쁠수록 부분범위 처리는 항상 빨라진다.

문제 8.

인덱스 컬럼 추가에 대한 설명으로 가장 적절한 것은?

- 1) SELECT 절에 있는 모든 컬럼을 인덱스에 추가하면 DML 비용 없이 항상 성능이 좋아진다.
- 2) 인덱스 컬럼을 추가하면 테이블 액세스 횟수는 줄 수 있지만 인덱스 스캔량은 절대 늘지 않는다.
- 3) 필요 컬럼을 인덱스에 포함하면 테이블 액세스를 줄이거나 인덱스만 읽고 처리할 수 있지만, 인덱스 크기와 DML 유지 비용 증가를 함께 고려해야 한다.
- 4) 인덱스 컬럼 추가는 ORDER BY 절의 소트 생략에는 영향을 줄 수 없다.

문제 9.

아래 Top-N 조회를 가장 효율적으로 수행하기 위한 판단으로 가장 적절한 것은?

```
SELECT *
FROM (
  SELECT 주문번호, 주문일자, 주문금액
  FROM   주문
  WHERE  고객번호 = :cust_no
  ORDER BY 주문일자 DESC
)
WHERE ROWNUM <= 10;
```

- 1) ORDER BY가 있으므로 어떤 인덱스가 있어도 전체 정렬은 피할 수 없다.
- 2) 인덱스가 (고객번호, 주문일자 DESC) 순서로 구성되어 있으면 정렬을 생략하고 앞쪽 10건만 읽는 방식이 가능하다.
- 3) ROWNUM 조건은 정렬 전에 항상 적용되므로 인덱스 설계와 무관하다.
- 4) Top-N 쿼리는 반드시 Hash Join을 사용해야 한다.

문제 10.

아래 SQL에서 고객을 선행 집합으로 두고 주문을 NL 조인으로 탐색하도록 유도하려는 경우, 가장 적절한 힌트 조합은?

```
SELECT /* 힌트 위치 */ c.고객명, o.주문번호
FROM   고객 c, 주문 o
WHERE  c.고객번호 = o.고객번호
AND    c.고객등급 = :grade
AND    o.주문일자 >= :d;
```

- 1) /*+ USE_HASH(o) FULL(o) */
- 2) /*+ ORDERED USE_MERGE(o) */
- 3) /*+ LEADING(o) USE_NL(c) */
- 4) /*+ LEADING(c) USE_NL(o) */

문제 11.

소트 머지 조인에 대한 설명으로 가장 적절한 것은?

- 1) 두 입력 집합을 조인 키 기준으로 정렬한 뒤 순차적으로 비교하므로, 이미 정렬된 입력을 활용할 수 있으면 유리할 수 있다.
- 2) 후행 테이블 인덱스를 선행 로우마다 반복 탐색하는 방식이다.
- 3) 항상 해시 테이블을 생성하므로 PGA가 부족하면 해시 파티션이 발생한다.
- 4) 동등 조인에서만 가능하고 범위 조인에는 사용할 수 없다.

문제 12.

스칼라 서브쿼리 캐싱 효과에 대한 설명으로 가장 적절한 것은?

- 1) 상관 서브쿼리는 항상 한 번만 실행되므로 캐싱 여부는 의미가 없다.
- 2) 외부 집합의 상관값이 모두 서로 다를수록 캐싱 효과가 커진다.
- 3) 외부 집합에서 같은 상관값이 반복될 때, 같은 입력값에 대한 스칼라 서브쿼리 결과를 재사용할 수 있어 반복 실행 부담을 줄일 수 있다.
- 4) 스칼라 서브쿼리 캐싱은 조인 결과의 중복을 자동으로 제거한다.

문제 13.

뷰와 조인에 대한 설명으로 가장 적절한 것은?

- 1) 인라인 뷰는 항상 별도 임시 테이블로 물리화되므로 외부 조건이 내부로 들어갈 수 없다.
- 2) 옵티마이저는 의미가 보존되는 경우 뷰 머징 또는 조인 조건/필터 조건의 내부 전달을 통해 불필요한 처리량을 줄일 수 있다.
- 3) GROUP BY, DISTINCT, ROWNUM이 포함된 뷰도 항상 완전히 병합된다.
- 4) 뷰를 사용하면 조인 순서와 조인 방법을 절대 변경할 수 없다.

문제 14.

Index Skip Scan을 검토할 만한 상황으로 가장 적절한 것은?

- 1) 복합 인덱스의 모든 선두 컬럼에 등치 조건이 있어 일반 Range Scan이 가능한 경우
- 2) 선두 컬럼의 값 종류가 매우 많고 후행 컬럼 조건의 선택도가 낮은 경우
- 3) 테이블 전체를 읽는 것이 항상 필요한 대량 집계인 경우
- 4) 복합 인덱스의 선두 컬럼 조건은 없지만 선두 컬럼의 값 종류가 적고 후행 컬럼 조건이 선택적인 경우

문제 15.

GROUP BY 또는 ORDER BY 소트 생략에 대한 설명으로 가장 적절한 것은?

- 1) 인덱스 컬럼 순서가 GROUP BY 또는 ORDER BY 요구 순서와 맞고 조건절이 그 순서를 깨지 않으면 소트 연산을 생략할 수 있다.
- 2) GROUP BY는 해시 연산만 사용하므로 인덱스 순서와 무관하다.
- 3) ORDER BY 컬럼에 인덱스가 하나라도 있으면 모든 소트가 생략된다.
- 4) 컬럼에 함수 가공이 있더라도 일반 인덱스 순서를 그대로 사용할 수 있다.

문제 16.

Direct Path I/O 활용에 대한 설명으로 가장 적절한 것은?

- 1) Direct Path I/O는 항상 버퍼 캐시를 더 많이 사용하게 만들어 OLTP 단건 조회에 최적화된다.
- 2) Direct Path Insert는 모든 제약조건과 인덱스 유지 비용을 완전히 제거한다.
- 3) 대량 처리에서 버퍼 캐시 경우 부담을 줄이고 직접 경로로 읽거나 쓸 수 있지만, 적용 가능 조건과 부작용을 함께 검토해야 한다.
- 4) Direct Path I/O는 실행계획과 무관하며 SQL 작성 방식으로는 영향을 줄 수 없다.

문제 17.

옵티마이저 통계정보와 비용 계산에 대한 설명으로 가장 적절한 것은?

- 1) 옵티마이저는 항상 실제 실행 시점의 정확한 로우 수를 알고 실행계획을 만든다.
- 2) 선택도와 카디널리티 추정은 비용 계산의 핵심이며, 통계정보가 부정확하면 조인 순서나 접근 경로 선택이 나빠질 수 있다.
- 3) 힌트를 사용하면 통계정보는 전혀 필요하지 않다.
- 4) 카디널리티 추정 오류는 소트 비용에는 영향을 주지 않고 조인 비용에만 영향을 준다.

문제 18.

아래 SQL을 위해 새 인덱스를 하나만 추가할 수 있다. 조건 선택도와 정렬 생략을 함께 고려할 때 가장 적절한 인덱스는?

```
-- 주문: 5천만 건
-- 고객번호 조건: 평균 0.02% 선택
-- 상태코드 = '완료': 해당 고객 주문 중 약 40%
-- 조회 기간: 해당 고객 주문 중 약 30%
```

```
SELECT 주문번호, 주문일자, 주문금액
FROM 주문
WHERE 고객번호 = :cust_no
AND 상태코드 = '완료'
AND 주문일자 BETWEEN :d1 AND :d2
ORDER BY 주문일자;
```

- 1) (주문일자, 고객번호, 상태코드)
- 2) (상태코드, 주문일자, 고객번호)
- 3) (고객번호, 주문일자, 상태코드)
- 4) (고객번호, 상태코드, 주문일자, 주문번호, 주문금액)

문제 19.

아래 SQL과 실행계획을 보고 가장 타당한 튜닝 판단을 고르시오.

```
SELECT c.고객명, SUM(o.주문금액)
FROM   고객 c, 주문 o
WHERE  c.고객번호 = o.고객번호
AND    c.고객등급 = 'VIP'
AND    o.주문일자 = :yyyymmdd
GROUP BY c.고객명;
```

```
-- 통계
-- 고객: 5,000,000건, VIP: 1,000,000건
-- 주문: 100,000,000건, 특정 일자 주문: 50,000건
-- 현재 인덱스: 고객(고객등급, 고객번호), 주문(고객번호, 주문일자)
```

Id	Operation	Name	Rows	Note
0	SELECT STATEMENT			
1	HASH GROUP BY		50K	
2	NESTED LOOPS		50K	1,000,000 probes
3	INDEX RANGE SCAN	고객_X1	1M	고객등급=VIP
4	TABLE ACCESS BY ROWID	주문	1	
5	INDEX RANGE SCAN	주문_X1	1	고객번호, 주문일자

- 1) VIP 고객 100만 건을 선행으로 두고 주문을 100만 번 탐색하므로, 주문일자 선택도를 활용해 주문을 먼저 줄이는 접근 경로 또는 조인 방식 변경을 검토한다.
- 2) 주문_X1을 사용했으므로 더 이상 개선할 수 없다.
- 3) GROUP BY가 있으므로 NL 조인이 항상 최선이다.
- 4) 고객등급 조건이 있으므로 고객을 선행으로 두는 현재 계획이 항상 최적이다.

문제 20.

아래 OR 조건 SQL에 대한 튜닝 판단으로 가장 적절한 것은?

```
-- 인덱스
-- 주문_X1(고객번호, 주문일자)
-- 주문_X2(상태코드, 주문일자)
```

```
SELECT 주문번호, 고객번호, 상태코드, 주문일자
FROM   주문
WHERE  주문일자 >= :d
AND    (고객번호 = :cust_no OR 상태코드 = '지연');
```

- 1) OR 조건이 있으므로 어떤 인덱스도 사용할 수 없다.
- 2) 두 인덱스를 모두 사용하려면 반드시 하나의 복합 인덱스로 합쳐야 한다.
- 3) 각 조건을 분리해 UNION ALL 형태로 재작성할 수 있지만, 두 조건을 동시에 만족하는 주문이 있으면 중복 제거 또는 반대 조건 추가를 고려해야 한다.
- 4) UNION으로 바꾸면 항상 정렬 부하가 없어서 가장 빠르다.

문제 21.

인덱스 Range Scan 결과가 정렬되어 나올 것이라고 기대하던 SQL에서, Batch I/O 기능 적용 후 출력 순서가 달라졌다. 가장 적절한 설명은?

- 1) 인덱스가 깨졌기 때문에 즉시 재생성해야 한다.
- 2) 테이블 블록 방문 효율을 높이기 위해 rowid 방문 순서가 조정될 수 있으므로, SQL 결과 순서를 보장하려면 명시적인 ORDER BY가 필요하다.
- 3) Index Range Scan은 어떤 실행 방식에서도 결과 순서를 절대 바꾸지 않는다.
- 4) ORDER BY가 없어도 SELECT 절 컬럼 순서가 결과 정렬 순서를 결정한다.

문제 22.

해시 조인에 대한 설명으로 가장 적절한 것은?

- 1) 해시 조인은 선행 집합의 모든로우마다 후행 테이블 인덱스를 반복 탐색한다.
- 2) 해시 조인은 두 입력 집합을 반드시 조인 키 순서로 정렬한 뒤 비교한다.
- 3) Build Input은 항상 큰 테이블이어야 해시 충돌이 줄어든다.
- 4) 필터 적용 후 더 작은 입력을 Build Input으로 선택하는 것이 일반적으로 유리하며, 메모리가 부족하면 파티션 분할 또는 TEMP I/O가 발생할 수 있다.

문제 23.

아래 EXISTS 서브쿼리를 조인으로 변환하지 않고 필터 방식으로 유지하면서, 가능하면 조인 전에 먼저 평가하게 하려는 힌트 의도로 가장 적절한 것은?

```
SELECT /* 힌트 위치 */ o.주문번호, o.주문금액
FROM   주문 o, 고객 c
WHERE  o.고객번호 = c.고객번호
AND    c.고객등급 = :grade
AND    EXISTS (
    SELECT 1
    FROM    배송 d
    WHERE   d.주문번호 = o.주문번호
    AND     d.배송상태 = '완료'
);
```

- 1) NO_UNNEST로 서브쿼리 변환을 막고, PUSH_SUBQ로 서브쿼리 필터 평가를 앞당기는 의도이다.
- 2) USE_HASH로 EXISTS 서브쿼리를 항상 스칼라 서브쿼리로 변환하는 의도이다.
- 3) ORDERED는 서브쿼리 실행 시점을 항상 가장 마지막으로 고정한다.
- 4) NO_MERGE는 EXISTS 서브쿼리를 반드시 조인으로 변환한다.

문제 24.

아래 스칼라 서브쿼리 SQL을 조인으로 바꾸려 할 때 가장 주의해야 할 점은?

```
SELECT c.고객번호,
       c.고객명,
       (SELECT o.주문금액
        FROM 주문 o
        WHERE o.고객번호 = c.고객번호
        AND    o.주문일자 = (SELECT MAX(o2.주문일자)
                             FROM 주문 o2
                             WHERE o2.고객번호 = c.고객번호)) AS 최근주문금액
FROM   고객 c;
```

- 1) 스칼라 서브쿼리는 항상 0건만 반환하므로 조인으로 바꾸면 결과가 반드시 늘어난다.
- 2) MAX 함수가 있으므로 고객별 결과가 자동으로 1건으로 보장된다.
- 3) 같은 고객의 최신 주문일자에 주문이 여러 건이면 스칼라 서브쿼리는 다중 행 오류 또는 의미 불명확 문제가 생길 수 있으므로, 조인 전환 시에도 단일 행 보장 기준을 명확히 해야 한다.
- 4) 스칼라 서브쿼리를 조인으로 바꾸면 어떤 경우에도 중복 가능성이 사라진다.

문제 25.

아래 분석함수 기반 Top-N SQL의 소트 부하를 줄이기 위한 판단으로 가장 적절한 것은?

```
SELECT *
FROM (
  SELECT 부서코드, 사원번호, 급여,
         ROW_NUMBER() OVER(PARTITION BY 부서코드 ORDER BY 급여 DESC) AS rn
  FROM   사원
  WHERE  재직상태 = 'Y'
)
WHERE rn <= 3;
```

- 1) 분석함수는 소트를 전혀 사용하지 않으므로 인덱스와 무관하다.
- 2) 재직상태 필터와 PARTITION BY/ORDER BY 순서에 맞는 인덱스가 있으면 소트 부담을 줄일 수 있으나, 조건과 인덱스 순서가 맞지 않으면 Window Sort가 발생할 수 있다.
- 3) PARTITION BY가 있으면 항상 해시 조인으로 변환된다.
- 4) 부서별 Top-N은 ROWNUM만 붙이면 정렬 없이 항상 정확하게 구할 수 있다.

정답지 및 해설

아래 해설의 페이지 번호는 책 목차에서 확인한 번호를 기준으로 표기했습니다. 세부 항목 위치가 확실하지 않은 경우 “페이지 번호 확실하지 않음”으로 표시했습니다.

문항 번호: 1 정답: 2

출제 의도: 하드 파싱/소프트 파싱, SQL 공유, 바인드 변수의 기본 효과를 구분하는지 확인한다.

해설: 정답은 2번이다. 바인드 변수는 SQL 텍스트를 재사용하기 쉽게 만들어 같은 SQL에 대해 기존 커서와 실행계획을 공유할 가능성을 높인다. 따라서 하드 파싱과 최적화 비용을 줄이는 데 유리하다.

오답 선택지 해설: 1번은 반대다. 바인드 변수는 일반적으로 하드 파싱을 줄이기 위한 수단이다. 3번은 과장이다. 소프트 파싱도 커서 탐색과 권한 확인 등 비용이 남는다. 4번은 틀렸다. 힌트도 SQL 텍스트의 일부로 공유 여부와 실행계획 선택에 영향을 준다.

책 근거: 1.1 SQL 파싱과 최적화, 1.2 SQL 공유 및 재사용 - 목차 기준 p.17, p.29.

문항 번호: 2 정답: 4

출제 의도: 논리적 I/O와 물리적 I/O를 구분하고, 캐시 적중률보다 블록 접근량 감소가 중요하다는 관점을 확인한다.

해설: 정답은 4번이다. SQL 튜닝은 단순히 디스크 읽기를 줄이는 것을 넘어, 버퍼 캐시에서 반복해서 읽는 논리적 I/O까지 줄이는 방향으로 판단해야 한다.

오답 선택지 해설: 1번은 물리적 I/O와 혼동한 설명이다. 2번은 논리적 I/O와 CPU 소모를 무시한다. 3번은 조회 범위가 넓거나 클러스터링 팩터가 나쁘면 인덱스 경유가 더 불리할 수 있어 틀렸다.

책 근거: 1.3 데이터 저장 구조 및 I/O 메커니즘, 블록 단위 I/O, 논리적 I/O vs 물리적 I/O, Table Full Scan vs Index Range Scan - 목차 기준 p.43, p.48, p.60.

문항 번호: 3 정답: 1

출제 의도: 복합 인덱스에서 선두 컬럼 조건이 Range Scan 가능성을 좌우한다는 점을 확인한다.

해설: 정답은 1번이다. 선두 컬럼인 주문일자 조건이 없고 두 번째 컬럼인 고객번호만 조건으로 주어졌으므로 일반적인 Index Range Scan으로 시작하기 어렵다. 단, 별도의 Skip Scan 가능성은 선두 컬럼의 NDV 등 조건을 따로 판단해야 한다.

오답 선택지 해설: 2번은 선두 컬럼 범위 조건이 있으므로 Range Scan이 가능하다. 3번은 선두 컬럼과 다음 컬럼의 등치 조건이 모두 있어 가장 전형적이다. 4번은 주문일자 범위로 접근하고 주문상태는 뒤쪽 필터로 쓰일 수 있다.

책 근거: 2.2.2 인덱스를 Range Scan 할 수 없는 이유, 2.3.1 Index Range Scan - 목차 기준 p.86, p.110.

문항 번호: 4 정답: 3

출제 의도: NL 조인의 반복 탐색 구조와 튜닝 포인트를 이해했는지 확인한다.

해설: 정답은 3번이다. NL 조인은 선행 로우마다 후행 테이블을 탐색하는 방식이다. 그래서 선행 집합을 작게 만들고 후행 집합에 적절한 인덱스 접근 경로를 갖추는 것이 중요하다.

오답 선택지 해설: 1번은 소트 머지 조인 설명이다. 2번은 해시 조인 설명이다. 4번은 틀렸다. 후행 테이블은 선행 로우 수만큼 반복 접근될 수 있다.

책 근거: 4.1 NL 조인, 4.1.1 기본 메커니즘 - 목차 기준 p.255.

문항 번호: 5 정답: 2

출제 의도: 소트가 발생하지 않도록 SQL을 작성하는 기본 패턴을 확인한다.

해설: 정답은 2번이다. UNION은 중복 제거가 필요하므로 정렬 또는 해시 기반 중복 제거 부하가 생길 수 있다. 중복 제거가 의미상 필요 없다면 UNION ALL이 일반적으로 유리하다.

오답 선택지 해설: 1번은 UNION의 특징을 UNION ALL에 잘못 붙였다. 3번은 반대다. 4번은 실행 부하 차이를 무시한다.

책 근거: 5.2.1 Union vs Union All - 목차 기준 p.343.

문항 번호: 6 정답: 4

출제 의도: 복합 인덱스에서 등치 조건, 범위 조건, 후행 컬럼 조건의 역할을 구분하게 한다.

해설: 정답은 4번이다. 선두 컬럼 부서코드는 등치 조건으로 범위를 좁히고, 다음 컬럼 매출일자는 범위 조건으로 스캔 범위를 정한다. 그 뒤 컬럼인 상태코드는 범위 스캔 내부에서 필터 조건으로 평가될 가능성이 크다.

오답 선택지 해설: 1번은 범위 조건 뒤 컬럼의 액세스 효과를 과대평가했다. 2번은 인덱스 컬럼 순서를 무시했다. 3번은 선두 등치 조건의 액세스 효과를 부정하므로 틀렸다.

책 근거: 3.3.3 액세스 조건과 필터 조건, 3.3.4 비교 연산자 종류와 컬럼 순서에 따른 군집성 - 목차 기준 p.185, p.186.

문항 번호: 7 정답: 1

출제 의도: 인덱스 손익분기점과 테이블 랜덤 액세스 비용을 연결해 판단하게 한다.

해설: 정답은 1번이다. 클러스터링 팩터가 나쁘면 인덱스 순서대로 읽은 rowid가 테이블 여러 블록으로 흩어져 테이블 액세스 비용이 커진다. 조회 범위가 넓으면 Full Scan이 더 유리할 수 있다.

오답 선택지 해설: 2번은 관련이 없다. 3번은 틀렸다. Range Scan에서 특히 테이블 랜덤 액세스 비용이 중요하다. 4번은 부분범위 처리와 클러스터링 팩터를 단정적으로 연결한 오류다.

책 근거: 3.1.2 인덱스 클러스터링 팩터, 3.1.3 인덱스 손익분기점 - 목차 기준 p.136, p.138.

문항 번호: 8 정답: 3

출제 의도: 커버링 인덱스의 이점과 부작용을 균형 있게 판단하게 한다.

해설: 정답은 3번이다. 인덱스에 필요한 컬럼을 추가하면 테이블 랜덤 액세스를 줄일 수 있고 경우에 따라 인덱스만 읽고 처리할 수 있다. 하지만 인덱스가 커지고 DML 유지 비용이 증가할 수 있다.

오답 선택지 해설: 1번은 DML 비용과 저장공간을 무시한다. 2번은 인덱스 크기 증가에 따른 비용을 무시한다. 4번은 틀렸다. 컬럼 순서가 맞으면 소트 생략에도 영향을 줄 수 있다.

책 근거: 3.1.4 인덱스 컬럼 추가, 3.1.5 인덱스만 읽고 처리, 3.4.5 소트 연산을 생략하기 위한 컬럼 추가 - 목차 기준 p.145, p.149, p.238.

문항 번호: 9 정답: 2

출제 의도: Top-N에서 인덱스 순서와 Stopkey 개념을 적용할 수 있는지 확인한다.

해설: 정답은 2번이다. 고객번호 등치 조건 뒤에 주문일자 내림차순 인덱스가 있으면 원하는 정렬 순서대로 읽으면서 필요한 10건에서 멈출 수 있다.

오답 선택지 해설: 1번은 인덱스 정렬 순서 활용 가능성을 무시한다. 3번은 서브쿼리 내부 정렬 뒤 바깥 ROWNUM을 적용하는 구조를 오해했다. 4번은 조인 방식과 무관한 단정이다.

책 근거: 5.3.2 Top N 쿼리, 2.2.4 인덱스를 이용한 소트 연산 생략 - 목차 기준 p.352, p.94.

문항 번호: 10 정답: 4

출제 의도: 조인 순서 제어와 조인 메소드 제어 힌트를 함께 판단하게 한다.

해설: 정답은 4번이다. LEADING(c)는 고객을 선행 집합으로 두도록 유도하고, USE_NL(o)는 주문을 후행 집합으로 NL 조인하도록 유도한다.

오답 선택지 해설: 1번은 해시 조인을 유도한다. 2번은 소트 머지 조인을 유도한다. 3번은 주문을 선행 집합으로 두는 반대 방향의 힌트다.

책 근거: 4.1.2 NL 조인 실행계획 제어, 1.1.5 옵티마이저 힌트 - 목차 기준 p.260, p.24.

문항 번호: 11 정답: 1

출제 의도: 소트 머지 조인을 NL/해시 조인과 구분하고 적용 가능성을 판단하게 한다.

해설: 정답은 1번이다. 소트 머지 조인은 입력을 조인 키 순서로 정렬하고 병합하는 방식이다. 이미 정렬된 입력이 있거나 정렬 비용이 감당 가능하면 선택될 수 있다.

오답 선택지 해설: 2번은 NL 조인 설명이다. 3번은 해시 조인 설명이다. 4번은 과도한 단정이다. 소트 머지 조인은 동등 조인 외 조건에서도 활용될 수 있다.

책 근거: 4.2 소트 머지 조인, 4.2.2 기본 메커니즘, 4.2.5 소트 머지 조인 제어하기 - 목차 기준 p.274, p.276, p.280.

문항 번호: 12 정답: 3

출제 의도: 스칼라 서브쿼리의 반복 실행 비용과 캐싱이 효과적인 조건을 이해했는지 확인한다.

해설: 정답은 3번이다. 같은 상관값이 반복되면 해당 입력값에 대한 결과를 재사용할 여지가 있어 캐싱 효과가 커진다. 반대로 상관값이 모두 다르다면 캐시 히트 가능성이 낮다.

오답 선택지 해설: 1번은 상관 서브쿼리 실행 구조를 오해했다. 2번은 반대다. 4번은 캐싱과 결과 중복 제거를 혼동했다.

책 근거: 4.4.4 스칼라 서브쿼리 조인 - 목차 기준 p.317.

문항 번호: 13 정답: 2

출제 의도: 뷰 머징과 조건 pushdown을 무조건/항상으로 오해하지 않도록 한다.

해설: 정답은 2번이다. 인라인 뷰는 항상 물리화되는 것이 아니며, 의미가 보존되면 옵티마이저가 뷰를 병합하거나 조건을 내부로 밀어 넣어 처리량을 줄일 수 있다.

오답 선택지 해설: 1번과 4번은 옵티마이저 변환 가능성을 부정한다. 3번은 반대로 과장이다. 집계, DISTINCT, ROWNUM 등은 병합을 제한할 수 있다.

책 근거: 4.4.3 뷰(View)와 조인, 4.4.1 서브쿼리 변환이 필요한 이유 - 목차 기준 p.311, p.299.

문항 번호: 14 정답: 4

출제 의도: Skip Scan의 성립 조건과 비용 판단 기준을 확인한다.

해설: 정답은 4번이다. Skip Scan은 선두 컬럼 조건이 없을 때, 선두 컬럼의 가능한 값을 나누어 후행 컬럼 조건을 활용하는 방식이다. 선두 컬럼 NDV가 낮고 후행 조건이 선택적일수록 검토 가치가 있다.

오답 선택지 해설: 1번은 일반 Range Scan 대상이다. 2번은 선두 컬럼 값 종류가 많아 비용이 커질 수 있다. 3번은 Skip Scan을 검토할 상황 이 아니다.

책 근거: 2.3.4 Index Skip Scan, 3.3.7 Index Skip Scan 활용 - 목차 기준 p.116, p.199.

문항 번호: 15 정답: 1

출제 의도: 인덱스 정렬 순서를 이용한 소트 생략 조건을 정확히 판단하게 한다.

해설: 정답은 1번이다. 인덱스의 물리적 정렬 순서가 SQL이 요구하는 정렬/그룹핑 순서와 맞으면 Sort Order By 또는 Sort Group By 를 생략할 수 있다.

오답 선택지 해설: 2번은 GROUP BY 처리 방식의 다양성을 무시한다. 3번은 컬럼 순서와 정렬 방향 조건을 무시한다. 4번은 함수 가공으로 일반 인덱스 순서를 활용하지 못할 수 있어 틀렸다.

책 근거: 5.3.1 Sort Order By 생략, 5.3.5 Sort Group By 생략 - 목차 기준 p.350, p.379.

문항 번호: 16 정답: 3

출제 의도: 대량 DML/Direct Path I/O의 효과를 무조건적 성능 개선으로 오해하지 않도록 한다.

해설: 정답은 3번이다. Direct Path I/O는 대량 처리에서 버퍼 캐시 경유 부담을 줄이는 데 유리할 수 있지만, 인덱스/제약/트랜잭션/동시성 등 적용 조건과 부작용을 함께 판단해야 한다.

오답 선택지 해설: 1번은 버퍼 캐시 활용 방향을 반대로 이해했다. 2번은 인덱스와 제약 유지 부담을 과도하게 제거한다고 단정했다. 4번은 Direct Path Insert 등 SQL 작성과 실행 방식의 영향을 무시한다.

책 근거: 6.2 Direct Path I/O 활용, 6.2.1 Direct Path I/O, 6.2.2 Direct Path Insert - 목차 기준 p.436, p.438.

문항 번호: 17 정답: 2

출제 의도: 통계정보, 선택도, 카디널리티와 실행계획 품질의 관계를 확인한다.

해설: 정답은 2번이다. 옵티마이저는 통계정보를 바탕으로 선택도와 카디널리티를 추정하고 비용을 비교한다. 이 추정이 틀리면 인덱스, 조인 순서, 조인 방식, 소트 방식 선택이 나빠질 수 있다.

오답 선택지 해설: 1번은 추정과 실제를 혼동했다. 3번은 힌트를 만능으로 보는 오류다. 4번은 카디널리티가 소트량과 메모리 사용에도 영향을 준다는 점을 무시한다.

책 근거: 7.1 통계정보와 비용 계산 원리, 7.1.1 선택도와 카디널리티, 7.1.3 비용 계산 원리 - 목차 기준 p.501, p.509.

문항 번호: 18 정답: 4

출제 의도: 등치 조건 우선, 범위 조건 위치, 정렬 생략, 테이블 액세스 최소화를 종합 판단하게 한다.

해설: 정답은 4번이다. 고객번호와 상태코드가 등치 조건이고 주문일자가 범위 및 정렬 컬럼이다. (고객번호, 상태코드, 주문일자) 순서는 스캔 범위를 좁히면서 주문일자 순서를 유지할 수 있다. 여기에 주문번호와 주문금액을 포함하면 테이블 액세스도 줄일 수 있다.

오답 선택지 해설: 1번은 주문일자 범위가 선두라 고객번호 선택도를 충분히 활용하지 못할 수 있다. 2번은 상태코드 선택도가 낮고 고객번호가 뒤로 밀린다. 3번은 주문일자 범위 뒤의 상태코드가 필터로 밀려 완료 조건의 액세스 효율이 떨어질 수 있다.

책 근거: 3.4 인덱스 설계, 3.4.2 가장 중요한 두 가지 선택 기준, 3.4.5 소트 연산을 생략하기 위한 컬럼 추가, 3.1.5 인덱스만 읽고 처리 - 목차 기준 p.230, p.232, p.238, p.149.

문항 번호: 19 정답: 1

출제 의도: 실행계획의 반복 탐색 횟수와 선택도 정보를 함께 읽고 조인 순서/인덱스 설계를 판단하게 한다.

해설: 정답은 1번이다. 고객 등급 조건은 100만 건으로 너무 넓고, 주문일자는 5만 건으로 훨씬 선택적이다. 현재 계획은 고객 100만 건을 선행으로 두어 주문 인덱스를 반복 탐색하므로 논리 I/O가 커질 수 있다. 주문일자 선두 인덱스나 주문 선행 조인, 해시 조인 등을 검토할 수 있다.

오답 선택지 해설: 2번은 인덱스를 사용했다는 사실만 보고 반복 횟수를 무시했다. 3번은 GROUP BY와 조인 방식의 관계를 단정했다. 4번은 고객등급 조건의 실제 선택도를 무시했다.

책 근거: 4.1.4 NL 조인 튜닝 포인트, 4.3.5 조인 메소드 선택 기준, 3.4 인덱스 설계 - 목차 기준 p.264, p.296, p.230.

문항 번호: 20 정답: 3

출제 의도: OR 조건 분해, UNION ALL 활용, 중복 의미 보존을 함께 판단하게 한다.

해설: 정답은 3번이다. 고객번호 조건과 상태코드 조건은 각각 다른 인덱스에 잘 맞는다. 조건을 분리해 UNION ALL로 작성하면 각 인덱스를 활용할 수 있지만, 두 조건을 동시에 만족하는 로우가 중복될 수 있으므로 의미 보존을 위해 중복 제거 또는 한쪽 분기에서 반대 조건을 추가해야 한다.

오답 선택지 해설: 1번은 OR 조건에서도 변환이나 분해로 인덱스를 활용할 수 있음을 무시한다. 2번은 반드시 하나로 합쳐야 한다는 단정이 틀렸다. 4번은 UNION의 중복 제거 비용을 무시한다.

책 근거: 3.3.11 다양한 옵션 조건 처리 방식의 장단점 비교, 5.2.1 Union vs Union All - 목차 기준 p.214, p.343.

문항 번호: 21 정답: 2

출제 의도: 인덱스 스캔 순서와 SQL 결과 순서 보장의 차이를 구분하게 한다.

해설: 정답은 2번이다. 인덱스 엔트리 순서로 읽더라도 테이블 액세스 최적화를 위해 rowid 방문이 배치 처리되면 반환 순서가 달라질 수 있다. SQL 결과 순서를 보장하는 유일한 방법은 ORDER BY를 명시하는 것이다.

오답 선택지 해설: 1번은 원인을 잘못 판단했다. 3번은 실행 방식에 따른 rowid 방문 조정 가능성을 무시한다. 4번은 SQL의 결과 순서 보장 규칙을 오해한 설명이다.

책 근거: 3.1 테이블 액세스 최소화, 3.2 부분범위 처리 활용, 인덱스 스캔과 테이블 액세스 관련 주제 - 목차 기준 p.129, p.158. Batch I/O 세부 페이지는 페이지 번호 확실하지 않음.

문항 번호: 22 정답: 4

출제 의도: 해시 조인의 Build/Probe 구조, 메모리 사용, 대량 조인 적합성을 확인한다.

해설: 정답은 4번이다. 해시 조인은 작은 입력을 해시 테이블로 만드는 Build 단계와 다른 입력을 탐색하는 Probe 단계로 이해할 수 있다. Build Input이 메모리에 적절히 들어가야 효율적이며, 부족하면 디스크 I/O가 발생할 수 있다.

오답 선택지 해설: 1번은 NL 조인 설명이다. 2번은 소트 머지 조인 설명이다. 3번은 반대다. 큰 테이블을 Build Input으로 선택하면 메모리 부담이 커진다.

책 근거: 4.3 해시 조인, 4.3.1 기본 메커니즘, 4.3.3 대용량 Build Input 처리 - 목차 기준 p.282, p.288.

문항 번호: 23 정답: 1

출제 의도: 서버쿼리 unnesting 억제와 subquery push의 의미를 구분하게 한다.

해설: 정답은 1번이다. NO_UNNEST는 서버쿼리를 조인으로 풀어내는 변환을 억제하려는 힌트이고, PUSH_SUBQ는 필터 서버쿼리 평가를 가능한 한 앞쪽으로 밀어 불필요한 조인 입력을 줄이려는 의도이다.

오답 선택지 해설: 2번은 USE_HASH의 의미를 잘못 붙였다. 3번은 ORDERED를 서버쿼리 평가 시점 고정 힌트로 오해했다. 4번은 NO_MERGE와 서버쿼리 unnesting을 혼동했다.

책 근거: 4.4 서버쿼리 조인, 4.4.1 서버쿼리 변환이 필요한 이유, 1.1.5 옵티마이저 힌트 - 목차 기준 p.299, p.24.

NO_UNNEST/PUSH_SUBQ 세부 페이지는 페이지 번호 확실하지 않음.

문항 번호: 24 정답: 3

출제 의도: 스칼라 서버쿼리의 단일 행 보장 조건과 조인 전환 시 결과 보존 문제를 확인한다.

해설: 정답은 3번이다. 스칼라 서버쿼리는 외부 로우당 한 값만 반환해야 한다. 최신 주문일자가 같아 여러 주문이 존재할 수 있으면 단일 행이 보장되지 않는다. 조인으로 바꿀 때도 ROW_NUMBER, 추가 정렬 기준, 집계 등으로 한 건 기준을 명확히 해야 한다.

오답 선택지 해설: 1번은 스칼라 서버쿼리의 반환 특성을 잘못 설명한다. 2번은 MAX 주문일자는 한 값이어도 그 일자에 여러 주문이 있을 수 있음을 놓쳤다. 4번은 조인 전환 시 중복 위험을 무시한다.

책 근거: 4.4.4 스칼라 서버쿼리 조인, 4.4.2 서버쿼리와 조인 - 목차 기준 p.317, p.302.

문항 번호: 25 정답: 2

출제 의도: 분석함수 Top-N에서 Window Sort와 인덱스 정렬 순서 활용 가능성을 판단하게 한다.

해설: 정답은 2번이다. ROW_NUMBER의 PARTITION BY/ORDER BY는 부서별 급여 내림차순 정렬을 요구한다. 조건과 인덱스 순서가 맞으면 정렬 부담을 줄일 수 있지만, 그렇지 않으면 Window Sort가 발생할 수 있다.

오답 선택지 해설: 1번은 분석함수의 정렬 요구를 무시한다. 3번은 분석함수와 조인 방식을 혼동했다. 4번은 전체 Top-N과 그룹별 Top-N을 혼동한 설명이다.

책 근거: 5.4.4 분석함수에서의 Top N 소트, 5.4.2 Top N 쿼리의 소트 부하 경감 원리, 5.3.2 Top N 쿼리 - 목차 기준 p.387, p.383, p.352.

SQLP 객관식 문제지

총 25문항 · 각 문항 4지선다형

1. 주문 상세 화면은 최근 3개월의 고객별 주문 목록을 자주 조회한다. ORDERS 테이블은 약 3천만 건이고, 한 고객 조건에 평균 80건이 반환된다. 현재 인덱스는 ORD_X01(CUST_ID, ORDER_DT) 하나이며, 실행 계획에는 ORDERS 테이블에 대한 TABLE ACCESS BY INDEX ROWID가 반복적으로 나타난다.

화면은 아래 SQL의 네 컬럼만 사용한다. 가장 안정적으로 랜덤 테이블 액세스를 줄이는 개선 방향으로 가장 적절한 것은?

```
SELECT order_id, order_dt, order_status, pay_amt
FROM   orders
WHERE  cust_id = :cust_id
AND    order_dt >= ADD_MONTHS(TRUNC(SYSDATE), -3)
ORDER BY order_dt DESC;
```

- A. CUST_ID 단일 인덱스를 추가해 고객 조건만 더 빠르게 찾도록 한다.
- B. 조회 컬럼까지 포함한 결합 인덱스를 검토해 테이블 액세스를 줄인다.
- C. ORDERS 테이블을 Full Scan하도록 유도해 랜덤 액세스를 없앤다.
- D. ORDER_DT 단일 인덱스를 추가해 기간 조건의 선택도를 높인다.

2. 다음 인덱스와 SQL 조건이 있다. 주문일자 조건은 보통 6개월 이상으로 넓게 들어오고, 회원등급 = 'VIP' 조건은 전체의 약 3%만 만족한다.

실행계획의 Predicate Information까지 고려할 때 가장 적절한 판단은?

```
-- 인덱스
CREATE INDEX 주문_X01 ON 주문(상품구분, 주문일자, 회원등급, 주문번호);
```

```
SELECT 주문번호, 주문일자, 결제금액
FROM   주문
WHERE  상품구분 = :gubun
AND    주문일자 BETWEEN :from_dt AND :to_dt
AND    회원등급 = 'VIP';
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				920
1	TABLE ACCESS BY INDEX ROWID	주문	1200	96K	920
2	INDEX RANGE SCAN	주문_X01	48000		130

```
Predicate Information:
  2 - access(상품구분=:GUBUN AND 주문일자>=:FROM_DT AND 주문일자<=:TO_DT)
      filter(회원등급='VIP')
```

- A. 회원등급은 인덱스에 있으므로 테이블 필터 없이 항상 효율적으로 처리된다.
- B. 주문일자가 BETWEEN이어도 뒤 컬럼 회원등급까지 스캔 시작점을 정한다.
- C. 주문일자가 범위 조건이라 회원등급은 읽은 리프 엔트리에서 걸러진다.
- D. 상품구분이 선두 등치 조건이면 기간이 길어도 스캔량은 항상 작다.

3. 월별판매 테이블에는 5년치 데이터가 있고, 판매구분 = 'B'는 전체의 약 8%다. 아래 SQL은 1년치 자료를 조회한다.

두 인덱스 중 하나만 선택해야 한다면, 스캔 효율 관점에서 가장 적절한 설명은?

```
-- 후보 인덱스
IDX_A : (판매월, 판매구분)
IDX_B : (판매구분, 판매월)

SELECT COUNT(*)
FROM 월별판매
WHERE 판매월 BETWEEN '202401' AND '202412'
AND 판매구분 = 'B';
```

- A. IDX_B가 유리하다. 선두 등치 후 월 범위를 타므로 불필요한 판매구분을 덜 읽는다.
- B. IDX_A가 유리하다. BETWEEN은 항상 등치처럼 작동해 뒤 컬럼 선택도가 반영된다.
- C. 두 인덱스는 동일하다. 두 컬럼이 모두 조건에 있으므로 스캔량 차이가 없다.
- D. 둘 다 부적합하다. BETWEEN 조건이 있으면 Index Range Scan이 불가능하다.

4. 게시글 목록 화면은 최신 글 10건만 보여 준다. 게시글_X01 인덱스는 (게시판ID, 등록일시 DESC, 글번호 DESC) 순서로 생성되어 있다.

다음 SQL과 실행계획에 대한 설명으로 가장 적절한 것은?

```
SELECT *
FROM (
    SELECT 글번호, 제목, 등록일시
    FROM 게시글
    WHERE 게시판ID = :board_id
    ORDER BY 등록일시 DESC, 글번호 DESC
)
WHERE ROWNUM <= 10;
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				8
1	COUNT STOPKEY		10		8
2	TABLE ACCESS BY INDEX ROWID	게시글	10	4K	8
3	INDEX RANGE SCAN	게시글_X01	10		3

- A. ROWNUM 조건은 항상 ORDER BY 이후 평가되므로 인덱스와 무관하게 정렬이 생략된다.
- B. INDEX RANGE SCAN은 정렬 순서를 보장하지 않으므로 반드시 SORT ORDER BY가 필요하다.
- C. COUNT STOPKEY는 10건 반환 후에도 조건 범위 전체를 끝까지 읽는다는 의미다.
- D. 인덱스 순서가 정렬 기준과 같아 앞쪽 10건만 읽고 멈출 수 있다는 의미다.

5. 아래 쿼리는 서울 거주 고객 중 혈액형이 A 또는 O인 고객을 나이순으로 출력한다. 인덱스는 고객_X01(거주지역, 혈액형, 나이)이다.

ORDER BY 생략 가능성을 판단한 설명으로 가장 적절한 것은?

```
SELECT 고객번호, 거주지역, 혈액형, 나이
FROM 고객
WHERE 거주지역 = '서울'
AND 혈액형 IN ('A', 'O')
ORDER BY 나이;
```

- A. IN 조건은 항상 단일 등치처럼 처리되므로 나이 순서가 전체적으로 보장된다.
- B. 혈액형별 범위를 따로 읽으면 나이 순서가 그룹마다 나뉘어 전체 정렬이 깨질 수 있다.
- C. 혈액형 값이 두 개뿐이면 옵티마이저는 반드시 Sort Unique 연산을 선택한다.
- D. 거주지역이 선두 등치 조건이면 ORDER BY 나이는 어떤 인덱스에서든 생략된다.

6. 회원 테이블에는 회원_X01(전화번호, 가입일시) 인덱스가 있다. 전화번호는 하이픈 없이 항상 11자리 숫자 문자열로 저장된다.

다음 조건 때문에 일반 인덱스 Range Scan 효율이 떨어질 때 가장 우선적으로 검토할 개선안은?

```
SELECT 회원번호, 회원명, 가입일시
FROM   회원
WHERE  SUBSTR(전화번호, 1, 3) = '010'
AND    가입일시 >= :from_dt;
```

- A. 전화번호 >= '010' AND 전화번호 < '011'처럼 컬럼 변형 없는 범위 조건으로 바꾼다.
- B. SUBSTR는 문자열 함수이므로 B-tree 인덱스가 함수 결과를 자동으로 저장한다.
- C. 함수 조건은 선택도가 높으므로 Full Scan보다 항상 Index Full Scan이 빠르다.
- D. 가입일시 선두 인덱스를 추가하면 전화번호 함수 조건도 access 조건이 된다.

7. 거래내역 조회 화면의 90%는 고객번호와 거래일시 기간을 입력하고 최신순 20건만 조회한다. 상품코드 조건은 10% 미만의 요청에서만 선택적으로 입력된다.

다음 SQL을 기준으로 가장 적절한 기본 인덱스 구성은?

```
SELECT 거래번호, 거래일시, 상품코드, 거래금액
FROM   거래내역
WHERE  고객번호 = :cust_id
AND    거래일시 >= :from_dt
AND    거래일시 < :to_dt
ORDER BY 거래일시 DESC
FETCH FIRST 20 ROWS ONLY;
```

- A. (거래일시 DESC, 고객번호, 상품코드) - ORDER BY 컬럼을 무조건 선두에 둔다.
- B. (상품코드, 고객번호, 거래일시 DESC) - 선택 조건을 모두 선두에 모은다.
- C. (고객번호, 거래일시 DESC, 상품코드) - 필수 등치와 최신순 접근을 우선한다.
- D. (고객번호, 상품코드, 거래일시) - 모든 선택 컬럼을 범위 컬럼 앞에 둔다.

8. 고객_X01 인덱스는 (성별, 생년월일, 고객번호) 순서이며 성별의 NDV는 2다. 아래 SQL에서 옵티마이저는 INDEX SKIP SCAN을 선택했다.

이 실행계획에 대한 설명으로 가장 적절한 것은?

```
SELECT /*+ index_ss(c 고객_X01) */ 고객번호, 고객명
FROM   고객 c
WHERE  생년월일 BETWEEN DATE '1990-01-01' AND DATE '1990-12-31'
AND    고객번호 LIKE 'A12%';
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				72
1	TABLE ACCESS BY INDEX ROWID	고객	180	18K	72
2	INDEX SKIP SCAN	고객_X01	180		41

- A. 선두 컬럼 조건이 없으면 B-tree 인덱스는 어떤 방식으로든 사용할 수 없다.
- B. Skip Scan은 인덱스를 한 번만 읽으므로 새 인덱스보다 항상 저렴하다.
- C. 고객번호에 LIKE가 있으므로 생년월일 조건은 인덱스에서 전혀 쓰이지 않는다.
- D. 선두 컬럼 NDV가 작아 각 하위 구간을 탐색하지만, 범위가 넓으면 비효율적일 수 있다.

9. 아래 SQL에서 DEPT.LOC = 'CHICAGO' 조건은 보통 1~2건만 반환하고, EMP에는 EMP_X01(DEPTNO) 인덱스가 있다.

실행계획의 조인 방식에 대한 설명으로 가장 적절한 것은?

```
SELECT e.empno, e.ename, d.dname
FROM   dept d, emp e
WHERE  d.deptno = e.deptno
AND    d.loc = 'CHICAGO';
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				12
1	NESTED LOOPS		8	720	12
2	TABLE ACCESS BY INDEX ROWID	DEPT	1	40	3
3	INDEX RANGE SCAN	DEPT_LOC_X1	1		1
4	TABLE ACCESS BY INDEX ROWID	EMP	8	640	9
5	INDEX RANGE SCAN	EMP_X01	8		2

- A. Outer 결과가 작고 Inner 인덱스 접근이 가능하므로 NL 조인이 적절하다.
- B. NL 조인은 Inner 테이블 인덱스를 사용할 수 없으므로 항상 비효율적이다.
- C. 두 테이블을 조인할 때는 결과 건수와 무관하게 Hash 조인이 항상 빠르다.
- D. 조인 조건이 같으면 어느 테이블을 먼저 읽어도 I/O 비용은 거의 같다.

10. EMP_X01 인덱스는 (DEPTNO, EMPNO) 순서다. 아래 SQL은 DEPTNO = 20인 사원을 EMPNO 순서로 출력하려고 한다. 실행계획에는 TABLE ACCESS BY INDEX ROWID BATCHED와 SORT ORDER BY가 함께 보인다.

가장 적절한 해석은?

```
SELECT empno, ename, sal
FROM   emp
WHERE  deptno = 20
ORDER BY empno;
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				18
1	SORT ORDER BY		120	9K	18
2	TABLE ACCESS BY INDEX ROWID BATCHED	EMP	120	9K	16
3	INDEX RANGE SCAN	EMP_X01	120		3

- A. BATCHED는 인덱스 필터를 포기하고 테이블 Full Scan으로 바꿨다는 뜻이다.
- B. BATCHED가 보이면 테이블 I/O가 사라지므로 정렬도 항상 생략된다.
- C. BATCHED는 Hash 조인에서만 나타나며 단일 테이블 조회에는 쓰이지 않는다.
- D. ROWID 묶음 접근은 랜덤 I/O를 줄일 수 있지만 인덱스 키 순서를 깨뜨릴 수 있다.

11. 두 대형 테이블을 상품번호로 조인한다. PRODUCTS는 조건 적용 후 약 2만 건, ORDER_ITEMS는 약 500만 건을 읽는다. ORDER_ITEMS에는 반복 조회에 유리한 선택적 인덱스가 없다.

다음 실행계획의 핵심 판단으로 가장 적절한 것은?

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				3180
1	HASH JOIN		520000	58M	3180
2	TABLE ACCESS FULL	PRODUCTS	20000	2M	210
3	TABLE ACCESS FULL	ORDER_ITEMS	5000000	410M	2870

- A. 대량 조인에서는 항상 큰 ORDER_ITEMS를 Build Input으로 잡는 것이 유리하다.
- B. 작은 쪽을 해시 테이블로 만들고 큰 쪽을 Probe하는 선택으로 볼 수 있다.
- C. Hash 조인은 두 입력이 조인 키 순서로 정렬되어 있을 때만 사용할 수 있다.
- D. Hash 조인은 등치 조인보다 비등치 조인에서만 비용상 장점이 생긴다.

12. 두 입력 집합이 모두 조인 키 순서로 인덱스 스캔 가능하고, 조인 결과도 조인 키 순서로 대량 출력해야 한다. 선택적인 Inner 인덱스 반복 탐색은 기대하기 어렵다.

Sort Merge Join을 고려할 수 있는 상황에 대한 설명으로 가장 적절한 것은?

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				1650
1	MERGE JOIN		900000	82M	1650
2	SORT JOIN		300000	18M	610
3	INDEX FULL SCAN	A_JOIN_X1	300000		420
4	SORT JOIN		900000	64M	1040
5	INDEX FULL SCAN	B_JOIN_X1	900000		790

- A. Outer가 한두 건이고 Inner 인덱스가 매우 선택적일 때 가장 먼저 선택한다.
- B. 작은 Build Input이 메모리에 들어갈 때만 사용할 수 있는 해시 기반 조인이다.
- C. 입력 정렬을 활용하거나 정렬 결과를 재사용할 수 있는 대량 조인에 맞을 수 있다.
- D. 조인 조건이 없어도 두 테이블을 한 번씩 읽기만 하면 결과가 정확해진다.

13. 아래 SQL은 최근 30일 주문이 있는 고객만 조회한다. 현재 실행계획에서는 CUSTOMERS를 읽은 뒤 상관 서브쿼리를 FILTER로 반복 수행한다.

옵티마이저 변환 관점에서 가장 적절한 설명은?

```
SELECT c.cust_id, c.cust_name
FROM customers c
WHERE EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.cust_id = c.cust_id
  AND o.order_dt >= TRUNC(SYSDATE) - 30
);
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				2400
1	FILTER				
2	TABLE ACCESS FULL	CUSTOMERS	1000000	60M	900
3	INDEX RANGE SCAN	ORD_X01	5		2

- A. 서브쿼리 Unnesting으로 Semi Join 형태가 되면 반복 FILTER 비용을 줄일 수 있다.
- B. EXISTS 서브쿼리는 의미상 항상 Full Scan으로만 처리되어 인덱스를 쓸 수 없다.
- C. Unnesting은 EXISTS를 일반 조인으로 바꾸므로 중복 고객을 반드시 출력한다.
- D. 상관 서브쿼리는 항상 no_unnest 힌트로 고정해야 옵티마이저가 안정된다.

14. 아래 SQL은 거래 200만 건을 읽으면서 지점명을 조회한다. 거래 테이블의 BRANCH_CD 값은 30종류 뿐이고 지점 테이블은 작다.

스칼라 서브쿼리와 관련한 설명으로 가장 적절한 것은?

```
SELECT a.acct_no,
       a.trx_amt,
       ( SELECT b.branch_name
         FROM   branch b
        WHERE  b.branch_cd = a.branch_cd ) AS branch_name
FROM   account_txn a
WHERE  a.trx_dt >= :from_dt;
```

- A. 스칼라 서브쿼리는 행마다 파싱되므로 같은 입력값이어도 캐시 효과가 없다.
- B. 입력값 반복이 많으면 캐시 효과가 생길 수 있지만 NDV와 캐시 크기를 봐야 한다.
- C. 스칼라 서브쿼리는 한 건만 반환하므로 어떤 조인보다 항상 빠르게 수행된다.
- D. 지점코드가 Unique이면 스칼라 서브쿼리 캐시는 자동으로 비활성화된다.

15. 다음 SQL은 부서와 직무별 급여 합계를 구한다. 실행계획에는 SORT GROUP BY가 아니라 HASH GROUP BY가 표시된다.

애플리케이션이 DEPTNO, JOB 순서의 정렬된 결과를 기대한다면 가장 적절한 설명은?

```
SELECT deptno, job, SUM(sal) AS sum_sal
FROM   emp
GROUP BY deptno, job;
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				12
1	HASH GROUP BY		12	600	12
2	TABLE ACCESS FULL	EMP	14000	560K	9

- A. GROUP BY 절은 표준적으로 결과 정렬을 보장하므로 ORDER BY는 불필요하다.
- B. HASH GROUP BY가 보이면 정렬보다 큰 Temp 정렬이 반드시 발생했다는 뜻이다.
- C. SUM 집계기가 있으면 Hash Group By는 사용할 수 없고 실행계획이 잘못된 것이다.
- D. 정렬된 결과가 필요하면 ORDER BY를 명시해야 하며 그룹 방식과 분리해 판단한다.

16. 아래 SQL은 결제수단별로 같은 날짜의 결제 데이터를 합쳐 조회한다. 결제수단 = 'M'과 결제수단 = 'C'는 서로 동시에 만족할 수 없다.

소트 부하를 줄이는 관점에서 가장 적절한 설명은?

```
SELECT 결제번호, 결제수단코드, 주문번호, 결제금액
FROM   결제
WHERE  결제수단코드 = 'M'
AND    결제일자 = :dt
UNION
SELECT 결제번호, 결제수단코드, 주문번호, 결제금액
FROM   결제
WHERE  결제수단코드 = 'C'
AND    결제일자 = :dt;
```

- A. UNION ALL은 항상 결과를 바꾸므로 중복 가능성과 무관하게 사용할 수 없다.
- B. UNION은 두 분기를 한 번씩만 읽으므로 UNION ALL보다 항상 소트 비용이 낮다.
- C. 분기 조건이 상호 배타적이면 UNION ALL로 Sort Unique를 피할 수 있다.
- D. UNION ALL은 내부적으로 Sort Unique를 수행하지만 비교 대상이 적어 더 빠르다.

17. 거래일시 기준 최신 10건을 정확히 조회하고, 가능하면 Top N Stopkey 최적화를 기대하려고 한다. 다음 중 가장 적절한 SQL 형태는?

- A. SELECT * FROM (SELECT * FROM 거래 WHERE 고객번호=:c ORDER BY 거래일시 DESC) WHERE ROWNUM <= 10
- B. SELECT * FROM 거래 WHERE 고객번호=:c AND ROWNUM <= 10 ORDER BY 거래일시 DESC
- C. SELECT * FROM 거래 WHERE 고객번호=:c ORDER BY 거래일시 DESC HAVING ROWNUM <= 10
- D. SELECT * FROM (SELECT * FROM 거래 WHERE 고객번호=:c AND ROWNUM <= 10) ORDER BY 거래일시 DESC

18. 이력_X01 인덱스는 (상태코드, 변경일자 DESC) 순서다. 아래 SQL의 실행계획에는 SORT ORDER BY가 남아 있다.

정렬 생략이 되지 않은 이유로 가장 타당한 것은?

```
SELECT 이력번호, 상태코드, 변경일자, 처리순번
FROM   상태이력
WHERE  상태코드 = :st
AND    변경일자 >= :from_dt
ORDER  BY 변경일자 DESC, 처리순번 DESC;
```

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT				95
1	SORT ORDER BY		4000	520K	95
2	TABLE ACCESS BY INDEX ROWID	상태이력	4000	520K	72
3	INDEX RANGE SCAN	이력_X01	4000		16

- A. 상태코드 등치 조건이 있으면 어떤 경우에도 인덱스 정렬 순서를 사용할 수 없다.
- B. Index Range Scan은 항상 무작위 순서로 반환하므로 ORDER BY를 제거할 수 없다.
- C. ORDER BY를 제거하려면 반드시 Unique 인덱스만 사용해야 하므로 현재 인덱스는 불가하다.
- D. ORDER BY의 처리순번까지 인덱스 순서가 맞지 않아 전체 정렬을 보장할 수 없다.

19. 장비별 최종 상태를 조회하기 위해 아래와 같은 분석함수 SQL을 작성했다. 상태변경이력은 매우 크고, 장비별 최종 1건만 필요하다.

성능 개선 검토 방향으로 가장 적절한 것은?

```
SELECT 장비번호, 상태코드, 변경일자
FROM (
    SELECT 장비번호, 상태코드, 변경일자,
           ROW_NUMBER() OVER(
               PARTITION BY 장비번호
               ORDER BY 변경일자 DESC, 변경순번 DESC
           ) AS rn
    FROM   상태변경이력
)
WHERE rn = 1;
```

- A. 분석함수는 파티션 키가 있으면 Window Sort 없이 항상 인덱스만으로 처리된다.
- B. 최종 1건만 필요하면 KEEP 집계나 Top N 접근으로 소트 범위를 줄일 수 있는지 본다.
- C. ROW_NUMBER와 RANK는 동률 처리와 결과 건수가 항상 같으므로 아무거나 써도 된다.
- D. MAX(변경일자)만 구하면 같은 행의 상태코드는 조인 없이 항상 정확히 따라온다.

20. 배치 작업이 TARGET 테이블에 1천만건을 INSERT한다. TARGET에는 PK 인덱스 1개와 일반 인덱스 11개가 있고, 작업 시간 대부분이 INSERT 구간에서 발생한다.

DML 성능 관점에서 가장 적절한 설명은?

```
INSERT INTO target
SELECT *
FROM source;
```

- A. 인덱스는 SELECT 성능에만 영향을 주므로 INSERT 속도와는 거의 무관하다.
- B. 모든 제약조건을 NOVALIDATE로 바꾸면 데이터 정합성 검토 없이 항상 안전하다.
- C. 각 행마다 여러 인덱스 유지 비용이 발생하므로 대량 적재 전략을 검토해야 한다.
- D. OLTP INSERT 성능을 높이려면 일반적으로 Bitmap 인덱스를 더 추가해야 한다.

21. 대량 적재 시간을 줄이기 위해 아래와 같이 APPEND 힌트를 사용하려고 한다. TARGET은 로깅 모드이며 인덱스와 제약조건이 존재한다.

Direct Path Insert에 대한 설명으로 가장 적절한 것은?

```
INSERT /*+ APPEND */ INTO target
SELECT *
FROM source;
```

- A. APPEND를 사용하면 테이블 로깅 속성과 무관하게 Redo가 전혀 생성되지 않는다.
- B. APPEND는 항상 일반 INSERT와 동일한 동시성을 제공해 테이블 Lock 영향이 없다.
- C. APPEND는 모든 인덱스 유지를 자동으로 건너뛰므로 인덱스 비용이 사라진다.
- D. 새 블록에 직접 적재하지만 로깅, 인덱스, 제약조건, 커밋 시점을 별도로 봐야 한다.

22. 거래 테이블은 월별 파티션이고, 2014년 12월 파티션의 500만 건에 대해 상태코드를 일괄 변경해야 한다. 변경 대상은 해당 파티션 대부분이며 서비스 영향 시간을 최소화해야 한다.

가장 적절한 튜닝 방향은?

```
UPDATE 거래
SET 상태코드 = 'ZZZ'
WHERE 거래일자 >= DATE '2014-12-01'
AND 거래일자 < DATE '2015-01-01';
```

- A. 임시 테이블에 변경 결과를 만들고 인덱스를 준비한 뒤 파티션 Exchange를 검토한다.
- B. UPDATE 문에 인덱스 힌트를 추가해 500만건을 모두 실시간 랜덤 갱신한다.
- C. 전체 거래 테이블을 Drop 후 모든 파티션을 다시 만드는 방식만 사용할 수 있다.
- D. 글로벌 인덱스를 하나 더 추가하면 Undo와 Redo 발생량이 대부분 사라진다.

23. 거래 테이블에서 2014년 12월 한 달치 데이터를 완전히 삭제해야 한다. 거래일자 기준 월별 파티션이 정확히 구성되어 있고, 대상 파티션 전체가 삭제 대상이다.

아래 DELETE를 대체할 수 있는 방법에 대한 설명으로 가장 적절한 것은?

```
DELETE FROM 거래
WHERE 거래일자 >= DATE '2014-12-01'
AND 거래일자 < DATE '2015-01-01';
```

- A. DELETE는 내부적으로 파티션 메타데이터만 바꾸므로 Drop Partition과 비용 차이가 없다.
- B. 파티션 전체가 대상이면 Drop 또는 Truncate Partition으로 대량 작업을 줄일 수 있다.
- C. Drop Partition은 로컬 인덱스를 영구적으로 모두 사용할 수 없게 만들어 항상 금지된다.
- D. Truncate Partition은 파티션 내부의 임의 조건 행만 골라 삭제할 때 가장 적합하다.

24. 여러 배치 프로세스가 READY 상태의 작업을 동시에 가져가 처리한다. 같은 작업을 두 프로세스가 동시에 가져가면 안 되지만, 이미 다른 프로세스가 잡은 행 때문에 전체 배치가 멈추는 것도 피하고 싶다.

SELECT FOR UPDATE 옵션에 대한 설명으로 가장 적절한 것은?

```
SELECT job_id
FROM   job_queue
WHERE  status = 'READY'
ORDER  BY priority, job_id
FOR UPDATE SKIP LOCKED;
```

- A. NOWAIT는 잠긴 행이 풀릴 때까지 무기한 기다린 뒤 같은 결과를 반환한다.
- B. WAIT 3은 3초가 지나면 잠긴 행을 조용히 건너뛰고 다음 행을 읽는다.
- C. SKIP LOCKED는 잠긴 행을 건너뛰므로 큐 처리에 유용하지만 누락처럼 보일 수 있다.
- D. 일반 SELECT도 Row Exclusive Lock을 잡으므로 FOR UPDATE는 의미가 없다.

25. 주문번호를 생성하기 위해 여러 세션이 동시에 아래 방식의 INSERT를 수행한다. 피크 시간에는 같은 테이블에 동시 INSERT가 매우 많다.

채번 방식과 Lock 경합에 대한 설명으로 가장 적절한 것은?

```
INSERT INTO 주문(주문번호, 고객ID, 주문일시)
SELECT NVL(MAX(주문번호), 0) + 1, :cust_id, SYSDATE
FROM   주문;
```

- A. Read Committed에서는 미커밋 행도 보이므로 MAX+1 방식의 중복 위험이 없다.
- B. Sequence Cache를 사용하면 번호 공백과 커밋 순서 역전이 원천적으로 사라진다.
- C. RAC 환경에서는 NOCACHE ORDER 시퀀스가 항상 가장 높은 처리량을 보장한다.
- D. MAX+1은 경합과 중복 위험이 크며, 시퀀스는 공백과 순서 요구를 별도로 봐야 한다.

전체 문제 정답 모음

문제지의 보기는 A~D로 수정되었으며, 기존 1~4 번호도 함께 표기했습니다.

문항	정답(A~D)	기존 번호
1	B	2
2	C	3
3	A	1
4	D	4
5	B	2
6	A	1
7	C	3
8	D	4
9	A	1
10	D	4
11	B	2
12	C	3
13	A	1
14	B	2
15	D	4
16	C	3
17	A	1
18	D	4
19	B	2
20	C	3
21	D	4
22	A	1
23	B	2
24	C	3
25	D	4

SQLP 객관식 정답 및 해설지

정답 번호와 문항별 상세 해설

문제 1 · 정답 2번

출처: 친절한 SQL 튜닝, 책 표기 p.151-153

핵심적으로 봐야 하는 조건

반환 건수는 많지 않지만, 현재 인덱스가 찾은 ROWID마다 테이블 블록을 방문한다는 점을 봐야 한다. 화면에서 필요한 컬럼이 한정되어 있으므로 인덱스만으로 결과를 만들 수 있는지 판단하는 문제다.

정답이 되는 이유

2번이 맞다. 인덱스 리프 블록에 조건 컬럼과 출력 컬럼이 모두 있으면 테이블을 다시 방문하지 않고 결과를 만들 수 있다. 책에서는 이런 형태를 테이블 랜덤 액세스를 줄이는 대표적인 방법으로 설명한다. 엘리베이터로 비유하면, 각 집 문 앞까지 80번 오르내리는 대신 1층 우편함에서 필요한 정보를 한 번에 꺼내는 구조에 가깝다.

오답이 틀린 이유

1번은 고객 조건은 찾겠지만 ORDER_DT와 출력 컬럼 문제를 해결하지 못한다. 3번은 3천만 건 전체를 훑을 가능성이 있어 소량 조회에는 부적절하다. 4번은 기간 기준으로 넓은 범위를 읽을 수 있고, 고객별 조회라는 핵심 조건을 선두에서 처리하지 못한다.

관련 SQL 튜닝 개념

인덱스 튜닝에서는 “인덱스를 탔는가”보다 “인덱스 이후 테이블을 얼마나 많이 방문했는가”가 더 중요하다. 필요한 컬럼을 모두 인덱스에 포함시키는 Covered 인덱스는 DML 비용 증가와 저장 공간 증가를 함께 검토해야 한다.

문제 2 · 정답 3번

출처: 친절한 SQL 튜닝, 책 표기 p.185-190

핵심적으로 봐야 하는 조건

Predicate Information에서 회원등급이 access가 아니라 filter로 표시되어 있다. 즉, 인덱스에 컬럼이 존재해도 스캔 범위를 줄이는 데 쓰였는지와 읽은 뒤 걸러내는 데 쓰였는지는 다르다.

정답이 되는 이유

3번이 맞다. 결합 인덱스에서 선두 컬럼 상품구분은 등치 조건이지만, 그 다음 주문일자가 BETWEEN 범위 조건이다. 범위 조건이 나오면 뒤 컬럼인 회원등급은 스캔 시작점과 종료점을 정하는 데 제한적으로만 쓰이고, 많은 경우 리프 블록을 읽은 뒤 필터로 평가된다.

오답이 틀린 이유

1번은 “인덱스에 있다”와 “스캔 범위를 줄인다”를 혼동했다. 2번은 범위 조건 뒤 컬럼이 항상 access 조건이 된다고 잘못 본 것이다. 4번은 선두 등치 조건이 있어도 두 번째 컬럼 범위가 넓으면 리프 블록을 많이 읽을 수 있다는 점을 놓쳤다.

관련 SQL 튜닝 개념

인덱스 조건은 크게 인덱스 액세스 조건, 인덱스 필터 조건, 테이블 필터 조건으로 나뉜다. 튜닝할 때는 실행계획의 operation 뿐 아니라 predicate를 함께 봐야 한다.

문제 3 · 정답 1번

출처: 친절한 SQL 튜닝, 책 표기 p.194-202

핵심적으로 봐야 하는 조건

판매월은 1년이라는 넓은 범위이고, 판매구분은 8%만 만족하는 비교적 선택적인 등치 조건이다. 결합 인덱스에서 어느 컬럼이 먼저 오느냐에 따라 리프 블록 스캔 범위가 크게 달라진다.

정답이 되는 이유

1번이 맞다. IDX_B는 판매구분 = 'B'를 먼저 고정한 뒤 그 안에서 202401~202412 범위를 읽는다. 반면 IDX_A는 1년치 월 범위를 먼저 훑으면서 각 월 안의 여러 판매구분 중 B만 걸러야 하므로 더 많은 리프 엔트리를 읽을 수 있다.

오답이 틀린 이유

2번은 BETWEEN을 등치 조건처럼 오해했다. 3번은 컬럼이 모두 조건에 있다는 사실만 보고 실제 스캔 범위를 고려하지 않았다. 4번은 잘못이다. BETWEEN은 대표적인 Index Range Scan 조건이다.

관련 SQL 튜닝 개념

범위 조건이 선두에 놓이면 뒤 컬럼의 선택도가 스캔 범위 축소에 충분히 반영되지 못할 수 있다. BETWEEN을 IN-list로 바꾸거나 인덱스 컬럼 순서를 조정하는 방법은 데이터 분포와 입력 값 개수에 따라 판단해야 한다.

문제 4 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.158-172

핵심적으로 봐야 하는 조건

WHERE 선두 조건 게시판ID가 등치이고, 인덱스의 뒤 컬럼 순서가 ORDER BY와 같다. 또한 ROWNUM <= 10이 바깥쪽에 있어 정렬된 결과 중 앞 10건만 요구한다.

정답이 되는 이유

4번이 맞다. 인덱스 자체가 게시판별 최신순으로 정렬된 목록처럼 되어 있으므로 DBMS는 필요한 앞쪽 10건만 읽고 멈출 수 있다. 실행계획의 COUNT STOPKEY는 “필요한 개수에 도달하면 더 읽지 말라”는 신호로 이해하면 된다.

오답이 틀린 이유

1번은 ROWNUM과 ORDER BY의 위치를 일반화했다. ROWNUM을 내부에 두면 정렬 전에 10건을 잘라 잘못된 결과가 나올 수 있다. 2번은 인덱스가 정렬된 구조라는 점을 놓쳤다. 3번은 STOPKEY의 의미를 반대로 해석했다.

관련 SQL 튜닝 개념

부분범위 처리는 전체 결과를 모두 만들지 않고 사용자가 요구한 일부만 빠르게 돌려주는 방식이다. Top N 쿼리에서는 인덱스 정렬 순서, ROWNUM 위치, 실행계획의 STOPKEY를 함께 확인해야 한다.

문제 5 · 정답 2번

출처: 친절한 SQL 튜닝, 책 표기 p.240-241

핵심적으로 봐야 하는 조건

인덱스 정렬은 거주지역 다음에 혈액형, 그 다음 나이 순서다. 혈액형 IN ('A', 'O')는 논리적으로 두 개의 혈액형 구간을 읽는 것과 비슷하게 동작할 수 있다.

정답이 되는 이유

2번이 맞다. 인덱스 안에서는 서울-A-나이순, 서울-O-나이순으로 각각 정렬되어 있을 뿐, 두 혈액형을 합친 전체 결과가 나이순이라고 보장되지는 않는다. 그래서 옵티마이저는 별도의 SORT ORDER BY가 필요하다고 판단할 수 있다.

오답이 틀린 이유

1번은 IN 조건을 ORDER BY 관점에서도 완전한 등치 조건으로 본 오류다. 3번의 Sort Unique는 중복 제거 연산이며 이 상황의 핵심이 아니다. 4번은 선두 등치 조건만으로 뒤쪽 ORDER BY가 항상 해결된다고 오해한 것이다.

관련 SQL 튜닝 개념

책에서는 IN 조건을 “= 조건처럼 보일 수 있지만 항상 그런 것은 아니다”라고 주의시킨다. 특히 정렬 생략을 기대할 때는 IN-list가 결과 순서를 여러 조각으로 나누지 않는지 확인해야 한다.

문제 6 · 정답 1번

출처: 친절한 SQL 튜닝, 책 표기 p.225-229

핵심적으로 봐야 하는 조건

전화번호 컬럼에 SUBSTR 함수를 씌웠다. 일반 B-tree 인덱스는 원래 컬럼 값의 정렬 구조를 가지고 있으므로, 컬럼을 변형하면 그 정렬 구조를 바로 활용하기 어렵다.

정답이 되는 이유

1번이 맞다. 저장 포맷이 고정되어 있으면 함수 대신 원본 컬럼에 대한 범위 조건으로 바꿔 인덱스 시작점과 종료점을 찾게 만들 수 있다. “책의 가나다순 색인에서 첫 세 글자를 잘라 다시 계산하지 말고, 해당 접두어 범위를 바로 펼친다”라고 생각하면 된다.

오답이 틀린 이유

2번은 함수 기반 인덱스를 따로 만들지 않는 한 맞지 않다. 3번은 Index Full Scan도 넓게 읽으면 비효율적일 수 있다는 점을 무시한다. 4번은 가입일시 인덱스가 전화번호 함수 조건의 문제를 직접 해결하지 못한다.

관련 SQL 튜닝 개념

함수 호출 부하 자체보다 더 큰 문제는 인덱스 액세스 조건으로 쓰지 못하는 경우다. 함수 기반 인덱스도 대안이지만, 먼저 컬럼을 변형하지 않는 조건으로 바꿀 수 있는지 검토한다.

문제 7 · 정답 3번

출처: 친절한 SQL 튜닝, 책 표기 p.232-244

핵심적으로 봐야 하는 조건

가장 자주 쓰는 패턴은 고객번호 등치 + 거래일시 범위 + 최신순 Top 20이다. 상품코드는 가끔만 입력되는 선택 조건이다.

정답이 되는 이유

3번이 맞다. 고객번호로 한 고객의 구간을 먼저 좁히고, 그 안에서 거래일시 DESC 순서로 앞쪽 20건을 빠르게 읽을 수 있다. 선택 조건인 상품코드를 앞쪽에 두면 상품코드가 없는 대부분의 SQL에서 인덱스 활용성이 떨어진다.

오답이 틀린 이유

1번은 전체 고객의 최신순 구조라 고객별 조회를 좁히기 어렵다. 2번은 상품코드가 없는 요청에서 선두 컬럼을 못 쓰는 문제가 크다. 4번은 상품코드가 선택 조건이라는 사실을 무시했고, 거래일시 DESC 정렬 요구도 약해진다.

관련 SQL 튜닝 개념

인덱스 설계는 공식 하나로 끝나지 않는다. 자주 사용하는 필수 등치 조건을 앞에 두고, 범위 조건과 ORDER BY, Top N 요구, 선택 조건의 사용 빈도를 함께 봐야 한다.

문제 8 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.199-202

핵심적으로 봐야 하는 조건

선두 컬럼 성별 조건은 없지만 NDV가 2로 매우 작다. 옵티마이저는 성별 값별로 작은 하위 인덱스를 여러 번 탐색하는 방식인 Skip Scan을 선택할 수 있다.

정답이 되는 이유

4번이 맞다. Skip Scan은 선두 컬럼 값의 종류가 적을 때 “남성 구간에서 찾고, 여성 구간에서 다시 찾는” 식으로 동작한다고 이해하면 쉽다. 다만 뒤쪽 범위가 넓거나 선두 컬럼 NDV가 커지면 반복 탐색 비용이 커져 전용 인덱스보다 나빠질 수 있다.

오답이 틀린 이유

1번은 Skip Scan의 존재를 부정한다. 2번은 항상 저렴하다고 단정해 틀렸다. 3번은 LIKE가 접두어 패턴이면 인덱스 조건으로 활용될 수 있다는 점을 놓쳤다.

관련 SQL 튜닝 개념

Skip Scan은 “없는 인덱스를 임시로 흉내 내는” 성격이 강하다. 자주 수행되고 비용이 크다면 실제 업무 패턴에 맞는 결합 인덱스를 설계하는 것이 원칙이다.

문제 9 · 정답 1번

출처: 친절한 SQL 튜닝, 책 표기 p.255-264

핵심적으로 봐야 하는 조건

DEPT에서 먼저 읽는 행이 매우 적고, 그 값으로 EMP_X01을 통해 EMP를 빠르게 찾을 수 있다. NL 조인은 이런 “작은 바깥쪽 결과 + 안쪽 인덱스” 구조에서 강하다.

정답이 되는 이유

1번이 맞다. NL 조인은 바깥쪽 행을 하나씩 가져와 안쪽 테이블을 반복 조회한다. 반복 자체는 비용이지만, 반복 횟수가 작고 안쪽 조회가 인덱스로 빠르면 전체 비용이 낮다.

오답이 틀린 이유

2번은 NL 조인의 핵심 장점인 Inner 인덱스 활용을 반대로 설명했다. 3번은 Hash 조인이 대량 처리에 강하다는 사실을 모든 경우로 일반화했다. 4번은 조인 순서가 랜덤 액세스 횟수와 조인 비용을 크게 바꿀 수 있다는 점을 무시했다.

관련 SQL 튜닝 개념

NL 조인은 개발자가 중첩 for문을 돌리는 것과 비슷하다. 바깥 루프가 작고 안쪽 조회가 빠르면 좋지만, 바깥 루프가 커지면 같은 인덱스를 수만 번 두드리는 병목이 된다.

문제 10 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.270-272

핵심적으로 봐야 하는 조건

인덱스는 EMPNO 순서를 제공할 수 있지만, 실행계획에 BATCHED 테이블 액세스와 SORT ORDER BY가 같이 있다. 배치 접근은 테이블 블록 방문을 효율화하기 위해 ROWID를 묶어 처리하는 방식이다.

정답이 되는 이유

4번이 맞다. ROWID를 블록 단위로 묶어 방문하면 테이블 랜덤 I/O를 줄이는 데 도움이 되지만, 인덱스에서 읽은 키 순서 그대로 결과가 나간다고 보장하기 어렵다. 그래서 ORDER BY가 필요하면 별도 정렬이 남을 수 있다.

오답이 틀린 이유

1번은 BATCHED를 Full Scan으로 오해했다. 2번은 테이블 I/O 감소와 정렬 생략을 같은 개념으로 보았다. 3번은 BATCHED가 단일 테이블 인덱스 접근에서도 나타날 수 있다는 점을 놓쳤다.

관련 SQL 튜닝 개념

성능 최적화는 I/O 감소와 정렬 생략이 서로 충돌할 때가 있다. 옵티마이저가 어떤 이득을 더 크게 보았는지 실행계획과 실제 통계로 확인해야 한다.

문제 11 · 정답 2번

출처: 친절한 SQL 튜닝, 책 표기 p.282-296

핵심적으로 봐야 하는 조건

반복적으로 Inner 인덱스를 찾을 만한 구조가 아니고, 두 집합을 대량으로 읽어 조인한다. HASH JOIN은 보통 작은 입력을 Build Input으로 삼아 메모리에 해시 테이블을 만들고 큰 입력을 Probe한다.

정답이 되는 이유

2번이 맞다. PRODUCTS 2만 건은 ORDER_ITEMS 500만 건보다 훨씬 작으므로 Build Input 후보로 자연스럽다. 작은 쪽으로 사전을 만들고 큰 쪽을 한 줄씩 대조하는 방식이라고 이해하면 된다.

오답이 틀린 이유

1번은 Build Input을 크게 잡아 메모리와 Temp 부담을 키운다. 3번은 Sort Merge Join의 성격에 가깝다. 4번은 Hash Join이 주로 등치 조인에서 강점을 갖는다는 점과 반대다.

관련 SQL 튜닝 개념

Hash Join은 인덱스를 반복 탐색하는 대신 두 집합을 읽어 메모리 기반으로 매칭한다. Build Input이 메모리에 못 들어가면 one-pass 또는 multi-pass 처리가 발생해 Temp I/O가 증가할 수 있다.

문제 12 · 정답 3번

출처: 친절한 SQL 튜닝, 책 표기 p.274-281

핵심적으로 봐야 하는 조건

입력이 조인 키 순서로 준비될 수 있고, 결과도 그 순서가 의미 있다. 또 NL 조인처럼 Inner를 반복적으로 빠르게 찾을 조건은 부족하다.

정답이 되는 이유

3번이 맞다. Sort Merge Join은 양쪽 입력을 조인 키 순서로 맞춘 뒤 흐름을 병합하면서 조인한다. 이미 정렬되어 있거나 정렬 비용이 다른 이점으로 상쇄될 때 대량 조인에서 후보가 될 수 있다.

오답이 틀린 이유

1번은 NL 조인에 더 가까운 상황이다. 2번은 Hash Join의 Build/Probe 개념이다. 4번은 조인 조건 없이 정확한 매칭을 만들 수 없으므로 잘못이다.

관련 SQL 튜닝 개념

조인 방식은 “어느 방식이 항상 빠르다”가 아니라 데이터량, 정렬 여부, 인덱스, 메모리, 결과 순서 요구에 따라 결정된다. Sort Merge Join은 정렬 비용과 병합 이점을 함께 비교해야 한다.

문제 13 · 정답 1번

출처: 친절한 SQL 튜닝, 책 표기 p.302-310

핵심적으로 봐야 하는 조건

현재 계획은 바깥쪽 고객 행마다 주문 서브쿼리를 다시 확인하는 구조다. EXISTS는 “존재 여부”만 필요하므로 Semi Join으로 바뀔 수 있는 대표 후보가 된다.

정답이 되는 이유

1번이 맞다. Subquery Unnesting이 가능하면 상관 서브쿼리를 조인 형태로 끌어올려 NL Semi Join, Hash Semi Join 같은 방식으로 처리할 수 있다. 그러면 같은 검사를 고객 수만큼 반복하는 부담을 줄일 여지가 생긴다.

오답이 틀린 이유

2번은 EXISTS도 인덱스를 사용할 수 있다는 점을 부정한다. 3번은 Semi Join의 의미를 놓쳤다. Semi Join은 매칭 존재 여부만 보므로 고객 중복을 만들지 않는다. 4번은 특정 상황에서 no_unnest가 필요할 수는 있지만 항상 고정해야 하는 것은 아니다.

관련 SQL 튜닝 개념

서브쿼리 튜닝은 힌트 하나로 끝나지 않는다. Unnesting, Pushing, Semi Join 선택 가능성을 보고, 변환이 결과 의미를 바꾸지 않는지 확인해야 한다.

문제 14 · 정답 2번

출처: 친절한 SQL 튜닝, 책 표기 p.317-325

핵심적으로 봐야 하는 조건

바깥쪽 거래는 200만 건이지만 지점코드는 30종류뿐이다. 같은 branch_cd로 같은 스칼라 서브쿼리가 반복 호출될 가능성이 높다.

정답이 되는 이유

2번이 맞다. Oracle은 스칼라 서브쿼리 결과를 캐시해 같은 입력값에 대한 반복 조회를 줄일 수 있다. 다만 캐시는 무한하지 않고, 입력값 종류가 많거나 호출 순서가 불리하면 효과가 작아진다.

오답이 틀린 이유

1번은 스칼라 서브쿼리 캐싱을 부정했다. 3번은 상황에 따른 비용 차이를 무시한 단정이다. 4번은 Unique 여부가 캐시 비활성화의 조건이라는 설명이 아니다.

관련 SQL 튜닝 개념

스칼라 서브쿼리는 작은 코드성 테이블 조회에서 깔끔하고 빠를 수 있다. 하지만 입력값 종류가 많고 반복이 적으면 조인이나 미리 집계한 인라인 뷰가 더 안정적일 수 있다.

문제 15 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.335-341

핵심적으로 봐야 하는 조건

실행계획이 HASH GROUP BY다. 이 방식은 그룹을 만들기 위해 해시 구조를 사용하며, 결과 출력 순서를 정렬 기준으로 보장하지 않는다.

정답이 되는 이유

4번이 맞다. GROUP BY는 집계 단위를 정의하는 절이지, 최종 출력 순서를 보장하는 절이 아니다. 정렬된 결과가 업무적으로 필요하면 ORDER BY deptno, job을 명시해야 한다.

오답이 틀린 이유

1번은 과거 경험상 정렬되어 보였던 결과를 표준 동작으로 오해한 것이다. 2번은 HASH GROUP BY가 항상 디스크 정렬을 의미한다는 잘못된 설명이다. 3번은 SUM, COUNT 같은 집계도 Hash Group By로 처리될 수 있으므로 틀렸다.

관련 SQL 튜닝 개념

Sort Group By라는 이름이 보이면 정렬이 포함될 수 있지만, Hash Group By는 정렬 보장이 없다. 결과 순서가 중요하다면 실행 계획에 우연히 의존하지 말고 ORDER BY를 써야 한다.

문제 16 · 정답 3번

출처: 친절한 SQL 튜닝, 책 표기 p.343-346

핵심적으로 봐야 하는 조건

두 분기의 조건이 서로 배타적이다. 같은 행이 양쪽 분기에 동시에 나올 수 없다면 UNION의 중복 제거는 불필요한 작업이 된다.

정답이 되는 이유

3번이 맞다. UNION은 결과 중복 제거를 위해 Sort Unique 또는 Hash Unique 같은 추가 작업이 필요할 수 있다. 중복이 논리적으로 불가능하다면 UNION ALL을 사용해 그 비용을 피하는 것이 좋다.

오답이 틀린 이유

1번은 중복 가능성을 따지지 않은 과도한 일반화다. 2번은 UNION의 중복 제거 비용을 무시했다. 4번은 UNION ALL이 Sort Unique를 수행한다는 설명이 틀렸다.

관련 SQL 튜닝 개념

소트 튜닝의 첫 단계는 “정렬이나 중복 제거가 정말 필요한가”를 묻는 것이다. 단, 조건이 상호 배타적인지 NULL과 범위 조건까지 포함해 정확히 확인해야 한다.

문제 17 · 정답 1번

출처: 친절한 SQL 튜닝, 책 표기 p.352-358

핵심적으로 봐야 하는 조건

정렬을 먼저 적용한 결과 중 상위 10건을 잘라야 한다. ROWNUM을 어느 위치에서 평가하느냐가 결과 정확성과 Stopkey 가 능성에 영향을 준다.

정답이 되는 이유

1번이 맞다. 인라인 뷰 안에서 고객의 거래를 거래일시 내림차순으로 정렬하고, 바깥에서 ROWNUM <= 10을 적용한다. 적절한 인덱스가 있으면 전체 정렬 없이 앞쪽 10건만 읽는 계획도 기대할 수 있다.

오답이 틀린 이유

2번은 임의의 10건을 먼저 자른 뒤 정렬할 수 있어 최신 10건이 아니다. 3번은 HAVING 위치와 의미가 부적절하다. 4번도 내부에서 10건을 먼저 제한하므로 최신 10건을 보장하지 않는다.

관련 SQL 튜닝 개념

Top N 쿼리는 “정렬 후 자르기”가 핵심이다. SQL 모양이 비슷해 보여도 ROWNUM 위치가 바뀌면 결과 의미가 완전히 달라질 수 있다.

문제 18 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.350-351, p.381-382

핵심적으로 봐야 하는 조건

인덱스에는 상태코드와 변경일자만 있다. SQL은 변경일자 DESC 다음 처리순번 DESC까지 정렬을 요구한다.

정답이 되는 이유

4번이 맞다. 상태코드 등치 조건 아래에서 변경일자 DESC 순서는 인덱스가 제공할 수 있지만, 같은 변경일자 안에서 처리순번 DESC 순서까지 보장하려면 처리순번도 적절한 위치와 방향으로 인덱스에 있어야 한다.

오답이 틀린 이유

1번은 등치 조건이 오히려 뒤 컬럼 정렬 활용에 도움이 될 수 있다는 점을 놓쳤다. 2번은 Range Scan도 인덱스 키 순서로 읽을 수 있으므로 틀렸다. 3번은 정렬 생략에 Unique 인덱스만 필요한 것은 아니다.

관련 SQL 튜닝 개념

인덱스로 Sort Order By를 생략하려면 WHERE의 선두 조건과 ORDER BY 컬럼 순서, 정렬 방향, 표현식 일치 여부를 함께 봐야 한다.

문제 19 · 정답 2번

출처: 친절한 SQL 튜닝, 책 표기 p.366-378

핵심적으로 봐야 하는 조건

전체 이력 중 장비별 최종 1건만 필요하다. 분석함수는 편리하지만 Window Sort가 크면 Temp I/O와 PGA 사용량이 커질 수 있다.

정답이 되는 이유

2번이 맞다. 책에서는 최종 이력 조회에서 KEEP 집계, 인덱스 역순 스캔, Top N 방식 등을 상황에 맞게 검토한다. 단순히 “최신 한 건”만 필요하다면 모든 행에 순위를 매기는 구조보다 더 가벼운 접근이 가능할 수 있다.

오답이 틀린 이유

1번은 분석함수가 항상 소트를 피한다고 잘못 설명했다. 3번은 동물이 있을 때 ROW_NUMBER와 RANK의 결과가 달라질 수 있다. 4번은 MAX(변경일자)와 같은 행의 상태코드를 함께 보장하지 못한다.

관련 SQL 튜닝 개념

분석함수는 강력하지만 정렬이 숨어 있다. 필요한 결과가 “그룹별 최종 1건”인지, “상위 N건 전체”인지에 따라 KEEP, 인덱스, Top N, Window Sort 중 선택이 달라진다.

문제 20 · 정답 3번

출처: 친절한 SQL 튜닝, 책 표기 p.393-399

핵심적으로 봐야 하는 조건

대상 테이블에 인덱스가 12개 있다. INSERT는 테이블에 행만 넣는 것이 아니라 관련 인덱스 엔트리도 함께 만들어야 한다.

정답이 되는 이유

3번이 맞다. 인덱스가 많을수록 INSERT, UPDATE, DELETE 때 유지해야 할 구조가 많아진다. 대량 적재에서는 Direct Path Insert, nologging 가능성, 인덱스 unusable/drop 후 rebuild, 제약조건 검증 방식을 함께 검토한다.

오답이 틀린 이유

1번은 DML의 인덱스 유지 비용을 무시했다. 2번은 무결성 검증을 생략해도 되는지 업무적으로 확인해야 하므로 위험하다. 4번은 Bitmap 인덱스가 OLTP DML 동시성에 불리할 수 있어 일반적인 해법이 아니다.

관련 SQL 튜닝 개념

인덱스는 조회의 친구지만 DML의 짐이 될 수 있다. 특히 대량 배치에서는 “조회에 필요한 인덱스”와 “적재 중 잠시 내려놓을 수 있는 인덱스”를 구분해야 한다.

문제 21 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.436-443

핵심적으로 봐야 하는 조건

APPEND는 Direct Path Insert를 유도하지만, 그것만으로 모든 비용이 사라지는 것은 아니다. 테이블 로깅 모드, 인덱스, 제약 조건, 락 영향을 함께 봐야 한다.

정답이 되는 이유

4번이 맞다. Direct Path Insert는 버퍼 캐시를 통한 일반 경로와 달리 새 영역에 직접 적재해 대량 INSERT에 유리할 수 있다. 그러나 nologging 설정이 없으면 Redo가 발생할 수 있고, 인덱스와 제약조건 관리 비용도 별도 고려 대상이다.

오답이 틀린 이유

1번은 nologging과 APPEND를 혼동했다. 2번은 Direct Path Insert의 락과 동시성 영향을 과소평가했다. 3번은 인덱스 유지 비용이 자동으로 모두 사라진다고 잘못 설명했다.

관련 SQL 튜닝 개념

Direct Path Insert는 고속도로 진입로를 따로 쓰는 것과 비슷하다. 빠를 수 있지만, 톨게이트인 로깅, 인덱스, 제약조건을 어떻게 통과할지는 별도 설계가 필요하다.

문제 22 · 정답 1번

출처: 친절한 SQL 튜닝, 책 표기 p.457-459

핵심적으로 봐야 하는 조건

변경 대상이 한 파티션 대부분이다. 행 단위 UPDATE는 테이블 행 변경, 인덱스 변경, Undo/Redo, Lock을 대량으로 발생시킨다.

정답이 되는 이유

1번이 맞다. 변경된 데이터를 임시 테이블에 새로 만들고 필요한 인덱스를 준비한 뒤 해당 파티션과 Exchange하면 row-by-row UPDATE보다 서비스 중 영향 시간을 줄일 수 있다. 책에서는 파티션 Exchange를 대량 변경의 중요한 해법으로 다룬다.

오답이 틀린 이유

2번은 인덱스 힌트만으로 대량 DML의 본질적인 인덱스 유지와 Undo/Redo 부담을 없애지 못한다. 3번은 지나치게 파괴적이다. 4번은 인덱스 추가가 오히려 DML 유지 비용을 늘릴 수 있다.

관련 SQL 튜닝 개념

파티션 단위 대량 변경은 “행을 하나씩 고치는 방식”보다 “완성된 새 칸을 끼워 넣는 방식”이 빠를 수 있다. Exchange 시에는 로컬/글로벌 인덱스, 제약조건, validation 여부를 반드시 확인해야 한다.

문제 23 · 정답 2번

출처: 친절한 SQL 튜닝, 책 표기 p.460-462

핵심적으로 봐야 하는 조건

삭제 조건이 파티션 경계와 정확히 일치하고, 해당 파티션 전체가 삭제 대상이다. DELETE는 행마다 처리하지만 Drop/Truncate Partition은 세그먼트 또는 메타데이터 단위로 처리할 수 있다.

정답이 되는 이유

2번이 맞다. 파티션 전체를 지울 때는 DELETE보다 Drop Partition 또는 Truncate Partition이 훨씬 적은 Undo/Redo와 인덱스 유지 부담으로 끝낼 수 있다. 단, 글로벌 인덱스와 참조 무결성 영향은 별도 검토가 필요하다.

오답이 틀린 이유

1번은 DELETE의 행 단위 처리 비용을 무시했다. 3번은 인덱스 상태 관리를 해야 한다는 뜻이지 항상 금지라는 의미가 아니다. 4번은 Truncate Partition이 조건별 행 삭제가 아니라 파티션 전체 삭제라는 점을 놓쳤다.

관련 SQL 튜닝 개념

파티션 튜닝의 장점은 데이터 범위를 물리적 단위로 다룰 수 있다는 데 있다. 조건이 파티션 경계와 맞을수록 DML을 DDL성 작업으로 바꾸는 기회가 생긴다.

문제 24 · 정답 3번

출처: 친절한 SQL 튜닝, 책 표기 p.466-478

핵심적으로 봐야 하는 조건

동시 작업자가 같은 행을 잡지 않아야 하며, 잠긴 행 때문에 전체가 대기하는 것도 피해야 한다. 이때 SELECT FOR UPDATE의 대기 정책 선택이 중요하다.

정답이 되는 이유

3번이 맞다. SKIP LOCKED는 이미 잠긴 행을 기다리지 않고 건너뛴다. 작업 큐에서는 여러 작업자가 서로 다른 행을 빠르게 가져가게 해 유용하지만, 특정 시점 조회 결과만 보면 일부 행이 빠진 것처럼 보일 수 있다.

오답이 틀린 이유

1번은 NOWAIT의 의미가 반대다. NOWAIT는 잠겨 있으면 즉시 예외가 난다. 2번은 WAIT가 지정 시간 동안 기다리는 옵션이지 자동으로 skip하는 옵션은 아니다. 4번은 일반 SELECT가 DML용 행 잠금을 잡는다는 설명이 틀렸다.

관련 SQL 튜닝 개념

Lock 튜닝은 단순히 Lock을 없애는 문제가 아니라, 기다릴지, 즉시 실패할지, 건너뛸지를 업무 성격에 맞게 선택하는 문제다.

문제 25 · 정답 4번

출처: 친절한 SQL 튜닝, 책 표기 p.480-498

핵심적으로 봐야 하는 조건

여러 세션이 동시에 MAX(주문번호)+1을 계산한다. 동시에 같은 최대값을 보고 같은 다음 번호를 만들 수 있고, 이를 막으려면 직렬화나 Lock이 필요하다.

정답이 되는 이유

4번이 맞다. MAX+1은 단순히 보이지만 동시 INSERT가 많으면 중복, 대기, 인덱스 우측 블록 경합을 만들 수 있다. 시퀀스는 이런 경합을 크게 줄이는 대표적인 방법이지만, Cache를 쓰면 번호 공백이나 생성 순서와 커밋 순서의 차이를 업무적으로 받아 들일 수 있는지 판단해야 한다.

오답이 틀린 이유

1번은 Read Committed에서 다른 세션의 미커밋 행이 보이지 않는다는 기본을 틀렸다. 2번은 Cache가 성능에는 유리하지만 장애나 롤백 시 번호 공백을 없애지 못한다. 3번은 ORDER/NOCACHE가 순서 보장에는 도움이 될 수 있어도 처리량에는 불리할 수 있다.

관련 SQL 튜닝 개념

채번은 SQL 튜닝과 동시성 설계가 만나는 지점이다. 업무가 정말 “공백 없는 연속 번호”를 요구하는지, 아니면 “유일한 식별자”면 충분한지 먼저 구분해야 한다.
